

Uniwersytet Gdański
Wydział Matematyki, Fizyki i Informatyki

Dawid Kacper Odolczyk
nr albumu: 298988
kierunek studiów: informatyka

Analiza porównawcza mechanizmów
Project Loom z modelami
współbieżności w środowisku JVM

Praca magisterska
wykonana pod kierunkiem
dra Tomasza Borzyszkowskiego

Gdańsk 2026

Spis treści

Streszczenie	7
1 Wstęp	11
1.1 Motywacja	11
1.1.1 Wprowadzenie i kontekst problemu	11
1.1.2 Ograniczenia istniejących modeli współbieżności	12
1.1.3 Mechanizmy współbieżności oferowane przez Project Loom . .	12
1.2 Problem badawczy	12
1.3 Cel i zakres pracy	13
2 Wprowadzenie teoretyczne	15
2.1 Podstawowe pojęcia	15
2.1.1 Współbieżność a równoległość	15
2.1.2 Zasada działania procesów i wątków	16
2.2 Problemy występujące w systemach współbieżnych	17
2.2.1 Race condition i pamięć współdzielona	17
2.2.2 Problemy żywotności: zakleszczenie, zagłodzenie, livelock . . .	19
2.2.3 Problemy związane z wydajnością	21
2.3 Modele współbieżności w środowisku JVM	21
2.3.1 Klasyczny model współbieżności	22
2.3.2 Model współbieżności oparty na wykorzystaniu puli wątków .	22
2.3.3 Reaktywny model współbieżności	24
3 Mechanizmy współbieżności oferowane przez Project Loom	27
3.1 Wprowadzenie do Project Loom	27
3.2 Wirtualne wątki	28
3.2.1 Zasada działania i cykl życia wirtualnych wątków	29
3.2.2 Pule wirtualnych wątków	31
3.3 Współbieżność strukturalna	31
3.3.1 Cechy współbieżności strukturalnej	31
3.3.2 Struktura i działanie zakresów zadań	32

3.3.3	Strategie synchronizacji podzadań	33
3.4	Wartości zakresowe	34
3.4.1	Opis mechanizmu <code>ThreadLocal</code>	34
3.4.2	Zasada działania wartości zakresowych	35
3.5	Przegląd podobnych mechanizmów współbieżności w innych środowiskach	36
4	Przegląd literatury i opis stanu badań nad Project Loom	39
4.1	Opis i charakterystyka badań nad Project Loom	39
4.2	Przegląd wybranych pozycji	40
4.2.1	Prace akademickie	40
4.2.2	Dokumentacja techniczna	41
4.3	Zdefiniowanie luki badawczej i kierunku własnych badań	41
5	Metodyka badań	43
5.1	Cel i zakres badań	43
5.2	Hipotezy badawcze	44
5.3	Szczegółowy opis scenariuszy testowych	46
5.3.1	Scenariusz $\mathcal{A}$: Integracja z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC)	46
5.3.2	Scenariusz $\mathcal{B}$: Agregacja wyników oparta na mechanizmie kworum	51
5.4	Środowisko i konfiguracja eksperymentu	58
5.4.1	Środowisko sprzętowe i programowe	58
5.4.2	Konstrukcja i konfiguracja środowiska bazodanowego	59
5.4.3	Konfiguracja benchmarków JMH	60
5.5	Kryteria oceny wyników	61
5.5.1	Metryki wydajnościowe	61
5.5.2	Analiza skalowalności	62
5.5.3	Ocena aspektów implementacyjnych	62
6	Wyniki badań	63
6.1	Prezentacja wyników pomiarów	63
6.1.1	Wyniki pomiarów w ramach scenariusza $\mathcal{A}$	63
6.1.2	Wyniki pomiarów w ramach scenariusza $\mathcal{B}$	74
6.2	Analiza uzyskanych wyników	77
6.2.1	Scenariusz $\mathcal{A}$	77
6.2.2	Scenariusz $\mathcal{B}$	78

7	Dyskusja i wnioski	79
7.1	Interpretacja uzyskanych wyników	79
7.1.1	Scenariusz $\mathcal{A}$	79
7.1.2	Scenariusz $\mathcal{B}$	81
7.2	Ocena złożoności implementacyjnej	82
7.2.1	Model programowania	83
7.2.2	Zarządzanie błędami i zasobami	83
7.2.3	Integracja z istniejącymi mechanizmami dostępnymi w środowisku JVM	84
7.3	Weryfikacja hipotez badawczych	85
7.4	Rekomendacje i wzorce zastosowań mechanizmów Project Loom . . .	88
7.4.1	Zastosowania wirtualnych wątków	88
7.4.2	Zastosowania współbieżności strukturalnej	89
7.5	Ograniczenia badań	90
7.6	Kierunki dalszych badań	91
8	Podsumowanie	93
	Bibliografia	95
	Spis rysunków	101
	Spis tabel	103
	Spis kodów źródłowych	104

Streszczenie

Praca przedstawia analizę porównawczą i jakościową mechanizmów Project Loom – wirtualnych wątków oraz współbieżności strukturalnej – z istniejącymi modelami współbieżności w środowisku JVM, przeprowadzoną w trybie benchmarków przy użyciu narzędzia Java Microbenchmark Harness (JMH) na platformie JDK 25. W scenariuszu opartym na integracji z bazą danych wirtualne wątki z blokującym sterownikiem JDBC osiągnęły wydajność porównywalną z wariantem reaktywnym na wszystkich badanych poziomach współbieżności, wyraźnie przewyższając wariant oparty na wątkach platformowych. Wariant reaktywny wykazał natomiast istotny wzrost opóźnień przy złożonym zapytaniu agregującym i wysokiej współbieżności. W scenariuszu równoległej agregacji opartej na mechanizmie kworum mechanizm współbieżności strukturalnej osiągnął wyniki porównywalne z podejściem reaktywnym, przy czym oba warianty przewyższyły natywne rozwiązanie, którego ograniczona skuteczność anulowania zadań negatywnie wpływała na czas odpowiedzi przy skrajnych opóźnieniach.

Jakościowa analiza implementacji wykazała, że migracja z wątków platformowych na wirtualne wymaga minimalnych zmian w kodzie, bez modyfikacji logiki biznesowej. Mechanizm współbieżności strukturalnej upraszcza implementację wzorców równoległej agregacji przy zachowaniu imperatywnego modelu programowania. Model reaktywny, choć wydajny, wiąże się z wyraźnie wyższą złożonością poznawczą i trudniejszą diagnostyką błędów.

Na podstawie przeprowadzonych badań potwierdzono hipotezę główną pracy: mechanizmy Project Loom umożliwiają osiągnięcie wydajności porównywalnej lub wyższej względem istniejących modeli współbieżności przy jednoczesnym zachowaniu prostszego, imperatywnego modelu programowania.

Słowa kluczowe: JVM, współbieżność, Project Loom, wirtualne wątki, współbieżność strukturalna, modele współbieżności, programowanie reaktywne, benchmark, JMH

Abstract

“Comparative analysis of Project Loom mechanisms and concurrency models in the JVM environment”

This thesis presents a comparative and qualitative analysis of Project Loom mechanisms – virtual threads and structured concurrency – against existing JVM concurrency models, conducted in benchmark mode using the Java Microbenchmark Harness (JMH) tool on JDK 25. In the database integration scenario, virtual threads with a blocking JDBC driver achieved performance comparable to the reactive variant across all tested concurrency levels, while significantly outperforming the platform-thread variant. However, the reactive variant exhibited a substantial increase in latency for a complex aggregating query under high concurrency. In the quorum-based parallel aggregation scenario, the structured concurrency mechanism matched the reactive approach, with both outperforming the native variant, whose limited task cancellation effectiveness degraded response times under extreme latency conditions.

A qualitative analysis showed that migrating from platform threads to virtual threads requires minimal code changes with no modifications to business logic. Structured concurrency simplifies parallel aggregation patterns while preserving the imperative programming model. The reactive model, although performant, entails significantly higher cognitive complexity and harder error diagnostics.

The main research hypothesis was confirmed: Project Loom mechanisms enable performance comparable to or higher than existing concurrency models while preserving a simpler, imperative programming model.

Keywords: JVM, concurrency, Project Loom, virtual threads, structured concurrency, reactive programming, benchmarks, JMH

Rozdział 1

Wstęp

Pierwszy rozdział pracy wprowadza w tematykę niniejszej pracy, przedstawiając kontekst i motywację podjętych badań, w szczególności ograniczenia istniejących modeli współbieżności dostępnych w środowisku JVM oraz możliwości, jakie w odpowiedzi na te ograniczenia oferują mechanizmy Project Loom. Rozdział formułuje również problem badawczy stanowiący fundament pracy, a także określa jej cel i zakres.

1.1 Motywacja

1.1.1 Wprowadzenie i kontekst problemu

Współczesne aplikacje serwerowe muszą obsługiwać tysiące lub miliony równoczesnych żądań, efektywnie zarządzając przy tym zasobami systemowymi. Środowisko JVM¹, na którym opierają się systemy budowane w językach Java, Kotlin czy Scala, stanowi jedną z najszerzej stosowanych platform do tworzenia oprogramowania po stronie serwera [2]. Kwestia doboru właściwego modelu współbieżności od dawna stanowi jedno z kluczowych wyzwań inżynierii oprogramowania w tym środowisku [3, 4].

Na przestrzeni lat w środowisku JVM wykształciło się wiele modeli obsługi współbieżności – od prostego podejścia opartego na wątkach systemu operacyjnego, przez pule wątków, aż po reaktywny model programowania asynchronicznego. Każde z tych podejść stanowiło odpowiedź na ograniczenia poprzedniego, lecz jednocześnie wprowadzało nowe kompromisy w zakresie wydajności, skalowalności lub złożoności implementacyjnej [4, 5].

¹JVM (*Java Virtual Machine*) to środowisko uruchomieniowe umożliwiające wykonywanie programów napisanych w językach platformy Java niezależnie od systemu operacyjnego i architektury sprzętowej [1].

1.1.2 Ograniczenia istniejących modeli współbieżności

Modele oparte na wątkach systemu operacyjnego nie skalują się dobrze w warunkach wysokiej współbieżności. Każdy taki wątek jest zasobożerny, a wykonywanie przez niego operacji blokującej wiąże się z zajmowaniem zasobów systemowych bez efektywnego wykonywania pracy [3, 6]. Programowanie reaktywne rozwiązuje ten problem, umożliwiając obsługę wielu żądań przy użyciu niewielkiej liczby wątków [7]. Osiąga to jednak kosztem wyraźnego wzrostu złożoności. Programowanie reaktywne wiąże się z użyciem paradygmatu innego niż model sekwencyjny, a diagnostyka błędów staje się utrudniona ze względu na asynchroniczny charakter wykonania [5, 8].

1.1.3 Mechanizmy współbieżności oferowane przez Project Loom

Project Loom, zainicjowany przez OpenJDK pod koniec 2017 roku, wprowadza do środowiska JVM zestaw mechanizmów mających pogodzić wysoką skalowalność z zachowaniem prostego, imperatywnego modelu programowania [9]. Jednym z nich jest mechanizm wirtualnych wątków (ang. *virtual threads*), czyli lekkich wątków zarządzanych przez JVM, które podczas operacji blokujących nie blokują bezpośrednio wątku systemowego, co pozwala obsłużyć bardzo dużą liczbę równoległych żądań przy minimalnym zużyciu zasobów [10, 11].

Project Loom obejmuje również mechanizm współbieżności strukturalnej (ang. *structured concurrency*), który upraszcza zarządzanie cyklem życia równoległych zadań i propagację błędów oraz wartości zakresowe (ang. *scoped values*), stanowiące bezpieczną alternatywę dla przekazywania danych kontekstowych w środowiskach wysokiej współbieżności [12, 13].

1.2 Problem badawczy

Wprowadzenie mechanizmów Project Loom do platformy JVM rodzi pytanie o zakres, w jakim zastępują lub uzupełniają one istniejące modele współbieżności. Z jednej strony wirtualne wątki oferują prostotę modelu imperatywnego przy potencjalnie porównywalnej wydajności z podejściem reaktywnym. Z drugiej strony reaktywne modele są rozwijane i optymalizowane od wielu lat, a ich charakterystyka wydajnościowa w warunkach wysokiej intensywności jest przedmiotem licznych badań [8, 14, 15]. Niniejsza praca podejmuje problem porównania wydajności i złożoności implementacyjnej mechanizmów Project Loom, w szczególności wirtualnych wątków oraz współbieżności strukturalnej, z istniejącymi modelami współbieżności w środowisku JVM: modelem opartym na wątkach platformowych,

modelem opartym na puli wątków oraz reaktywnym modelem programowania. Badanie koncentruje się na odpowiedzi na pytanie, czy mechanizmy Project Loom umożliwiają osiągnięcie wydajności porównywalnej z podejściem reaktywnym przy jednoczesnym zachowaniu prostszego, imperatywnego modelu programowania.

1.3 Cel i zakres pracy

Celem niniejszej pracy jest przeprowadzenie analizy porównawczej i integracyjnej mechanizmów oferowanych przez Project Loom z istniejącymi modelami współbieżności w środowisku JVM. Zakres badań obejmuje dwa scenariusze testowe: integrację z bazą danych przy użyciu sterownika blokującego i reaktywnego oraz agregację wyników opartą na mechanizmie kworum, a także jakościową ocenę złożoności implementacyjnej porównywanych wariantów. Badania przeprowadzono na platformie JVM przy użyciu JDK w wersji 25 przy użyciu narzędzia JMH (Java Microbenchmark Harness). Poza oceną wydajnościową praca obejmuje analizę złożoności implementacyjnej badanych podejść w zakresie modelu programowania, zarządzania błędami i zasobami oraz stopnia integracji z istniejącymi mechanizmami obecnymi w środowisku JVM.

Rozdział 2

Wprowadzenie teoretyczne

Drugi rozdział pracy poświęcony jest teoretycznemu omówieniu podstawowych pojęć związanych z równoległością i współbieżnością, obejmującemu opis działania wątków we współczesnych systemach operacyjnych, zdefiniowanie typowych problemów występujących w systemach współbieżnych oraz przedstawienie najważniejszych założeń różnych modeli współbieżności dostępnych w ramach środowiska JVM, w szczególności klasycznego modelu współbieżności, modelu opartego na wykorzystaniu puli wątków oraz reaktywnego modelu współbieżności.

2.1 Podstawowe pojęcia

2.1.1 Współbieżność a równoległość

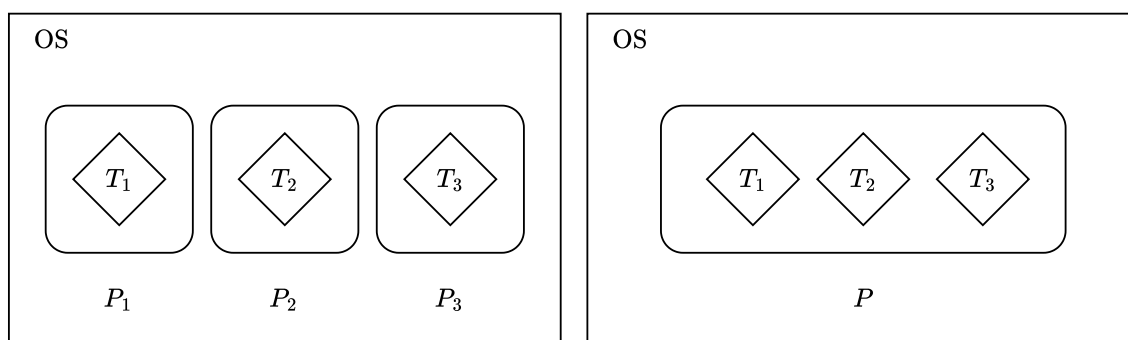
Jak podają Tanenbaum i Bos [3], *współbieżność* (ang. *concurrency*) oznacza zdolność systemu do obsługi wielu zadań w taki sposób, że ich wykonywanie nakłada się w czasie, nawet jeśli w danym momencie wykonywane jest tylko jedno z nich. *Równoległość* (ang. *parallelism*) oznacza obsługę wielu zadań w tym samym czasie przy wykorzystaniu wielu jednostek obliczeniowych.

Innymi słowy, współbieżność odnosi się do strukturalnej organizacji systemu oraz sposobu modelowania i zarządzania wieloma zadaniami, natomiast równoległość oznacza ich faktyczne, jednoczesne wykonywanie na wielu procesorach lub rdzeniach. We współczesnych systemach operacyjnych zadania te są wykonywane w ramach *procesów i wątków*.

2.1.2 Zasada działania procesów i wątków

We współczesnych systemach operacyjnych najważniejszą abstrakcją umożliwiającą współbieżne wykonywanie operacji są *procesy* (ang. *processes*). Jak wskazują Tanenbaum i Bos [3], procesy umożliwiają przekształcenie jednej jednostki obliczeniowej (CPU) na wiele wirtualnych jednostek, w ramach których mogą współbieżnie wykonywać zadania poprzez umiejętność CPU do szybkiego przełączania się między procesami.

W celu umożliwienia bardziej efektywnej współbieżności wewnątrz pojedynczego procesu, współczesne systemy operacyjne oferują *wątki* (ang. *threads*), definiowane jako lżejsze jednostki wykonania. Wątki działają w obrębie jednego procesu i współdzielą z nim przestrzeń adresową oraz zasoby, zachowując jednocześnie własny kontekst wykonania [3]. Różnice między procesem a wątkiem prezentuje rysunek 2.1.



(a) trzy jednowątkowe procesy

(b) jeden trzywątkowy proces

Rysunek 2.1: Rozróżnienie procesów i wątków w systemie operacyjnym

(źródło: opracowanie własne na podstawie [3])

Mimo że oba systemy przedstawione powyżej operują na trzech wątkach, wątki na rysunku 2.1a korzystają z odrębnych przestrzeni adresowych, podczas gdy wątki przedstawione na rysunku 2.1b działają w obrębie jednej (wspólnej) przestrzeni adresowej, co wynika z faktu operowania wewnątrz jednego (wspólnego) procesu P .

Wprowadzenie wątków jako lekkich jednostek wykonania znacząco zwiększa efektywność wykorzystania zasobów systemowych, jednak jednocześnie wprowadza nowe wyzwania związane z koniecznością synchronizacji dostępu do współdzielonych danych. Współdzielenie przestrzeni adresowej przez wątki powoduje ryzyko występowania różnego rodzaju zjawisk, opisanych szczegółowo w dalszej części. Z tego względu poprawne projektowanie systemów wielowątkowych wymaga nie tylko zrozumienia modelu wykonania wątków, ale również potencjalnych problemów wynikających z ich współbieżnej natury.

2.2 Problemy występujące w systemach współbieżnych

Mimo wielu zalet wykorzystania wątków do współbieżnego wykonywania zadań, podejście to wiąże się z istnieniem wielu problemów obecnych w systemach współbieżnych. Ben-Ari [16] wskazuje zasadę *wzajemnego wykluczenia* (ang. *mutual exclusion*) jako jeden z najważniejszych elementów programowania współbieżnego z racji bycia abstrakcją dla wielu problemów związanych z zarządzaniem wątkami. Zgodnie z definicją Herlihy'ego i Shavita [17], wątki *wykluczają się wzajemnie*, jeśli co najmniej dwa z nich próbują równocześnie wykonać wspólny blok kodu, nazywany *sekcją krytyczną* (ang. *critical section*), ale w danym momencie tylko jeden z nich ma do niej dostęp. Tę cechę systemu określa się również mianem *synchronizacji* wątków [18]. Oznacza to, że dany wątek ma dostęp do sekcji krytycznej tylko wtedy, gdy inny wątek zakończy w niej swoje operacje.

Zapewnienie tej cechy jest istotne w kontekście poprawności i jednoznaczności wyniku wielowątkowego zadania, a jej brak powoduje występowanie wielu kategorii problemów. Wśród nich można wyróżnić: problem *race condition* i pamięci współdzielonej, problemy żywotności, takie jak *zakleszczenie* (ang. *deadlock*), *zagłodzenie* (ang. *starvation*) i *livelock*, a także problemy związane z wydajnością. Poniżej opisano charakterystykę każdego z nich.

2.2.1 Race condition i pamięć współdzielona

Goetz [4] definiuje problem *race condition* (w polskiej literaturze często określany mianem *warunku wyścigu* lub *stanu wyścigu* [19, 20]) jako zachowanie systemu współbieżnego, w którym poprawność obliczeń zależy od względnego czasu lub przeplatania wielu wątków przez środowisko wykonawcze. Innymi słowy, problem ten występuje, gdy uzyskanie oczekiwanego rezultatu jest zależne od zbiegu okoliczności. Najczęstszym rodzajem tego problemu jest sytuacja, w której potencjalnie nieaktualna wartość jest wykorzystywana do dalszych obliczeń, co może prowadzić do niezamierzonych rezultatów. Poniższy przykład szczegółowo prezentuje to zjawisko.

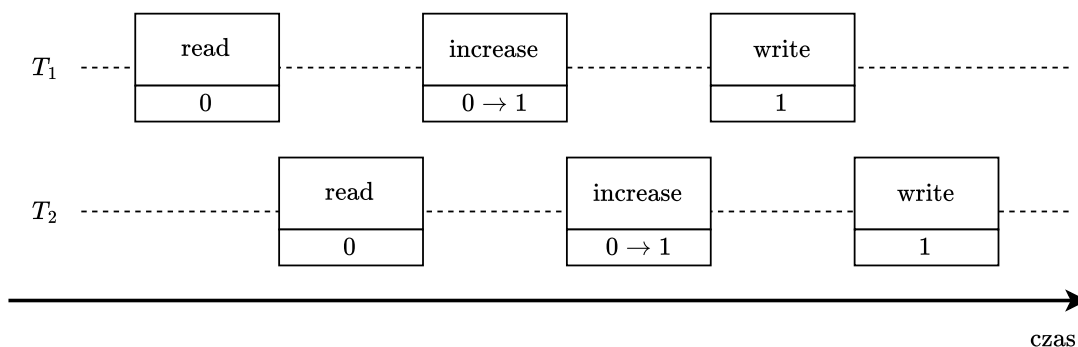
Niech procedura `increaseByOne` składa się z trzech wykonywanych sekwencyjnie operacji:

1. odczytania wartości pewnej zmiennej globalnej (ozn. `read`),
2. zwiększenia odczytanej wartości o 1 (ozn. `increase`),
3. zapisania nowej wartości do zmiennej globalnej (ozn. `write`).

Procedurę tę uruchomiono współbieżnie z wykorzystaniem wątków T_1 i T_2 dla tej samej wartości początkowej, tj. pewnej zmiennej globalnej G o wartości początkowej

równej 0. Oczekiwanym rezultatem dwukrotnego wywołania tej procedury jest zwiększenie wartości zmiennej G do wartości równej 2.

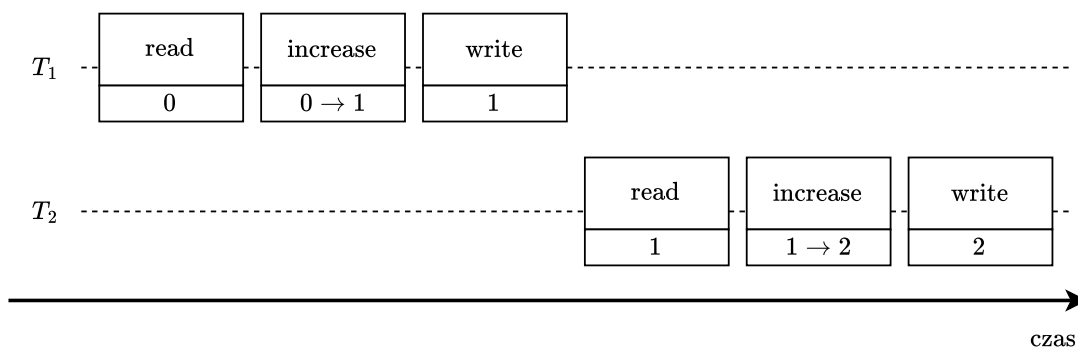
Rysunki 2.2 i 2.3 przedstawiają różne modele wywołań procedury `increaseByOne` na wątkach T_1 i T_2 . Prezentują one wywołania poszczególnych operacji tej procedury dla poszczególnych wątków w czasie. Każda operacja zawiera dwie informacje: nazwę oraz aktualną wartość zmiennej globalnej G . Wyjątek stanowi operacja `increase`, dla wartości której przyjęto oznaczenie „ $A \rightarrow B$ ”, gdzie A oznacza wartość zmiennej G przed dokonaniem na niej operacji zwiększenia wartości o 1 (tj. jej aktualna wartość), a B oznacza wartość zmiennej G po wykonaniu na niej operacji.



Rysunek 2.2: Ilustracja zjawiska *race condition* na podstawie wywołania procedury `increaseByOne`

(źródło: opracowanie własne na podstawie [4])

Rysunek 2.2 prezentuje jeden z przypadków wystąpienia zjawiska *race condition*, w którym poszczególne operacje procedury `increaseByOne` przeplatają się w czasie. W rezultacie wątek T_2 odczytał wartość zmiennej globalnej G w momencie, gdy nie została ona jeszcze zaktualizowana przez wątek T_1 , co w konsekwencji spowodowało, że wartość zmiennej G wynosi 1 zamiast przewidywanej wartości 2. Rozwiązaniem tego problemu jest wcześniej wspomniana synchronizacja wątków T_1 i T_2 .



Rysunek 2.3: Wywołanie procedury `increaseByOne` z wykorzystaniem synchronizacji wątków

(źródło: opracowanie własne na podstawie [4])

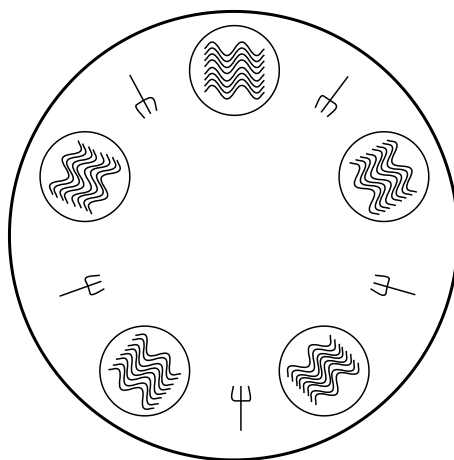
Rysunek 2.3 przedstawia wywołanie procedury `increaseByOne` na obu zsynchronizowanych wątkach, tj. wątek T_2 wykonał przydzielone mu operacje dokładnie wtedy, gdy wątek T_1 wykonał wszystkie swoje operacje. W rezultacie wartość zmiennej globalnej G wynosi 2, ponieważ procedura wywołana w wątku T_1 zwiększyła wartość zmiennej G o 1, a następnie to samo wykonała procedura wywołana w wątku T_2 , wykorzystując wynik operacji wykonanych przez wątek T_1 .

2.2.2 Problemy żywotności: zakleszczenie, zagłodzenie, livelock

Goetz [4] definiuje *żywotność* (ang. *liveness*) jako właściwość systemu, która zapewnia, że koniec końców pewna własność się wydarzy, czyli że system nie będzie pozostawał w stanie zawieszenia lub bezczynności. Problemy żywotności występują, gdy system nie jest w stanie kontynuować wykonywania zadań z powodu wzajemnych zależności między wątkami lub braku zasobów. Najpopularniejszymi z nich są *zakleszczenie*, *zagłodzenie* i *livelock*.

Problem uczających filozofów

Klasyczną ilustracją problemów żywotności, często przytaczaną w literaturze, jest wymyślony w 1965 roku przez Edsgera Dijkstrę *problem uczających filozofów* (ang. *dining philosophers problem*) [21]. Opisuje on pięciu filozofów siedzących przy okrągłym stole. Każdy filozof ma przed sobą talerz z posiłkiem, a między każdą sąsiadującą parą talerzy znajduje się widelec. Filozof może wykonywać jedną z dwóch czynności: albo je, albo rozmyśla. Zakłada się, że filozof może jeść tylko przy użyciu dwóch widelców. Dodatkowo, filozof może korzystać jedynie z widelców sąsiadujących z jego talerzem [3, 16].



Rysunek 2.4: Ilustracja problemu uczających filozofów

(źródło: opracowanie własne na podstawie [3])

W powyższym problemie poszczególne jego elementy stanowią abstrakcyjne reprezentacje składników systemu współbieżnego, tj. filozofowie odpowiadają procesom lub wątkom wykonującym się równolegle, które cyklicznie przechodzą pomiędzy sekcją niekrytyczną (rozmyślanie) a sekcją krytyczną (jedzenie), zaś widelce reprezentują współdzielone zasoby, do których dostęp musi być synchronizowany.

Zakleszczenie

Zakleszczeniem (ang. *deadlock*) określa się sytuację w systemie współbieżnym, w której co najmniej dwa procesy lub wątki czekają na siebie nawzajem w taki sposób, że żaden z nich nie może kontynuować wykonania [4].

Przykładem zakleszczenia w powyższym problemie jest sytuacja, w której każdy filozof bierze do jednej ręki widelec po lewej stronie swojego talerza. Aby móc zacząć jeść, filozof potrzebuje drugiego widelca, lecz każdy z nich nie posiada wolnego widelca po prawej stronie swojego talerza. Powoduje to wzajemną zależność czekania na zwolnienie zasobu, a w konsekwencji żaden z filozofów nie przystąpi do jedzenia.

Zagłodzenie

Zagłodzeniem (ang. *starvation*) wątku nazywa się sytuację, w której wątek jest stale pozbawiony dostępu do wspólnego zasobu, przez co nie może wykonać przydzielonych mu operacji [4].

Przykładem zagłodzenia w kontekście problemu uczujących filozofów jest sytuacja, w której jeden filozof nigdy nie może przystąpić do jedzenia, ponieważ sąsiadujący z nim filozofowie zawsze wygrywają w walce o wspólny zasób, jakim jest widelec (zarówno ten po lewej, jak i po prawej stronie talerza).

Livelock

Livelock jest stanem systemu współbieżnego, w którym procesy nie są zablokowane i wykonują operacje, jednak wskutek ciągłych wzajemnych reakcji nie dochodzi do postępu w realizacji ich zadań [4].

Livelock w kontekście wspomnianego wyżej problemu występuje, gdy filozofowie, próbując uniknąć zakleszczenia, zwalniają posiadane zasoby w przypadku niepowodzenia i ponawiają próby w sposób symetryczny, co prowadzi do cyklicznego wzorca zachowań. W rezultacie system pozostaje aktywny, jednak żaden z procesów nie uzyskuje jednocześnie wymaganych zasobów, przez co nie dochodzi do postępu.

2.2.3 Problemy związane z wydajnością

Goetz [4] wskazuje, że równie ważną właściwością systemu współbieżnego jak żywotność jest *wydajność* (ang. *performance*). Podczas gdy żywotność zapewnia, że koniec końców pewna własność się wydarzy, pojęcie wydajności obejmuje szerszy zakres – uwzględnia ono wykonywanie jak największej liczby zadań przy wykorzystaniu zasobów na jak najmniejszym poziomie. Problemy związane z wydajnością obejmują szeroki zakres problemów, w tym krótki czas obsługi, responsywność, przepustowość, zużycie zasobów czy skalowalność. Podobnie jak w przypadku bezpieczeństwa i żywotności, programy wielowątkowe są narażone na wszystkie zagrożenia wydajnościowe programów jednowątkowych, a także na inne, które wynikają z użycia wątków. Jednym z najistotniejszych problemów związanych z wydajnością jest problem nadmiernego przełączania kontekstu.

Przełączanie kontekstu

Mianem *przełączania kontekstu* (ang. *context switching*) określa się mechanizm systemowy polegający na wstrzymaniu wykonywania jednego wątku oraz przywróceniu stanu innego wątku, umożliwiając współdzielenie zasobów procesora pomiędzy wieloma jednostkami wykonawczymi. Występuje on w sytuacji, gdy liczba wątków możliwych do uruchomienia jest większa niż liczba dostępnych procesorów logicznych. [4]

Nadmierna liczba przełączeń kontekstu prowadzi do istotnego obniżenia wydajności systemu. Wynika to z faktu, że czas procesora jest wtedy zużywany na operacje związane z zarządzaniem tym mechanizmem zamiast na wykonywanie właściwych obliczeń. Dodatkowo, jak podają Mogul i Borg [6], częste przełączanie kontekstu negatywnie wpływa na efektywność pamięci podręcznej procesora, powodując utratę lokalności danych i zwiększenie opóźnień dostępu do pamięci.

2.3 Modele współbieżności w środowisku JVM

Środowisko JVM oferuje wiele różnych modeli umożliwiających tworzenie współbieżnych aplikacji. Ta część poświęcona jest omówieniu najpopularniejszych podejść, w tym podstawowego modelu, będącego częścią Thread API, modelu opartego na wykorzystaniu puli wątków, dostępnego w ramach interfejsu `ExecutorService`, a także modelu reaktywnego.

2.3.1 Klasyczny model współbieżności

Podstawowym modelem współbieżności w środowisku JVM są klasa `Thread` i interfejs `Runnable`, będące częścią pakietu `java.lang` i oferujące podstawowe metody umożliwiające zarządzanie wątkami. Interfejs `Runnable` reprezentuje zadanie przeznaczone do współbieżnego wykonania, natomiast klasa `Thread` odpowiada za jego uruchomienie i wykonanie [22, 23].

Klasa `Thread` jest klasą opakowującą (ang. *wrapper class*) wątki systemu operacyjnego w relacji jeden do jednego. Oznacza to, że każdy obiekt tej klasy odpowiada dokładnie jednemu wątkowi systemowemu, nazywanemu *wątkiem platformowym* (ang. *platform thread*) [24]. Zarządzanie cyklem życia i przełączanie kontekstu realizowane jest przez system operacyjny, a nie przez środowisko JVM. Konsekwencją takiego podejścia jest stosunkowo wysoki koszt tworzenia wątków, zarówno pod względem czasu, jak i zużycia pamięci, co ogranicza ich skalowalność w aplikacjach wymagających obsługi dużej liczby współbieżnych zadań [11].

Jak wskazuje Lea [25], model ten opiera się na współdzielonej pamięci, co oznacza, że wątki mogą uzyskiwać dostęp do tych samych danych. W celu zapewnienia poprawności działania programu konieczne jest stosowanie mechanizmów synchronizacji. Przykładem jest obecne w środowisku JVM słowo kluczowe `synchronized` [26], które umożliwia wzajemne wykluczanie dostępu do sekcji krytycznej.

Pomimo swojej prostoty i bezpośredniego odwzorowania na mechanizmy systemu operacyjnego, model ten posiada istotne ograniczenia. Wysoki koszt tworzenia i zarządzania wątkami oraz podatność na problemy współbieżności, o których wspomniano w podrozdziale 2.2, sprawiają, że w praktyce często stosuje się bardziej zaawansowane modele, opisane niżej.

2.3.2 Model współbieżności oparty na wykorzystaniu puli wątków

Jednym z rozwiązań wspomnianego wyżej problemu było wprowadzenie w ramach JDK 1.5 interfejsu `ExecutorService` [27]. Jego głównym celem jest uproszczenie zarządzania współbieżnością poprzez uniezależnienie definicji zadań od ich wykonania, stanowiąc bardziej elastyczną alternatywę dla bezpośredniego użycia klasy `Thread` i interfejsu `Runnable`.

Interfejs `ExecutorService` w dużej mierze opiera się o wykorzystanie wzorca projektowego, jakim jest *Pula obiektów* (ang. *Object Pool*) [28]. W tym celu interfejs wykorzystuje pulę wątków. Takie podejście pozwala na wielokrotne wykorzystanie tych samych wątków, eliminując konieczność ich częstego tworzenia i niszczenia, co w klasycznym modelu stanowi istotne obciążenie czasowe i pamięciowe [11].

Podejście to jednak nie jest pozbawione ograniczeń. Jednym z nich jest problem odpowiedniego doboru rozmiaru puli wątków.

Określanie rozmiaru puli wątków

Rozmiar puli wątków definiuje się jako liczbę wątków zarządzanych przez mechanizm wykonawczy, które mogą być wykorzystane do równoczesnego wykonywania zadań [29]. Zdaniem Goetza [4], liczba ta powinna być wyznaczana dynamicznie, przykładowo na podstawie liczby dostępnych procesorów logicznych. W środowisku JVM wartość ta jest możliwa do wyznaczenia przy użyciu metody `availableProcessors`, dostępnej w ramach klasy `Runtime` [30].

Goetz wskazuje również, że przy ustalaniu rozmiaru puli wątków należy unikać wartości ekstremalnych. Jeśli pula wątków jest za duża, wątki mogą ze sobą konkurować o ograniczone zasoby procesora i pamięci, co może skutkować większym użyciem pamięci i możliwym wyczerpaniem zasobów, zaś zbyt mała pula wątków może ograniczać przepustowość poprzez niewykorzystanie procesorów logicznych pomimo dostępności zadań do wykonania. Ustalenie właściwego rozmiaru puli wątków zależy między innymi od dostępności zasobów i natury współbieżnie wykonywanych zadań. Goetz podaje, że optymalny rozmiar N_T puli wątków można wyrazić wzorem

$$N_T = N_{\text{CPU}} \cdot U_{\text{CPU}} \cdot \left(1 + \frac{T_W}{T_C}\right),$$

gdzie:

- N_{CPU} oznacza liczbę dostępnych procesorów logicznych,
- U_{CPU} jest wartością docelowego wykorzystania procesora, gdzie $0 \leq U_{\text{CPU}} \leq 1$,
- T_W (ang. *wait time*) reprezentuje średni czas oczekiwania wątku na możliwość kontynuowania pracy, obejmujący między innymi czas oczekiwania na operacje wejścia-wyjścia, dostęp do zasobów współdzielonych lub synchronizację z innymi wątkami,
- T_C (ang. *compute time*) oznacza średni czas, w którym wątek aktywnie wykonuje operacje obliczeniowe, wykorzystując procesor.

Stosunek $\frac{T_W}{T_C}$ odzwierciedla charakter zadań wykonywanych w ramach puli wątków. Niskie wartości tego ilorazu wskazują na zadania obliczeniowe, w których dominującą rolę odgrywa czas przetwarzania (ang. *CPU-bound*), natomiast wysokie wartości są charakterystyczne dla zadań wymagających częstego oczekiwania, takich jak operacje wejścia-wyjścia (ang. *I/O-bound*).

2.3.3 Reaktywny model współbieżności

W odpowiedzi na ograniczenia klasycznych modeli współbieżności opartych na blokujących operacjach i bezpośrednim zarządzaniu wątkami, rozwinięto model reaktywny, którego celem jest efektywne przetwarzanie danych w sposób asynchroniczny i nieblokujący. Jednym z kluczowych standardów tego podejścia jest Reactive Streams [31], którego twórcy na oficjalnej stronie projektu podają:

*Reactive Streams to inicjatywa mająca na celu zapewnienie standardu asynchronicznego przetwarzania strumieniowego z wykorzystaniem nieblokującego mechanizmu kontroli przepływu. Obejmuje ona działania skierowane zarówno do środowisk uruchomieniowych [...], jak również dla protokołów sieciowych.*¹

W środowisku JVM model reaktywny obejmuje zestaw wzorców i mechanizmów umożliwiających asynchroniczne, zdarzeniowe oraz nieblokujące przetwarzanie danych z wykorzystaniem strumieni. W praktyce jego realizacja opiera się na specyfikacji Reactive Streams oraz bibliotekach takich jak Project Reactor [32], RxJava [33] czy Akka Streams [34], które umożliwiają tworzenie skalowalnych aplikacji współbieżnych.

Podstawowe pojęcia związane z programowaniem reaktywnym

Jak podają Dokuka i Lozynski [35], przekazywanie informacji w modelu reaktywnym odbywa się w sposób *asynchroniczny*, tj. operacje nie są wykonywane w sposób blokujący, a ich wynik nie jest dostępny natychmiast po wywołaniu, jak w modelu klasycznym. Taki system oparty jest na przekazywaniu *komunikatów* (ang. *messages*) za pomocą *strumieni danych* (ang. *data streams*). Dane te, w zależności od roli, mogą być wysyłane i odbierane przez specjalnie zdefiniowane komponenty. Dokumentacja Reactive Streams [31] wskazuje, że w reaktywnym modelu współbieżności wyróżnia się cztery podstawowe role:

- *producent* (ang. *producer*, *publisher*), który publikuje komunikaty,
- *odbiorca* (ang. *receiver*, *subscriber*), który otrzymuje komunikaty od producenta,
- *procesor* (ang. *processor*), który pełni rolę zarówno producenta, jak i odbiorcy,
- *subskrypcja* (ang. *subscription*), która jest mechanizmem zarządzającym relacją między producentem a odbiorcą.

¹Tłumaczenie własne na podstawie [31].

W środowisku JVM role te są zaimplementowane odpowiednio w ramach interfejsów `Publisher`, `Subscriber`, `Processor` i `Subscription`, będących składowymi klasy `java.util.concurrent.Flow` [36].

Model reaktywny jest silnie powiązany ze wzorcem projektowym *Obserwator* (ang. *Observer*) [37]. W obu przypadkach komunikacja pomiędzy komponentami opiera się na mechanizmie publikowania i subskrypcji zdarzeń. Kluczową różnicą jest jednak obecność mechanizmu *backpressure*, który umożliwia odbiorcy kontrolę tempa napływu danych, czego klasyczna definicja tego wzorca nie zapewnia.

Cechy modelu reaktywnego

Autorzy dokumentu *The Reactive Manifesto* [7], stanowiącego zbiór zasad i wytycznych dotyczących projektowania systemów reaktywnych, podają następujące cechy systemu opartego na reaktywnym modelu przetwarzania danych:

- *responsywność* – zapewnienie szybkich i spójnych czasów reakcji celem zapewnienia jak najwyższej jakości usług,
- *odporność* – zapewnienie responsywności systemu w przypadku wystąpienia awarii poprzez hermetyzację i izolację przyczyny awarii od reszty systemu, a także replikację i delegowanie przetwarzania do innych, zewnętrznych komponentów,
- *elastyczność* – zapewnienie responsywności przy zmiennym obciążeniu systemu poprzez zastosowanie różnych algorytmów skalowania,
- *zarządzanie za pomocą komunikatów* (ang. *message-driven*) – zapewnienie komunikacji między komponentami poprzez asynchroniczne przekazywanie komunikatów.

Wymienione właściwości stanowią zestaw ogólnych zasad projektowych, które definiują sposób budowy systemów reaktywnych. Ich zastosowanie wpływa na sposób organizacji komunikacji między komponentami oraz zarządzania obciążeniem i błędami w systemach współbieżnych [7].

Mechanizm kontroli przepływu danych (*backpressure*)

Jednym z kluczowych mechanizmów wykorzystywanych w reaktywnym modelu współbieżności jest *mechanizm kontroli przepływu danych*, częściej w literaturze określany mianem *backpressure*. Jak podaje Davis [5], mechanizm ten umożliwia odbiorcy ograniczenie tempa napływu danych od producenta. Jego celem jest zapobieganie przeciążeniu odbiorcy poprzez dostosowanie szybkości generowania danych do możliwości ich przetwarzania. W praktyce realizowane jest to poprzez

jawne sygnalizowanie przez odbiorcę zapotrzebowania na określoną liczbę elementów, co pozwala producentowi kontrolować tempo emisji danych i dostosować je do aktualnych możliwości odbiorcy. W środowisku JVM mechanizm ten realizowany jest między innymi w ramach specyfikacji Reactive Streams poprzez metodę `request` wewnątrz interfejsu `Subscription`, za pomocą której odbiorca określa liczbę elementów, jakie jest gotowy przetworzyć [36].

Rozdział 3

Mechanizmy współbieżności oferowane przez Project Loom

Trzeci rozdział pracy poświęcony jest omówieniu najważniejszych mechanizmów dostępnych w ramach Project Loom – inicjatywy usprawniającej tworzenie aplikacji współbieżnych w środowisku JVM. Rozdział prezentuje historię powstawania projektu, a także omawia najważniejsze mechanizmy, które on oferuje, tj. mechanizm wirtualnych wątków (ang. *virtual threads*), mechanizm współbieżności strukturalnej (ang. *structured concurrency*) oraz mechanizm wartości zakresowych (ang. *scoped values*).

3.1 Wprowadzenie do Project Loom

Project Loom jest inicjatywą zapoczątkowaną pod koniec 2017 roku przez OpenJDK. Wyróżnia się jego trzy główne składowe:

- mechanizm *wirtualnych wątków* (ang. *virtual threads*), będący alternatywą dla klasycznego mechanizmu wątków platformowych i zdefiniowany w dokumencie JEP¹ 444 [10],
- mechanizm *współbieżności strukturalnej* (ang. *structured concurrency*), umożliwiający uporządkowanie pracy z wątkami poprzez dodanie *zakresów zadań* (ang. *task scopes*) i strategii zachowań w przypadku wystąpienia błędu lub przerwania działania wątku, opisany w dokumencie JEP 525 [12],
- mechanizm *wartości zakresowych* (ang. *scoped values*), będący alternatywą dla klasy `ThreadLocal` w systemach współbieżnych i opisany w dokumencie JEP 506 [13].

¹JEP (*JDK Enhancement Proposal*) to formalny dokument opisujący proponowane zmiany, nowe funkcjonalności lub usprawnienia platformy JDK. [38]

Początkowo ideą projektu było wprowadzenie lekkich wątków użytkownika (ang. *user-mode threads*) oraz uproszczenie programowania współbieżnego, szczególnie w aplikacjach, których dużą częścią są blokujące zadania, takich jak serwery i systemy sieciowe [9]. Wraz z upływem czasu zakres projektu został rozszerzony o mechanizmy zwiększające bezpieczeństwo aplikacji współbieżnych i ustrukturyzowanie pracy z wątkami. Project Loom wciąż ewoluuje, głównie w kontekście zmian w API poszczególnych mechanizmów, lecz większość z nich jest integralną częścią JDK, dodanych w ramach oficjalnych wydań.

3.2 Wirtualne wątki

Mechanizm wirtualnych wątków, we wczesnych fazach projektu określanych mianem *fibers* [9], to chronologicznie pierwsza z trzech głównych składowych projektu Loom. Po raz pierwszy mechanizm ten pojawił się w środowisku JDK w wersji 19 w ramach wersji pogładowej (ang. *preview*), natomiast jego stabilna wersja została udostępniona w ramach JDK 21. Stanowi on alternatywę dla wątków platformowych, opisanych w rozdziale drugim, w sekcji 2.3.1.

W celu wyjaśnienia potrzeby wprowadzenia mechanizmu wirtualnych wątków do środowiska JVM, dokument JEP 444 [10] odwołuje się do prawa Little’a [39], które wiąże ze sobą latencję, współbieżność i przepustowość w kontekście skalowalności aplikacji współbieżnych:

Dla danego czasu przetwarzania żądania (tj. latencji) liczba żądań obsługiwanych jednocześnie przez aplikację (tj. współbieżność) musi rosnąć proporcjonalnie do tempa napływu żądań (tj. przepustowości).²

Z zależności tej wynika, że zwiększenie przepustowości systemu wymaga zwiększenia liczby jednocześnie obsługiwanych zadań. W klasycznym modelu *thread-per-request*³ prowadzi to jednak do konieczności tworzenia dużej liczby wątków platformowych, co wiąże się z istotnym narzutem pamięciowym oraz kosztami zarządzania, takimi jak przełączanie kontekstu, o czym wspomniano w rozdziale drugim (patrz sekcje 2.2.3 i 2.3.1). W konsekwencji ogranicza to skalowalność aplikacji oraz utrudnia efektywne wykorzystanie dostępnych zasobów sprzętowych.

Alternatywą dla tego podejścia jest model asynchroniczny, w którym liczba wątków jest ograniczana poprzez ich współdzielenie pomiędzy wieloma zadaniami. Rozwiązanie to pozwala zwiększyć skalowalność, jednak kosztem znacznego wzrostu

²Tłumaczenie własne na podstawie [10].

³Paradygmat *thread-per-request* zakłada, że każde przychodzące żądanie jest obsługiwane przez dedykowany wątek. [4]

złożoności kodu, wymagając dekompozycji logiki na mniejsze etapy oraz, w przypadku reaktywnego modelu współbieżności, zrezygnowania z imperatywnego (tj. sekwencyjnego) paradygmatu pisania kodu, co stanowi dodatkowy wkład programisty w utrzymywanie kodu. W rezultacie pojawiła się potrzeba rozwiązania, które łączyłoby prostotę modelu synchronicznego z wydajnością podejścia asynchronicznego, co stanowi bezpośrednią motywację dla wprowadzenia wątków wirtualnych. Stanowią one próbę pogodzenia tych dwóch podejść poprzez umożliwienie tworzenia bardzo dużej liczby lekkich wątków przy zachowaniu prostego, sekwencyjnego modelu programowania. Dzięki temu możliwe jest zwiększenie poziomu współbieżności bez ponoszenia kosztów charakterystycznych dla wątków platformowych oraz bez konieczności stosowania złożonych mechanizmów asynchronicznych.

3.2.1 Zasada działania i cykl życia wirtualnych wątków

Każdorazowo, gdy tworzony jest wirtualny wątek, zostaje on dodany do *kolejki wirtualnych wątków*, będącą kolejką typu FIFO⁴. Wirtualny wątek może wykonać przydzielone mu zadanie, gdy zostanie przypisany do *wątku nośnego* (ang. *carrier thread*). Reprezentuje on wątek platformowy po stronie platformy JVM [11]. Każdy wątek nośny ma przypisany odpowiadający mu wątek platformowy i domyślnie ich liczba jest równa liczbie dostępnych (fizycznych) procesorów logicznych [41]. W odróżnieniu od wątków platformowych i nośnych, liczba wątków wirtualnych nie jest zależna od liczby dostępnych procesorów logicznych, a w większości przypadków ją przekracza. Wynika to ze sposobu zarządzania cyklem ich życia, który został przedstawiony na rysunku 3.1.

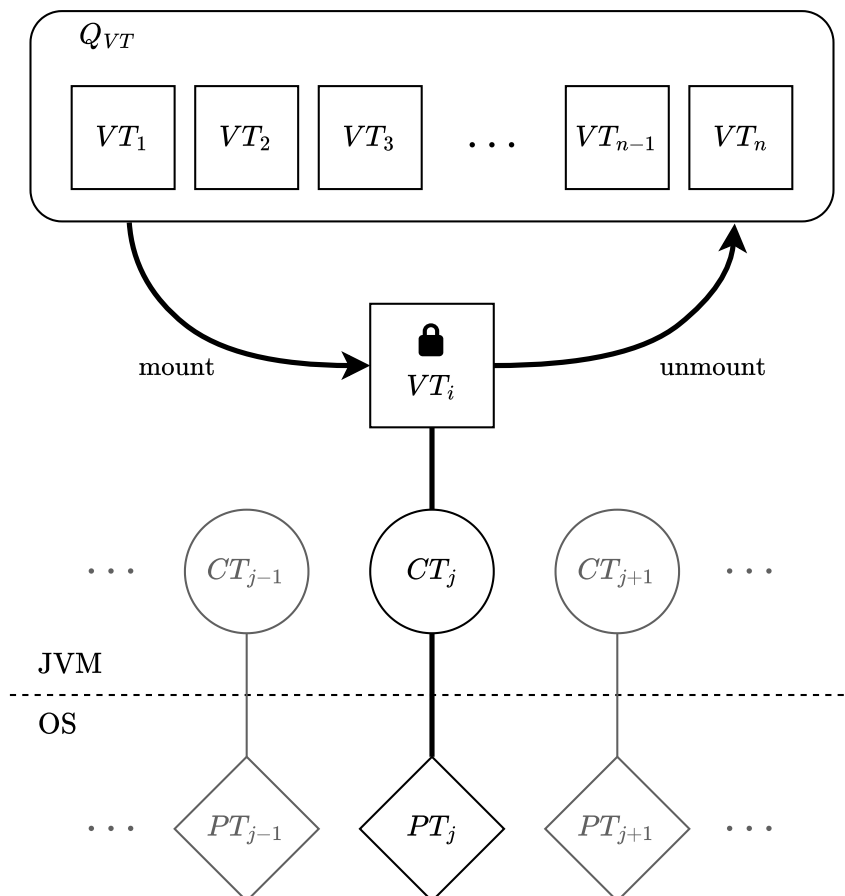
Korzystając z oznaczeń zawartych na rysunku 3.1, cykl życia wirtualnych wątków w środowisku JVM w kontekście jednego wątku nośnego można opisać następująco:

1. Każdy nowy wirtualny wątek VT zostaje dodany na koniec kolejki wirtualnych wątków Q_{VT} .
2. Planista (ang. *scheduler*) wątków wirtualnych, obecny w środowisku JVM, bierze pierwszy wolny wątek wirtualny z kolejki i przypisuje go (ang. *mount*) do wolnego wątku nośnego CT_j .
3. Wątek nośny CT_j , odpowiadający wątkowi platformowemu PT_j , wykonuje kod przypisany do przydzielonego mu wątku wirtualnego VT_i .
4. Gdy wątek wirtualny VT_i wykonuje operację blokującą (stan reprezentowany przez symbol )⁵, JVM zapisuje jego aktualny stan, dodaje go na koniec

⁴Kolejka typu FIFO (ang. *First-In, First-Out*) jest strukturą danych, w której elementy są usuwane w tej samej kolejności, w jakiej zostały dodane, tj. pierwszy dodany element jest obsługiwany jako pierwszy. [40]

kolejki Q_{VT} , jednocześnie zwalniając (ang. *unmount*) wątek nośny CT_j , co jest określane mianem *parkowania* wątku wirtualnego.

5. JVM przypisuje kolejny, tj. pierwszy w kolejce Q_{VT} , wątek wirtualny do wątku nośnego CT_j .



Rysunek 3.1: Cykl życia wirtualnych wątków w środowisku JVM

(źródło: opracowanie własne na podstawie [11])

Kluczową różnicą względem wątków platformowych w powyższym opisie jest punkt 4. Podczas wykonywania operacji blokującej wątek nośny nie wstrzymuje dalszego wykonywania operacji, lecz zapisuje aktualny stan przypisanego do niego wątku wirtualnego, dodaje go do kolejki Q_{VT} (tj. parkuje ten wątek wirtualny) i wykonuje operacje przypisane do następnego w kolejce wątku wirtualnego. Analogiczna sytuacja z wykorzystaniem wątków platformowych skutkowałaby zablokowaniem tego wątku na czas wykonywania operacji blokującej, tj. wątek ten nie mógłby w tym czasie wykonać żadnej innej operacji. Taki sposób zarządzania wątkami pozwala efektywniej wykorzystać czas bezczynności wątku wywoływany przez operacje blokujące, a tym samym pozwala zwiększyć liczbę żądań obsługiwanych jednocześnie przez aplikację. [11]

3.2.2 Pule wirtualnych wątków

Jak wskazują autorzy dokumentu JEP 444 [10], wirtualne wątki nie powinny być wykorzystywane do tworzenia puli wątków. Wynika to z faktu, że w ogólności wzorzec puli obiektów jest przeznaczony do współdzielenia kosztownych zasobów. W przeciwieństwie do wątków platformowych, koszt tworzenia i utrzymywania wątków wirtualnych jest niski, co jest głównym powodem ku temu, aby nie rozważać puli wirtualnych wątków. Ta cecha umożliwia stosowanie wcześniej wspomnianego podejścia *thread-per-request*, a w przypadku potrzeby zdefiniowania maksymalnej liczby współbieżnie wykonywanych zadań, do czego byłoby można zastosować pulę wątków wirtualnych o ustalonym maksymalnym rozmiarze, autorzy dokumentu rekomendują wykorzystanie semaforów [42].

3.3 Współbieżność strukturalna

Współbieżność strukturalna (ang. *structured concurrency*) jest kolejną z trzech głównych składowych projektu Loom. Mechanizm ten pojawił się po raz pierwszy wraz z mechanizmem wirtualnych wątków w JDK 19 w ramach wersji pogładowej, lecz nie został on do tej pory oficjalnie wydany⁵. Współbieżność strukturalna dostarcza mechanizmy umożliwiające uporządkowanie relacji pomiędzy zadaniami wykonywanymi równolegle poprzez odwzorowanie ich w strukturze programu. W odróżnieniu od innych mechanizmów współbieżności, w których zadania mogą być uruchamiane i kończone niezależnie od siebie, mechanizm ten traktuje grupę powiązanych zadań jako jedną jednostkę obliczeniową, umożliwiając ich wspólne zarządzanie, synchronizację oraz obsługę błędów. [12]

Jak podają Veen i Vlijmincx [11], podejście to opiera się na założeniu, że jeśli zadanie dzieli się na podzadania wykonywane współbieżnie, to wszystkie powinny zakończyć się w tym samym miejscu w kodzie, w którym zostały utworzone. Dzięki temu cykl życia zadań jest ściśle powiązany ze strukturą składniową programu, co upraszcza analizę, zwiększa czytelność oraz poprawia niezawodność aplikacji współbieżnych.

3.3.1 Cechy współbieżności strukturalnej

W klasycznym podejściu do współbieżności, określanym jako *współbieżność niestukturalna* (ang. *unstructured concurrency*), zadania są uruchamiane niezależnie od struktury programu, przykładowo przy użyciu interfejsu `ExecutorService`.

⁵Zgodnie ze stanem na kwiecień 2026 roku najnowsza wersja mechanizmu współbieżności strukturalnej została udostępniona w wersji *preview*, co oznacza, że jest dostępna w standardowym JDK, lecz jej składnia lub semantyka mogą ulec zmianie przed ostatecznym włączeniem do specyfikacji platformy Java.

W takim modelu cykl życia podzadań nie jest bezpośrednio powiązany z zadaniem nadrzędnym, co prowadzi do sytuacji, w której podzadania mogą zakończyć się później niż zadanie, które je utworzyło lub kontynuować działanie, mimo wystąpienia błędu w innym fragmencie programu.

W przeciwieństwie do tego podejścia, współbieżność strukturalna narzuca hierarchiczną organizację zadań, w której każde podzadanie jest jednoznacznie powiązane z zadaniem nadrzędnym. Wszystkie podzadania muszą zakończyć się przed zakończeniem zadania nadrzędnego, a ich cykl życia jest ograniczony do określonego bloku kodu. Takie podejście umożliwia traktowanie grupy zadań jako jednej jednostki, co upraszcza zarządzanie ich wykonaniem oraz obsługę błędów. [11]

Autorzy dokumentu JEP 525 [12] wskazują, że mechanizm współbieżności strukturalnej oferuje automatyczną propagację błędów oraz anulowanie powiązanych podzadań. W przypadku niepowodzenia jednego z podzadań możliwe jest anulowanie pozostałych, co zapobiega niekontrolowanemu zużyciu zasobów i upraszcza logikę obsługi błędów.

3.3.2 Struktura i działanie zakresów zadań

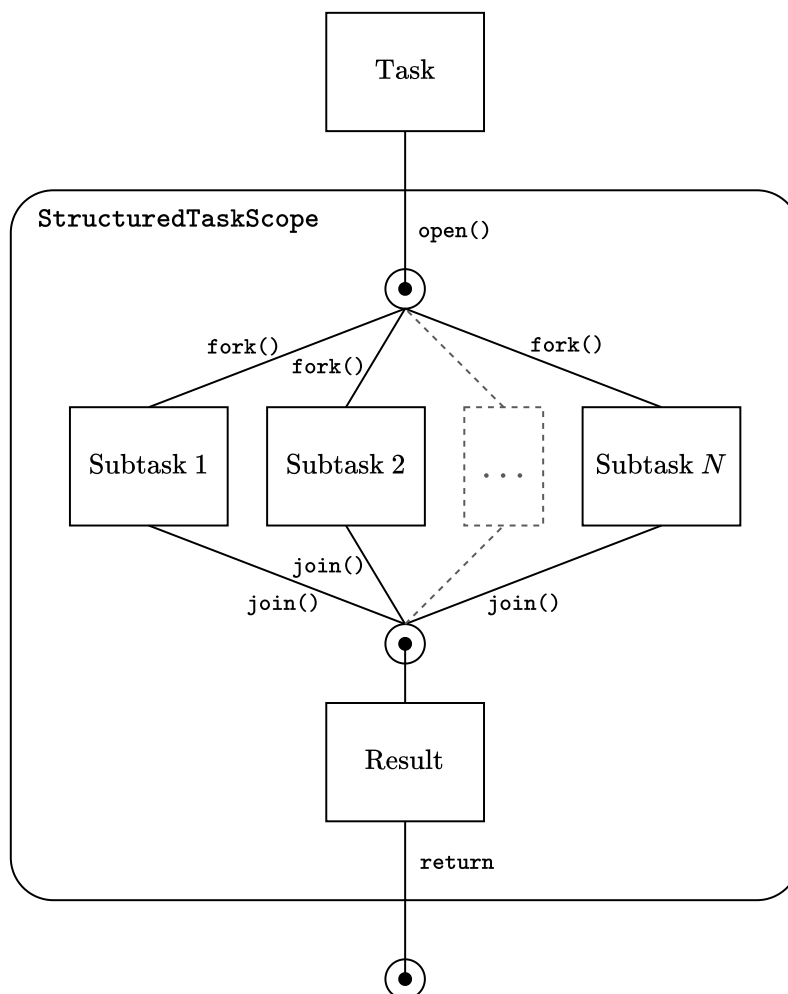
Podstawowym elementem API współbieżności strukturalnej w środowisku JVM jest klasa `StructuredTaskScope` [43], która umożliwia definiowanie zakresu wykonywania współbieżnych podzadań. Zakres ten wyznacza kontekst, w którym tworzone są podzadania oraz w którym następuje ich synchronizacja i obsługa wyników.

Dokument JEP 525 wskazuje, że ogólny przepływ pracy kodu wykorzystującego klasę `StructuredTaskScope` można przedstawić następująco:

1. Utworzenie nowego zakresu zadań (ang. *task scope*) poprzez wywołanie jednej z metod `open`. Wątek, który otwiera zakres, staje się jego właścicielem.
2. Uruchomienie podzadań (ang. *subtasks*) w ramach zakresu przy użyciu metody `fork`.
3. Synchronizacja wszystkich podzadań zakresu jako całości za pomocą metody `join`, która może zgłosić wyjątek.
4. Przetworzenie uzyskanych wyników.
5. Zamknięcie zakresu (najczęściej w sposób niejawni przy użyciu konstrukcji `try-with-resources`⁶), co powoduje anulowanie zakresu (jeżeli nie został wcześniej anulowany), a tym samym anulowanie wszystkich pozostałych podzadań oraz oczekiwanie na ich zakończenie.

⁶Konstrukcja `try-with-resources` umożliwia automatyczne zamykanie zasobów po zakończeniu ich użycia. [44]

Rysunek 3.2 przedstawia zgodny z powyższym opisem schemat działania współbieżności strukturalnej w ramach klasy `StructuredTaskScope`.



Rysunek 3.2: Schemat działania zakresu zadań w ramach klasy `StructuredTaskScope` w środowisku JVM

(źródło: opracowanie własne na podstawie [11])

3.3.3 Strategie synchronizacji podzadań

Mechanizm współbieżności strukturalnej umożliwia definiowanie różnych strategii synchronizacji podzadań za pomocą interfejsu `Joiner` [45]. Odpowiada on za określenie sposobu zakończenia całego zakresu oraz przetwarzania wyników podzadań. Interfejs posiada statyczne metody definiujące cztery podstawowe strategie synchronizacji podzadań:

- `allSuccessfulOrThrow()` – strategia wymagająca poprawnego zakończenia wszystkich podzadań; w przypadku błędu któregośkolwiek z nich zakres zostaje anulowany, a metoda `join` zgłasza wyjątek,

- `anySuccessfulResultOrThrow()` – strategia zwracająca wynik pierwszego poprawnie zakończonego podzadania; jeżeli wszystkie podzadania zakończą się błędem, zgłaszany jest wyjątek,
- `awaitAllSuccessfulOrThrow()` – strategia oczekująca na zakończenie wszystkich podzadań, przy czym wymaga ich poprawnego wykonania; w przypadku błędu któregośkolwiek z nich zakres zostaje anulowany, a metoda `join` zgłasza wyjątek (strategia ta nie zwraca wyników, a jedynie sygnalizuje powodzenie lub porażkę całej operacji),
- `awaitAll()` – strategia oczekująca na zakończenie wszystkich podzadań niezależnie od ich wyniku; nie powoduje anulowania zakresu ani zgłoszenia wyjątku w przypadku błędów, co czyni ją użyteczną w scenariuszach opartych na efektach ubocznych zamiast zwracanych wartościach.

Wybór odpowiedniej strategii synchronizacji zależy od charakteru problemu oraz oczekiwanego sposobu obsługi błędów i wyników podzadań. Interfejs `Joiner` umożliwia również tworzenie własnych strategii synchronizacji. Odbywa się to poprzez implementację interfejsu w klasie reprezentującej własną strategię oraz odpowiednie nadpisanie metod `onFork`, `onComplete`, `onTimeout` i `result`. [12, 45]

3.4 Wartości zakresowe

Wartości zakresowe (ang. *scoped values*) stanowią mechanizm projektu Loom, umożliwiający współdzielenie niezmiennych danych pomiędzy metodą a jej wywołaniami w obrębie jednego wątku, a także pomiędzy wątkiem nadrzędnym i jego wątkami potomnymi. Ich głównym celem jest zastąpienie mechanizmu oferowanego przez klasę `ThreadLocal` [46], który umożliwia przechowywanie danych specyficznych dla danego wątku, zapewniając każdemu wątkowi własną, niezależną kopię zmiennej. Jak podaje dokument JEP 506, mechanizm wartości zakresowych charakteryzuje się niższą złożonością czasową i pamięciową niż mechanizm dostępny w ramach klasy `ThreadLocal`, zwłaszcza w połączeniu z innymi mechanizmami oferowanymi przez Project Loom [13]. Mechanizm wartości zakresowych po raz pierwszy pojawił się w JDK 20 w wersji eksperymentalnej, a oficjalnie został włączony do środowiska JVM w ramach wydania JDK 25.

3.4.1 Opis mechanizmu `ThreadLocal`

Jak podają Veen i Vlijmincx [11], klasa `ThreadLocal` stanowi mechanizm umożliwiający przechowywanie danych specyficznych dla danego wątku. Zmienne typu `ThreadLocal` są powiązane z konkretnym wątkiem, dzięki czemu każdy

wątek posiada własną, niezależną wartość tej zmiennej i może uzyskać dostęp wyłącznie do swojej kopii. Mechanizm ten zapewnia wbudowaną izolację danych pomiędzy wątkami, eliminując konieczność stosowania synchronizacji i upraszczając kod. Umożliwia również wygodne przechowywanie informacji kontekstowych, takich jak dane użytkownika czy kontekst transakcji, bez potrzeby przekazywania ich jako parametrów. Dodatkowo pozwala na definiowanie konfiguracji specyficznej dla wątku. [11, 46]

Pomimo swojej użyteczności, klasa `ThreadLocal` posiada istotne ograniczenia. Jednym z nich jest fakt, że wartości przechowywane wewnątrz obiektu tej klasy są mutowalne, tj. mogą być modyfikowane w dowolnym miejscu kodu, co utrudnia analizę przepływu danych oraz znacząco komplikuje proces debugowania. Ponadto brak jawnego zarządzania cyklem życia wartości może prowadzić do wycieków pamięci, szczególnie w przypadku ponownego wykorzystywania wątków w pulach, gdzie nieusunięte dane pozostają powiązane z ponownie używanymi wątkami. Dodatkowo w środowiskach współbieżnych, zwłaszcza przy wykorzystaniu wątków wirtualnych, mechanizm ten może powodować nadmierne zużycie pamięci, wynikające z konieczności kopiowania wartości do wątków potomnych, co w konsekwencji może prowadzić do problemów ze skalowalnością. [11, 46]

3.4.2 Zasada działania wartości zakresowych

Mechanizm wartości zakresowych stanowi alternatywę dla klasy `ThreadLocal`, wprowadzając odmienny model współdzielenia danych. W przeciwieństwie do `ThreadLocal`, gdzie wartości są powiązane z całym cyklem życia wątku, wartości zakresowe obowiązują wyłącznie w określonym zakresie wykonania programu. Oznacza to, że ich widoczność jest ograniczona do dynamicznego kontekstu wywołań metod, w którym zostały zdefiniowane, a po jego zakończeniu są automatycznie usuwane. Takie podejście eliminuje konieczność ręcznego zarządzania cyklem życia danych oraz ogranicza ryzyko błędów wynikających z niekontrolowanego współdzielenia stanu. [13]

Podstawą działania mechanizmu jest jawne powiązanie wartości z określonym zakresem wykonania (ang. *binding*). Realizowane jest ono poprzez metody klasy `ScopedValue` [47], które umożliwiają utworzenie kontekstu, w ramach którego dana wartość jest dostępna. Wartość ta może być następnie odczytywana przez wszystkie metody wywoływane bezpośrednio lub pośrednio w obrębie tego zakresu. Kluczową cechą tego podejścia jest dynamiczny charakter zakresu, tj. jego granice są wyznaczane przez przebieg wykonania programu, a nie przez strukturę składniową kodu. [13, 11]

Główny mechanizm wartości zakresowych opiera się na kilku podstawowych metodach klasy `ScopedValue` i klas wewnętrznych. Metody `where(SCOPE).run()` oraz `where(SCOPE).call()` umożliwiają powiązanie wartości z danym zakresem `SCOPE` oraz wykonanie kodu w jego obrębie, implementującego odpowiednio interfejsy `Runnable` i `Callable`. Dostęp do wartości realizowany jest za pomocą metody `get`. Dodatkowo klasa oferuje metodę `isBound`, pozwalającą sprawdzić, czy dana wartość jest aktualnie powiązana z bieżącym kontekstem wykonania [47].

W odróżnieniu od mechanizmu oferowanego w ramach klasy `ThreadLocal`, wartości zakresowe są niezienne w obrębie danego zakresu, co oznacza, że po ich przypisaniu nie mogą być modyfikowane. Możliwe jest jednak ich ponowne powiązanie w zagnieżdżonym zakresie, co prowadzi do tymczasowego przesłonięcia wcześniejszej wartości. Takie podejście zapewnia większą przewidywalność działania programu oraz ułatwia analizę przepływu danych, szczególnie w aplikacjach współbieżnych. [47]

3.5 Przegląd podobnych mechanizmów współbieżności w innych środowiskach

Mechanizmy dostępne w ramach Project Loom, w szczególności wirtualne wątki oraz współbieżność strukturalna, nie stanowią rozwiązań unikalnych w skali języków programowania i środowisk wykonawczych. Podobne koncepcje zostały wcześniej zaimplementowane w innych środowiskach. Wybrane rozwiązania przedstawiono poniżej.

Korutyny (język Kotlin)

W środowisku JVM rozwiązaniem częściowo zbliżonym do wątków wirtualnych są *korutyny* (ang. *coroutines*) dostępne w języku Kotlin. Pozwalają one na realizację dużej liczby współbieżnych operacji przy niewielkim koszcie zasobowym, jednak wymagają stosowania dedykowanego modelu programowania opartego na funkcjach oznaczonych słowem kluczowym `suspend`. W przeciwieństwie do tego podejścia wątki wirtualne zachowują tradycyjny model programowania oparty na wywołaniach blokujących [48].

Gorutyny (język Go)

Jednym z najczęściej przywoływanych odpowiedników wątków wirtualnych poza środowiskiem JVM są *gorutyny* (ang. *goroutines*) dostępne w języku Go. Stanowią one lekkie jednostki wykonania zarządzane przez środowisko uruchomieniowe, które mogą być tworzone w dużych ilościach przy niewielkim narzucie pamięciowym

i obliczeniowym. Podobnie jak wątki wirtualne, gorutyny umożliwiają implementację aplikacji o wysokim stopniu współbieżności bez konieczności tworzenia dużej liczby wątków systemowych [49].

Lekkie procesy (język Erlang)

Podobną ideę realizują również *lekkie procesy* (ang. *lightweight processes*) wykorzystywane przez środowisko Erlang/BEAM. Umożliwiają one uruchamianie ogromnej liczby współbieżnych zadań zarządzanych przez maszynę wirtualną języka. W odróżnieniu od klasycznego modelu wątków opierają się jednak na izolacji pamięci i komunikacji za pomocą wiadomości, co skutkuje odmiennym modelem programowania [50].

Mechanizm CoroutineScope (język Kotlin)

Odpowiedniki współbieżności strukturalnej można znaleźć między innymi w bibliotece korutyn języka Kotlin. Mechanizm `CoroutineScope` organizuje zadania w strukturę hierarchiczną, w której zadania potomne są powiązane z cyklem życia zadania nadrzędnego. Podobne założenia przyjęto w mechanizmie `StructuredTaskScope` wprowadzonym przez Project Loom [51].

Klasa TaskGroup (język Python)

Zbliżone rozwiązania zostały również zaimplementowane w innych współczesnych językach programowania. Przykładem jest klasa `TaskGroup` dostępna od wersji 3.11 języka Python, umożliwiająca grupowanie zadań asynchronicznych oraz wspólne zarządzanie ich cyklem życia [52]. Podobne mechanizmy oferuje także język Swift, w którym współbieżność strukturalna stanowi jeden z podstawowych elementów modelu programowania współbieżnego [53].

Przedstawione przykłady pokazują, że zarówno lekkie jednostki wykonania, jak i współbieżność strukturalna są koncepcjami obecnymi w wielu współczesnych środowiskach programistycznych. Project Loom nie wprowadza zatem całkowicie nowych idei, lecz adaptuje sprawdzone rozwiązania do specyfiki platformy JVM, zachowując jednocześnie zgodność z istniejącymi bibliotekami oraz imperatywnym modelem programowania.

Rozdział 4

Przegląd literatury i opis stanu badań nad Project Loom

Czwarty rozdział pracy zawiera opis stanu badań i wnioski wynikające z przeglądu literatury we względnie nowej dziedzinie, jaką jest Project Loom, kładąc szczególny nacisk na badania opisujące zastosowania tych mechanizmów. Rozdział ten definiuje także lukę badawczą na podstawie dokonanego przeglądu oraz określa kierunek przeprowadzonych badań.

4.1 Opis i charakterystyka badań nad Project Loom

Mimo że Project Loom i badania wykorzystujące jego mechanizmy są obszarem stosunkowo nowym, istnieją już prace empiryczne, analizy porównawcze i prace dyplomowe badające ich zalety, ograniczenia i praktyczne efekty. Wyróżnia się trzy kategorie, na które można podzielić publikacje opisujące lub badające tematykę Project Loom:

- prace akademickie, skupiające się w dużej mierze na testach empirycznych, porównując mechanizmy oferowane przez Project Loom z innymi modelami współbieżności, w szczególności pod kątem wysokiego obciążenia mocy obliczeniowej procesora (ang. *CPU-bound*) i wysokiego obciążenia wejścia-wyjścia (ang. *I/O-bound*),
- dokumentacja techniczna i dokumenty JEP, stanowiące fundament teoretyczny i szczegółowo opisujące potrzebę wprowadzenia mechanizmów będących częścią Project Loom, ich zastosowania i dogłębne aspekty techniczne,
- inne teksty dotyczące mechanizmów oferowanych przez Project Loom, w tym nietechniczne artykuły oraz wpisy na blogach i forach.

4.2 Przegląd wybranych pozycji

4.2.1 Prace akademickie

Ponnampalam w pracy *Evaluation of Virtual Threads and RxJava: A performance and code complexity analysis in a real-time clearing system* [14] porównuje wątki wirtualne z biblioteką reaktywną RxJava w realnym systemie transakcyjnym pod względem wydajności (mierząc takie parametry, jak latencja i przepustowość), zużycia pamięci i procesora oraz złożoności kodu. Wyniki uzyskane przez autora wskazują, że wirtualne wątki przewyższają mechanizmy reaktywne w scenariuszach z dużą liczbą blokujących operacji i przy wysokiej współbieżności, zużywając mniej zasobów i upraszczając kod. Ponnampalam zaznacza jednak, że w wybranych, wysoce zoptymalizowanych przypadkach reaktywnych, zastosowanie biblioteki RxJava może być porównywalnie wydajne jak zastosowanie wirtualnych wątków.

Podobne wnioski wysnuł Pandita w pracy *Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads* [54], w ramach której badana była wydajność i wykorzystanie zasobów przez wątki wirtualne i platformowe w obciążeniach intensywnie wykorzystujących procesor i przy wysokim obciążeniu wejścia-wyjścia. Autor wskazał, że dla obciążeń I/O-bound wirtualne wątki wykazują większą skalowalność i obsługują większe obciążenie przy jednoczesnym mniejszym zużyciu pamięci i niższej latencji. Dodał również, że dla obciążeń CPU-bound oba modele zachowują się podobnie. Ponadto, wysoka współbieżność przy zastosowaniu wirtualnych wątków zwiększa przepustowość nawet o około 60%, a latencja spada o około 28% w porównaniu do wątków platformowych.

Praca Gustafssona i Perssona zatytułowana *Comparison of Concurrency Technologies in Java* [15] dotyczy empirycznego porównania trzech podejść do współbieżności w Javie w kontekście aplikacji serwerowych o dużym obciążeniu. Autorzy koncentrują się na trzech modelach: wątkach platformowych, wirtualnych i modelu reaktywnym. Autorzy zaimplementowali w tym celu serwer HTTP, inspirowany benchmarkiem Fortune (Techempower) z dodatkowymi opóźnieniami, aby wymusić różne wzorce obciążenia. Autorzy pracy doszli do następujących wniosków w kontekście wirtualnych wątków:

- wirtualne wątki przewyższają wątki platformowe w przepustowości i latencji w wielu scenariuszach aż do pewnego progu obciążenia,
- wirtualne wątki bardziej obciążają zużycie procesora i pamięci niż model reaktywny, szczególnie przy krótkich operacjach blokujących,
- w testach CPU-bound wątki platformowe były w niektórych przypadkach szybsze od wirtualnych wątków, ale zużywały mniej CPU tylko w konkretnych ustawieniach.

Ponadto autorzy zauważyli, że wątki wirtualne mogą szybciej osiągać wysoki poziom wykorzystania pamięci, co jest zgodne z efektem widocznym w praktyce przy intensywnym tworzeniu i obsłudze wątków, szczególnie przy operacjach wejścia-wyjścia. Wskazują również, że różnice w wydajności wirtualnych wątków i modelu reaktywnego mogą wynikać z zachowania garbage collector, który intensywniej działa przy dużej liczbie krótkotrwałych obiektów.

Publikacje innych autorów [11, 55, 56, 8, 57] w dużym stopniu pokrywają lub uzupełniają tematykę i zakres badań przytoczonych wyżej prac. Przegląd literatury nie wykazał istnienia prac pokrywających inne zagadnienia związane z mechanizmami Project Loom niż przytoczone wyżej.

4.2.2 Dokumentacja techniczna

Dokumentacja techniczna związana z mechanizmami oferowanymi przez Project Loom ogranicza się do dokumentów JEP oraz do oficjalnej dokumentacji języka Java. Dokumenty *JEP 444: Virtual Threads* [10], *JEP 525: Structured Concurrency (Sixth Preview)* [12] i *JEP 506: Scoped Values* [13] szczegółowo opisują techniczne aspekty wskazanych mechanizmów współbieżności i potrzebę ich wprowadzenia. Wskazują również podstawowe zastosowania i ograniczenia wprowadzonych mechanizmów.

4.3 Zdefiniowanie luki badawczej i kierunku własnych badań

Przegląd publikacji związanych z zastosowaniem mechanizmów oferowanych przez Project Loom wykazał brak pokrycia innych aspektów ich wykorzystania niż w kontekście wysokiego obciążenia mocy obliczeniowej procesora i wysokiego obciążenia wejścia i wyjścia. Zauważono również, że wszystkie publikacje badają zastosowanie wirtualnych wątków, bez uwzględnienia pozostałych mechanizmów, tj. współbieżności strukturalnej i wartości zakresowych. Nie znaleziono innych publikacji badających integrację mechanizmów dostępnych w ramach Project Loom z istniejącymi mechanizmami dostępnymi w środowisku JVM niż przytoczone, głównie opartymi na operacjach blokujących lub programowanie asynchroniczne.

W ramach niniejszej pracy zdecydowano się pokryć lukę badawczą poprzez implementację dwóch scenariuszy: zastosowania mechanizmu wirtualnych wątków w kontekście integracji z bazą danych z wykorzystaniem sterownika blokującego (JDBC) i reaktywnego (R2DBC) oraz zastosowania mechanizmu współbieżności strukturalnej w kontekście agregacji wyników opartej na mechanizmie kworum. Szczegółowy opis implementacji obu scenariuszy zawarty jest w rozdziale piątym.

Rozdział 5

Metodyka badań

Piąty rozdział prezentuje metodykę badań przeprowadzonych w ramach pracy, w tym cel i zakres badań, hipotezy badawcze oraz szczegółowy opis dwóch scenariuszy testowych: scenariusza $\mathcal{A}$, opartego na integracji z bazą danych ze sterownikiem blokującym i reaktywnym oraz scenariusza $\mathcal{B}$, opartego na agregacji wyników opartej na mechanizmie kworum, prezentując dla każdego z nich przyjęte założenia i opisując ich implementację w ramach wybranych modeli współbieżności. Rozdział zawiera również opis środowiska i konfiguracji przeprowadzonych eksperymentów, a także opisuje kryteria oceny wyników.

5.1 Cel i zakres badań

Celem niniejszej pracy jest analiza porównawcza oraz zbadanie możliwości integracyjnych mechanizmów wprowadzonych w ramach Project Loom, w szczególności wirtualnych wątków oraz mechanizmu współbieżności strukturalnej, z istniejącymi modelami współbieżności w środowisku JVM. Badania koncentrują się na weryfikacji wydajności oraz właściwości implementacyjnych tych rozwiązań w kontekście dwóch scenariuszy badawczych:

- **Scenariusz $\mathcal{A}$: Integracja z bazą danych ze sterownikiem blokującym i reaktywnym**, który dotyczy analizy operacji blokujących wejścia-wyjścia przy wykorzystaniu różnych sterowników umożliwiających nawiązania połączenia z bazą danych: standardowego sterownika blokującego (JDBC) oraz jego reaktywnego odpowiednika (R2DBC),
- **Scenariusz $\mathcal{B}$: Agregacja wyników oparta na mechanizmie kworum**, który obejmuje implementację procesu i ocenę mechanizmów koordynacji zadań i zarządzania ich cyklem życia w ramach równoległego pobierania danych z wielu źródeł, gdzie wynik końcowy jest determinowany przez odpowiedzi od określonej liczby serwisów.

W ramach pracy zdefiniowano następujące cele szczegółowe:

1. **ocena charakterystyki wydajnościowej:** porównanie przepustowości oraz opóźnień wirtualnych wątków z istniejącymi modelami współbieżności w oparciu o wyniki uzyskane z benchmarków JMH,
2. **ocena wpływu mechanizmów Project Loom na model programowania współbieżnego:** analiza zmian w sposobie strukturyzacji kodu przy przejściu z paradygmatu asynchronicznego (reaktywnego) na imperatywny model oparty na wirtualnych wątkach,
3. **badanie kompatybilności z istniejącymi bibliotekami i frameworkami:** weryfikacja łatwości integracji mechanizmów Project Loom z obecnymi narzędziami dostępnymi w środowisku JVM oraz identyfikacja potencjalnych ograniczeń z tym związanych,
4. **analiza jakościowa kodu pod kątem złożoności implementacyjnej:** porównanie struktury kodu powstałego w ramach implementacji scenariuszy $\mathcal{A}$ i $\mathcal{B}$ ze szczególnym uwzględnieniem czytelności zapisu logiki biznesowej oraz trudności w utrzymaniu kodu w różnych modelach współbieżności,
5. **sformułowanie rekomendacji technicznych:** opracowanie wytycznych dotyczących doboru odpowiedniego modelu współbieżności w zależności od specyfiki problemu obliczeniowego oraz wymagań systemowych dotyczących skalowalności i czasu odpowiedzi.

5.2 Hipotezy badawcze

Jak wspomniano wyżej, głównym celem pracy jest porównanie mechanizmów oferowanych w ramach Project Loom, w szczególności mechanizmu wirtualnych wątków i mechanizmu współbieżności strukturalnej, z istniejącymi modelami współbieżności pod kątem wydajności i złożoności implementacyjnej. Na podstawie dokonanego przeglądu literatury i przyjętych szczegółowych celów badawczych, zdefiniowano następującą hipotezę główną:

Mechanizmy oferowane w ramach Project Loom umożliwiają osiągnięcie porównywalnej lub wyższej wydajności względem istniejących modeli współbieżności przy jednoczesnym zachowaniu imperatywnego modelu programowania i uproszczeniu implementacji wybranych scenariuszy współbieżnych.

Poniżej znajdują się hipotezy szczegółowe, wynikające z hipotezy głównej. Zostały one podzielone na trzy kategorie: hipotezy dotyczące scenariusza $\mathcal{A}$, hipotezy dotyczące scenariusza $\mathcal{B}$ oraz hipotezy dotyczące modelu programowania.

Hipotezy szczegółowe dla scenariusza A

- H1.** Zastosowanie wirtualnych wątków w przypadku operacji wejścia-wyjścia opartych na komunikacji z bazą danych pozwala osiągnąć wyższą przepustowość niż podejście oparte na wątkach platformowych.
- H2.** Wraz ze wzrostem poziomu współbieżności rośnie różnica w wydajności pomiędzy podejściem opartym na wątkach platformowych a wątkach wirtualnych, szczególnie dla operacji o charakterze blokującym.
- H3.** Podejście wykorzystujące reaktywny sterownik dostępu do bazy danych umożliwia osiągnięcie wysokiej przepustowości i niskich czasów odpowiedzi, jednak wiąże się z większą złożonością implementacyjną niż rozwiązania wykorzystujące wirtualne wątki.
- H4.** Zastosowanie wirtualnych wątków w połączeniu z blokującym sterownikiem dostępu do bazy danych pozwala osiągnąć wydajność porównywalną do podejścia reaktywnego przy zachowaniu prostszego, imperatywnego modelu programowania.

Hipotezy szczegółowe dla scenariusza B

- H5.** Mechanizm współbieżności strukturalnej upraszcza implementację scenariuszy równoległej agregacji wyników względem podejścia reaktywnego oraz klasycznego modelu opartego na pulach wątków.
- H6.** W wariantach scenariusza obejmujących serwisy o wysokich opóźnieniach mechanizmy Project Loom umożliwiają osiągnięcie czasów odpowiedzi porównywalnych z modelem reaktywnym.
- H7.** Współbieżność strukturalna zapewnia bardziej przewidywalny model zarządzania cyklem życia zadań oraz propagacją błędów niż klasyczne podejścia wykorzystujące natywne rozwiązania lub programowanie reaktywne.

Hipotezy szczegółowe dotyczące modelu programowania

- H8.** Podejście reaktywne charakteryzuje się wyższym poziomem złożoności implementacyjnej i większą trudnością analizy przepływu wykonania niż podejścia imperatywne oparte na wątkach platformowych lub wirtualnych.
- H9.** Mechanizmy oferowane przez Project Loom umożliwiają implementację aplikacji współbieżnych z wykorzystaniem modelu programowania bliższego sekwencyjnemu przepływowi wykonania, co upraszcza implementację i analizę kodu.

5.3 Szczegółowy opis scenariuszy testowych

5.3.1 Scenariusz A: Integracja z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC)

Scenariusz A ma na celu zbadanie wpływu doboru modelu współbieżności na integrację z bazą danych przy wykorzystaniu różnych sterowników odpowiedzialnych za komunikację z bazą danych, ze szczególnym uwzględnieniem zastosowania mechanizmu wirtualnych wątków, oferowanego w ramach Project Loom. W środowisku JVM wyróżnia się dwa główne sterowniki odpowiedzialne za nawiązanie połączenia z bazą danych:

- JDBC (ang. *Java Database Connectivity*), oferujący standardowe API, w którym operacje są wykonywane w sposób blokujący [58],
- R2DBC (ang. *Reactive Relational Database Connectivity*), oferujący API zaprojektowane do podejścia reaktywnego, w którym operacje na bazie są nieblokujące i oparte na strumieniach danych [59].

Warianty scenariusza

W ramach eksperymentu skonstruowano trzy warianty reprezentujące różne modele współbieżności, wykorzystujące wymienione wyżej rodzaje sterowników umożliwiających łączenie się z bazą danych:

- JDBC+PT, wykorzystujący sterownik JDBC przy wykorzystaniu wątków platformowych,
- R2DBC, wykorzystujący sterownik R2DBC oraz reaktywny model współbieżności,
- JDBC+VT, wykorzystujący sterownik JDBC, lecz przy wykorzystaniu wątków wirtualnych, oferowanych przez Project Loom.

Typy i konstrukcja zapytań do bazy danych

Dla każdego wariantu eksperymentu skonstruowano zestaw trzech zapytań do bazy danych, reprezentujących różne rodzaje obciążeń. Szczegóły dotyczące konfiguracji i konstrukcji bazy danych, a także budowania poniższych zapytań przedstawiono w dalszej części rozdziału, w sekcji 5.4.2.

Zapytanie Q1 jest prostym pobraniem wartości po kluczu głównym, a jego implementacja w języku SQL została przedstawiona w listingu 5.1.

```
SELECT name, email
FROM users
WHERE id = <ID>;
```

Listing 5.1: Zapytanie Q1 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$
(źródło: opracowanie własne)

Zapytanie Q2 jest prostą zmianą wartości po kluczu głównym, którego implementacja została przedstawiona w listingu 5.2.

```
UPDATE orders
SET total = total + 10
WHERE id = <ID>;
```

Listing 5.2: Zapytanie Q2 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$
(źródło: opracowanie własne)

Zapytanie Q3 reprezentuje złożone zapytanie przy wykorzystaniu agregacji i wielokrotnego złączenia, co zostało przedstawione w listingu 5.3.

```
SELECT COUNT(DISTINCT o.id),
       SUM(oi.quantity),
       SUM(oi.quantity * p.price)
FROM users u JOIN orders o ON o.user_id = u.id
             JOIN order_items oi ON oi.order_id = o.id
             JOIN products p ON p.id = oi.product_id
WHERE u.id = <ID>;
```

Listing 5.3: Zapytanie Q3 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$
(źródło: opracowanie własne)

Poziom współbieżności

Na potrzeby eksperymentu zdefiniowano pojęcie *poziomu współbieżności*. W zależności od wariantu eksperymentu, fraza ta oznacza:

- dla wariantu JDBC+PT: rozmiar puli wątków platformowych,
- dla wariantu R2DBC: liczbę równocześnie aktywnych subskrypcji,
- dla wariantu JDBC+VT: liczbę tworzonych wątków wirtualnych.

Z racji, że scenariusz jest badany pod kątem różnych poziomów współbieżności, w każdym wariancie wszystkie wątki rozpoczynają wykonywanie zapytania w tym samym momencie, aby nie zaburzyć wyniku z powodu powolnego rozłożenia startu, a także zadbane o poprawne zamknięcie połączenia z bazą po wykonaniu każdego zapytania.

Implementacja wariantów scenariusza

Listing 5.4 prezentuje uproszczoną implementację współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+PT. Wariant ten wykorzystuje pulę platformowych wątków (patrz linie 3. i 8.–9.) oraz sterownik JDBC do wykonania zapytania do bazy danych (patrz linia 15.). Szczegółową implementację metody `executeQueryUsingJDBC`, wykonującej zapytanie przy użyciu sterownika JDBC, przedstawiono w listingu 5.7. Obiekty klasy `CountDownLatch`: `countDownLatchStart` i `countDownLatchDone` zapewniają wcześniej wspomniany wymóg rozpoczęcia wykonania zapytania przez wszystkie wątki w tym samym momencie. Wartości zmiennych `concurrency` i `query`, widoczne w pierwszych dwóch liniach kodu, reprezentują odpowiednio poziom współbieżności i wykonywane zapytanie.

```

1  var concurrency = TESTED_CONCURRENCY_LEVEL;
2  var query = TESTED_QUERY;
3  var platformThreadFactory = Thread.ofPlatform().factory();
4  var countDownLatchStart = new CountDownLatch(1);
5  var countDownLatchDone = new CountDownLatch(concurrency);
6
7  try (
8      var executor = Executors.newFixedThreadPool(
9          concurrency, platformThreadFactory)
10 ) {
11     for (int i = 0; i < concurrency; i++) {
12         executor.submit(() -> {
13             try {
14                 countDownLatchStart.await();
15                 executeQueryUsingJDBC(query);
16                 countDownLatchDone.countDown();
17             } catch (Exception e) {
18                 ...
19             } finally {
20                 countDownLatchDone.countDown();
21             }
22         });
23     }
24 }
25
26 countDownLatchStart.countDown();
27 countDownLatchDone.await();

```

Listing 5.4: Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+PT

(źródło: opracowanie własne)

Listing 5.5 prezentuje uproszczoną implementację wykonania zapytania w wariancie JDBC+VT. Jediną różnicą względem wariantu JDBC+PT jest wykorzystanie innego mechanizmu wykonawczego zadań (ang. *executor*), angażującego wirtualne wątki (patrz linia 7.) zgodnie z paradygmatem *thread-per-request*, o którym wspomniano w podrozdziale 3.2.

```

1  var concurrency = CONCURRENCY_LEVEL;
2  var query = QUERY;
3  var countDownLatchStart = new CountDownLatch(1);
4  var countDownLatchDone = new CountDownLatch(concurrency);
5
6  try (var executor =
7      Executors.newVirtualThreadPerTaskExecutor()) {
8      for (int i = 0; i < concurrency; i++) {
9          executor.submit(() -> {
10             try {
11                 countDownLatchStart.await();
12                 executeQueryUsingJDBC(query);
13                 countDownLatchDone.countDown();
14             } catch (Exception e) {
15                 ...
16             } finally {
17                 countDownLatchDone.countDown();
18             }
19         });
20     }
21 }
22
23 countDownLatchStart.countDown();
24 countDownLatchDone.await();

```

Listing 5.5: Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+VT

(źródło: opracowanie własne)

Listing 5.6 przedstawia uproszczoną implementację współbieżnego wykonania zapytania w wariancie R2DBC. Wykorzystuje ona w tym celu operatory strumieniowe, takie jak `range`, `flatMap`, `then` i `block` wraz z odpowiednim wywołaniem metody `executeQueryUsingR2DBC`, wykonującej zapytanie do bazy danych przy użyciu sterownika R2DBC (patrz linia 5.). Szczegółową implementację tej metody przedstawiono w listingu 5.8.

```

1  var concurrency = CONCURRENCY_LEVEL;
2  var query = QUERY;
3
4  Flux.range(1, concurrency)
5      .flatMap(_ -> executeQueryUsingR2DBC(query), concurrency)
6      .then()
7      .block();

```

Listing 5.6: Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie R2DBC
(źródło: opracowanie własne)

Jak wspomniano wyżej, warianty JDBC+PT i JDBC+VT scenariusza $\mathcal{A}$ do wykonania zapytania do bazy danych wykorzystują metodę `executeQueryUsingJDBC`, zaś wariant R2DBC wykorzystuje w tym celu metodę `executeQueryUsingR2DBC`. Ich uproszczoną implementację prezentują odpowiednio listingi 5.7 i 5.8.

Listing 5.7 przedstawia implementację metody `executeQueryUsingJDBC`, wykonującej zapytanie do bazy danych przy użyciu sterownika JDBC. Reprezentuje ona klasyczne podejście do nawiązywania połączenia z bazą danych i wykonania zapytania. Metoda ta wykorzystuje obiekt klasy `Blackhole` (patrz linie 2. i 12.), dostępny w ramach narzędzia JMH (Java Microbenchmark Harness) i służący do poprawnego przetwarzania danych w czasie przeprowadzania benchmarku. Narzędzie JMH i obiekt klasy `Blackhole` zostały szczegółowo opisane w dalszej części, w sekcji 5.4.3.

```

1  var id = generateNextIdForQuery();
2  var blackhole = getBenchmarkBlackhole();
3
4  void executeQueryUsingJDBC(String query) {
5      try (var connection = getConnectionByJDBC();
6          var statement = connection.prepareStatement(query)
7      ) {
8          statement.setLong("<ID>", id);
9          try (var resultSet = statement.executeQuery()) {
10             while (resultSet.next()) {
11                 var value = resultSet.getString(1);
12                 blackhole.consume(value);
13                 ...
14             }
15         }
16     } catch (SQLException e) { ... }
17 }

```

Listing 5.7: Uproszczona implementacja metody wykonującej zapytanie do bazy danych przy użyciu sterownika JDBC w ramach scenariusza $\mathcal{A}$
(źródło: opracowanie własne)

Listing 5.8 przedstawia implementację metody `executeQueryUsingR2DBC`, wykonującej zapytanie do bazy danych przy użyciu sterownika R2DBC. Cechuje ją użycie podejścia reaktywnego, w tym operatorów strumieniowych celem przetworzenia odpowiedzi uzyskanej z bazy danych. Metoda ta również wykorzystuje wspomniany wcześniej obiekt klasy `Blackhole` (patrz linie 3. i 16.). Wyrażenie lambda wewnątrz metody `map` (patrz linie 13.–18.) zwraca instancję typu `Object` ze względu na specyfikę narzędzia JMH. Metoda ta może zwracać cokolwiek ze względu na fakt, że wynik jest wcześniej konsumowany przez obiekt typu `Blackhole`.

```

1      var connectionFactory = getConnectionFactoryForR2DBC();
2      var id = generateNextIdForQuery();
3      var blackhole = getBenchmarkBlackhole();
4
5      void executeQueryUsingR2DBC(String query) {
6          return Mono.usingWhen(
7              connectionFactory.create(),
8              connection -> {
9                  Flux.from(connection.createStatement(query)
10                      .bind("<ID>", id)
11                      .execute())
12                      .flatMap(result ->
13                          result.map((row, _) -> {
14                              var value =
15                                  row.get(COLUMN_NAME, String.class);
16                              blackhole.consume(value);
17                              ...
18                              return new Object();
19                          }))
20                      .then()
21                  },
22                  Connection::close
23          );
24      }

```

Listing 5.8: Uproszczona implementacja metody wykonującej zapytanie do bazy danych przy użyciu sterownika R2DBC w ramach scenariusza $\mathcal{A}$

(źródło: opracowanie własne)

5.3.2 Scenariusz $\mathcal{B}$: Agregacja wyników oparta na mechanizmie kworum

Scenariusz $\mathcal{B}$ ma na celu zbadanie wpływu doboru mechanizmu oferującego współbieżne przetwarzanie danych, ze szczególnym uwzględnieniem mechanizmu współbieżności strukturalnej, w kontekście zadania polegającego na agregacji wyników opartej na mechanizmie kworum (ang. *quorum mechanism*). Mechanizm ten

polega na podejmowaniu decyzji lub uznaniu operacji za zakończoną dopiero po uzyskaniu określonej liczby odpowiedzi (tzw. *wartość kworum*), zamiast oczekiwania na wszystkie odpowiedzi.

Opis logiki mechanizmu kworum

W analizowanym scenariuszu rozpatrywany jest zbiór pięciu niezależnych operacji, reprezentujących wywołania zewnętrznych serwisów. Operacje te są inicjowane równoległe i wykonywane współbieżnie. Każda z nich może zakończyć się sukcesem, zwracając wynik, lub niepowodzeniem. Celem przetwarzania jest uzyskanie co najmniej trzech poprawnych wyników (tj. wartość kworum jest równa 3). Osiągnięcie tego progu traktowane jest jako warunek sukcesu całego procesu agregacji. W momencie spełnienia tego warunku dalsze oczekiwanie na pozostałe wyniki zostaje przerwane, a wszystkie niezakończone operacje są anulowane. W przypadku, gdy liczba poprawnie zakończonych operacji jest mniejsza niż wartość kworum, proces uznawany jest za zakończony niepowodzeniem.

Warianty scenariusza

W ramach eksperymentu zdecydowano się na zbadanie trzech wariantów umożliwiających implementację logiki zawartej w scenariuszu $\mathcal{B}$:

- **Legacy**, wykorzystujący w tym celu interfejs `ExecutorService` i natywne narzędzia upraszczające współbieżne przetwarzanie danych, w tym klasę `CompletableFuture`,
- **Reactive**, wykorzystujący w tym celu mechanizmy programowania reaktywnego, dostępne w ramach biblioteki `Reactive Streams`,
- **Loom**, wykorzystujący w tym celu mechanizm współbieżności strukturalnej w oparciu o własną implementację strategii synchronizacji podzadań.

Opis symulowanych serwisów

Warianty scenariusza testowego $\mathcal{B}$ wykorzystują różne typy serwisów. Każdy serwis jest definiowany przez zestaw następujących parametrów:

- D_{base} – wartość bazowego opóźnienia odpowiedzi serwisu, podana w milisekundach i wyznaczana z zadanego przedziału (ang. *base delay*),
- J_{base} – wartość odchylenia od bazowego opóźnienia, podana w milisekundach i wyznaczana z zadanego przedziału (ang. *jitter*),
- P_{outlier} – prawdopodobieństwo wystąpienia bardzo wolnej odpowiedzi, gdzie $0 \leq P_{\text{outlier}} \leq 1$ (ang. *outlier probability*),

- L_{outlier} – wartość opóźnienia podczas wystąpienia bardzo wolnej odpowiedzi, podana w milisekundach (ang. *outlier latency*).

Na potrzeby eksperymentu wartości bazowego opóźnienia (D_{base}) i odchylenia od tej wartości (J_{base}) oraz ustalenie wystąpienia bardzo wolnej odpowiedzi (P_{outlier}) są wyznaczane losowo przy użyciu ustalonej wartości ziarna (ang. *seed*), co gwarantuje deterministyczne odtworzenie dokładnych wartości opóźnień symulowanych serwisów dla każdego wariantu badanego modelu współbieżności.

Tabela 5.1 przedstawia cztery typy serwisów wraz z ich parametrami, symulujące różne rodzaje opóźnień i czasu przetwarzania. Serwisy typu **FAST** i **MEDIUM** charakteryzują się odpowiednio krótkim i umiarkowanie krótkim czasem odpowiedzi przy małym prawdopodobieństwie wystąpienia bardzo wolnej odpowiedzi. Serwis typu **SLOW** cechuje się długim czasem odpowiedzi przy nieco większym prawdopodobieństwie wystąpienia bardzo wolnej odpowiedzi niż poprzednie serwisy. Odmiennym zachowaniem charakteryzuje się serwis typu **OUTLIER_PRONE**, który bazowo zwraca odpowiedź w umiarkowanie krótkim czasie, lecz posiada największe prawdopodobieństwo wystąpienia bardzo wolnej odpowiedzi w porównaniu do pozostałych serwisów.

Typ serwisu	D_{base}	J_{base}	P_{outlier}	L_{outlier}
FAST	5–10 ms	2–4 ms	0.03	40 ms
MEDIUM	30–45 ms	5–10 ms	0.03	150 ms
SLOW	70–120 ms	10–20 ms	0.05	250 ms
OUTLIER_PRONE	25–40 ms	10–20 ms	0.2	300 ms

Tabela 5.1: Typy i parametry symulowanych serwisów wykorzystywanych w ramach scenariusza $\mathcal{B}$

(źródło: opracowanie własne)

W ramach przeprowadzonego eksperymentu skonstruowano i wykorzystano cztery konfiguracje serwisów:

1. **THREE_FAST_TWO_SLOW**, wykorzystujący trzy serwisy typu **FAST** i dwa serwisy typu **SLOW**,
2. **ONE_FAST_FOUR_SLOW**, wykorzystujący jeden serwis typu **FAST** i cztery serwisy typu **SLOW**,
3. **FIVE_SIMILAR**, wykorzystujący pięć serwisów typu **MEDIUM**,
4. **OUTLIER_HEAVY**, wykorzystujący pięć serwisów typu **OUTLIER_PRONE**.

Opis implementacji wariantów scenariusza

Listingi 5.9 i 5.10 przedstawiają uproszczoną implementację logiki zawartej w scenariuszu $\mathcal{B}$ w wariantcie *Legacy*. Listing 5.9 prezentuje wysokopoziomowy mechanizm współbieżnego wywołania pięciu serwisów, a listing 5.10 zawiera właściwą logikę odpowiedzialną za zwrócenie wyniku lub wywołanie błędu agregacji. Wariant *Legacy* implementuje wariant agregacji wyników oparty na klasycznym mechanizmie wykorzystującym klasę `CompletableFuture` [60] i mechanizm wykonawczy zadań `ExecutorService` [27]. Dla każdej konfiguracji eksperymentu uruchamiane są równoległe wywołania wszystkich serwisów. Uzyskane odpowiedzi są odczytywane do momentu osiągnięcia wymaganej wartości kworum. Każde wywołanie serwisu jest przekazywane do mechanizmu wykonawczego przez metodę `supplyAsync` (patrz linia 6. w listingu 5.9), natomiast oczekiwanie na odpowiedzi odbywa się poprzez blokujące wywołanie metody `get` (patrz linia 8. w listingu 5.10). Po osiągnięciu wartości kworum wyniki są agregowane, a pozostałe niezakończone zadania są anulowane. W wariantcie *Legacy* implementacja wymaga ręcznego anulowania wywołań serwisów w momencie zakończenia pracy poprzez blok `finally` (patrz linia 13. w listingu 5.9).

```
1  var quorum = 3;
2  var poolSize = 5;
3  var executor = Executors.newFixedThreadPool(poolSize);
4  var services = getServices(TESTED_CONFIGURATION);
5  var futures = services.stream()
6      .map(service -> CompletableFuture.supplyAsync(
7          service::call, executor))
8      .toList();
9
10 try {
11     return aggregateQuorum(futures, quorum);
12 } finally {
13     futures.forEach(future -> future.cancel(true));
14 }
```

Listing 5.9: Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariantcie *Legacy*

(źródło: opracowanie własne)

```

1  String aggregateQuorum(
2      List<CompletableFuture<String>> futures,
3      int quorum
4  ) {
5      var successes = new ArrayList<String>(quorum);
6      for (var future : futures) {
7          try {
8              var result = future.get();
9              successes.add(result);
10             if (successes.size() >= quorum) {
11                 return aggregate(successes);
12             }
13         } catch (Exception e) { ... }
14     }
15
16     if (successes.size() >= quorum) {
17         return aggregate(successes);
18     }
19     throw new IllegalStateException("Quorum not achieved");
20 }

```

Listing 5.10: Metoda zawierająca właściwą logikę agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Legacy

(źródło: opracowanie własne)

Listing 5.11 prezentuje uproszczoną implementację logiki w ramach wariantu **Reactive**. Reprezentuje ona podejście reaktywne, w którym współbieżność i przetwarzanie odpowiedzi są realizowane przez operatory strumieniowe zamiast przez bezpośrednie zarządzanie obiektami typu `Future`, jak w przypadku wariantu **Legacy**. Dla każdej konfiguracji eksperymentu tworzony jest strumień danych za pomocą metody `Flux.fromIterable` (patrz linia 4.), w którym wywołania poszczególnych serwisów są uruchamiane równolegle w ramach metody `Mono.fromCallable` (patrz linia 6.). Operator `take` kończy zbieranie wyników po uzyskaniu wymaganej liczby poprawnych odpowiedzi, a błędne odpowiedzi są pomijane przez operator `onErrorResume`. W przypadku nieosiągnięcia wartości kworum zwracany jest obiekt `Mono.error`, informujący o błędzie agregacji (patrz linie 14.–20.). W przeciwnym wypadku zebrane wyniki są agregowane do pojedynczego wyniku tekstowego i zwracane synchronicznie za pomocą metody `block`.

```
1  var quorum = 3;
2  var services = getServices(TESTED_CONFIGURATION);
3
4  return Flux.fromIterable(services)
5      .flatMap(service ->
6          Mono.fromCallable(service::call)
7              .subscribeOn(Schedulers.boundedElastic())
8              .onErrorResume(_ -> Mono.empty()),
9          services.size()
10     )
11     .take(quorum)
12     .collectList()
13     .flatMap(results -> {
14         if (results.size() < quorum) {
15             return Mono.error(
16                 new IllegalStateException(
17                     "Quorum not achieved"
18                 )
19             );
20         }
21         return Mono.just(aggregate(results));
22     })
23     .block();
```

Listing 5.11: Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Reactive

(źródło: opracowanie własne)

Listingi 5.12 i 5.13 przedstawiają uproszczoną implementację logiki w ramach wariantu Loom. Wykorzystuje on mechanizm współbieżności strukturalnej i własną implementację strategii agregacji wyników, uwzględnioną wewnątrz klasy `QuorumJoiner` (patrz listing 5.13). Dla każdej konfiguracji eksperymentu tworzony jest zakres zadań, w którym każdy serwis wywoływany jest jako osobne podzadanie przy użyciu metody `fork` (patrz linia 7. w listingu 5.12). Do sterowania zakończeniem oczekiwania wykorzystywana jest niestandardowa strategia, zaimplementowana w ramach klasy `QuorumJoiner`, która zbiera poprawne odpowiedzi i kończy oczekiwanie po osiągnięciu wymaganej wartości kworum. Po zebraniu wyników za pomocą metody `join`, w przypadku uzyskania wymaganej wartości kworum następuje agregacja i zwrócenie wyniku, a w przeciwnym razie następuje zwrócenie błędu agregacji. Użycie konstrukcji `try-with-resources` zapewnia automatyczne przerwanie wywołań serwisów po uzyskaniu wyniku (patrz linia 5. w listingu 5.12).


```

1  var quorum = 3;
2  var services = getServices(TESTED_CONFIGURATION);
3  var joiner = new QuorumJoiner(quorum);
4
5  try (var scope = StructuredTaskScope.open(joiner)) {
6      for (var service : services) {
7          scope.fork(service::call);
8      }
9      var successes = scope.join();
10     if (successes.size() < quorum) {
11         throw new IllegalStateException("Quorum not achieved");
12     }
13     return aggregate(successes);
14 }

```

Listing 5.12: Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Loom

(źródło: opracowanie własne)

```

1  public final class QuorumJoiner
2  implements StructuredTaskScope.Joiner<String, List<String>> {
3
4      private final int quorum;
5      private final ConcurrentLinkedQueue<String> successes;
6      private final AtomicInteger successCount;
7
8      public QuorumJoiner(int quorum) { ... }
9
10     @Override
11     public boolean onComplete(
12         Subtask<? extends String> subtask
13     ) {
14         if (subtask.state() == Subtask.State.SUCCESS) {
15             successes.add(subtask.get());
16         }
17         return successCount.incrementAndGet() >= quorum;
18     }
19
20     @Override
21     public List<String> result() {
22         return List.copyOf(successes);
23     }
24 }

```

Listing 5.13: Implementacja własnej strategii agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Loom

(źródło: opracowanie własne)

5.4 Środowisko i konfiguracja eksperymentu

5.4.1 Środowisko sprzętowe i programowe

Eksperymenty przeprowadzono na maszynie o konfiguracji sprzętowej zestawionej w tabeli 5.2.

Parametr	Wartość
Procesor	AMD Ryzen 5 5600H
Bazowe taktowanie procesora	3.30 GHz
Liczba rdzeni procesora	6
Liczba procesorów logicznych	12
Pamięć operacyjna (RAM)	32 GB
System operacyjny	Windows 11 Pro (25H2)
Typ dysku	SSD (NVMe)

Tabela 5.2: Konfiguracja sprzętowa środowiska eksperymentalnego
(źródło: opracowanie własne)

W ramach eksperymentu wykorzystano narzędzia i biblioteki zestawione w tabeli 5.3.

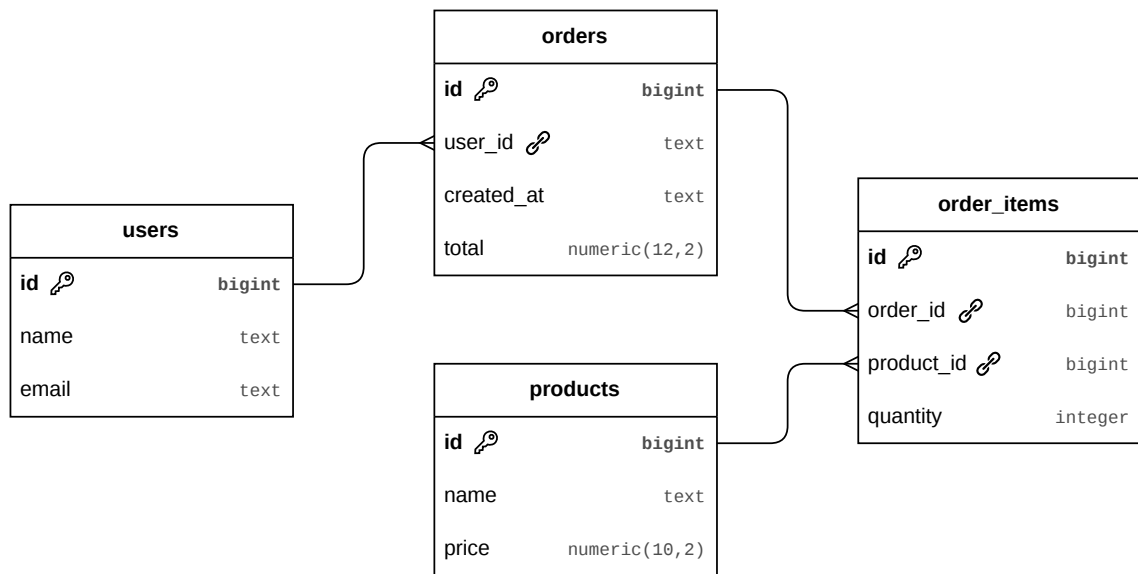
Narzędzie / biblioteka	Wersja
Java (JDK)	25
Gradle	9.2.1
JMH (Java Microbenchmark Harness)	1.37
Project Reactor	3.8.2
Sterownik JDBC (PostgreSQL)	42.7.3
Sterownik R2DBC (PostgreSQL)	1.1.1.RELEASE
Docker	29.1.3
PostgreSQL (obraz Docker)	16

Tabela 5.3: Wersje narzędzi i bibliotek wykorzystanych w eksperymencie
(źródło: opracowanie własne)

Uwaga: Z uwagi na fakt, że mechanizm współbieżności strukturalnej pozostaje w użytej wersji platformy Java funkcją poglądową (ang. *preview feature*), kod benchmarków skompilowano i uruchomiono z włączoną flagą `--enable-preview`.

5.4.2 Konstrukcja i konfiguracja środowiska bazodanowego

W każdym wariancie scenariusza testowego $\mathcal{A}$ wykorzystano relacyjną bazę danych PostgreSQL ze względu na wbudowane wsparcie dla równoległego przetwarzania zapytań, a także wsparcie połączenia za pośrednictwem sterowników JDBC i R2DBC [61]. Bazę danych uruchomiono w odrębnym środowisku przy wykorzystaniu konteneryzacji oferowanej przez narzędzie Docker [62]. Skonstruowano prosty schemat bazy danych oparty na systemie składania zamówień, przedstawiony na rysunku 5.1.



Rysunek 5.1: Schemat bazy danych wykorzystanej w ramach scenariusza $\mathcal{A}$
(źródło: opracowanie własne)

W celu zapewnienia spójności danych i powtarzalności eksperymentu zdecydowano się na stworzenie deterministycznego generatora, który dla danej wartości ziarna generuje powtarzalny zestaw danych w bazie. Aby zredukować wpływ bazy danych na wynik eksperymentu, ustalono stałą maksymalną wartość równoczesnych połączeń z bazą danych, równą 32.

Każdy wariant scenariusza $\mathcal{A}$ wykorzystuje generator liczb losowych ze stałą, ustaloną wartością ziarna do generowania kolejnych identyfikatorów wymaganych w zapytaniach (patrz miejsca oznaczone jako $\langle \text{ID} \rangle$ w listingach 5.1, 5.2 i 5.3), co umożliwia deterministyczne odtworzenie sekwencji tych samych zapytań dla każdego wywołania testu o tej samej wartości ziarna.

5.4.3 Konfiguracja benchmarków JMH

Badania zdefiniowane w ramach scenariuszy $\mathcal{A}$ i $\mathcal{B}$ przeprowadzono w postaci benchmarków przy użyciu narzędzia Java Microbenchmark Harness (JMH) [63], stanowiącego standardowy framework służący do przeprowadzania pomiarów wydajności aplikacji działających w środowisku JVM. Narzędzie to umożliwia wykonywanie powtarzalnych i możliwie precyzyjnych pomiarów czasu wykonania kodu, uwzględniając specyfikę działania maszyny wirtualnej Java, w szczególności mechanizmy takie jak kompilacja Just-In-Time (JIT), optymalizacje wykonywanego kodu czy proces rozgrzewania środowiska uruchomieniowego.

W celu uniknięcia niezamierzonych optymalizacji przez środowisko JVM każdy wariant obu scenariuszy wykorzystuje narzędzie `Blackhole` [64], dostępne w ramach JMH. Konsumuje ono wyniki obliczeń w sposób, którego kompilator nie może zoptymalizować, gwarantując tym samym, że mierzony kod faktycznie zostaje wykonany, a uzyskane wyniki odzwierciedlają rzeczywisty koszt obliczeń.

Konfiguracja benchmarku dla scenariusza $\mathcal{A}$

Każdy z trzech wariantów scenariusza $\mathcal{A}$ (JDBC+PT, R2DBC, JDBC+VT) został uruchomiony dla każdego z trzech zapytań (Q1, Q2, Q3) i pięciu poziomów współbieżności (16, 32, 64, 256, 1024) w następującej konfiguracji narzędzia JMH:

- `BenchmarkMode` – tryby benchmarku: `Throughput` i `SampleTime`,
- `Warmup` – liczba iteracji rozgrzewkowych¹: 5, każda trwająca 10 sekund,
- `Measurement` – liczba iteracji pomiarów właściwych: 10, każda trwająca 10 sekund,
- `Fork` – liczba rozgałęzień²: 3,
- `Threads` – liczba wątków jednocześnie wykonujących kod benchmarku: 1.

Uwaga: Liczba wątków w powyższej konfiguracji została ustawiona na 1 ze względu na fakt, że zarządzanie liczbą i rodzajem wątków zostało przeniesione bezpośrednio do metod benchmarkowanych – jest ono reprezentowane przez poziom współbieżności, opisany w sekcji 5.3.1.

¹Iteracje rozgrzewkowe pozwalają na ustabilizowanie działania środowiska uruchomieniowego JVM, m.in. poprzez aktywację kompilatora JIT i optymalizacji wykonywanego kodu, przed rozpoczęciem właściwych pomiarów. [65]

²Rozgałęzienia to liczba niezależnych uruchomień środowiska JVM, w których wykonywany jest pomiar. [65]

Konfiguracja benchmarku dla scenariusza $\mathcal{B}$

Każdy z trzech wariantów scenariusza $\mathcal{B}$ (`Legacy`, `Reactive`, `Loom`) został uruchomiony dla każdego z czterech konfiguracji serwisów (`THREE_FAST_TWO_SLOW`, `ONE_FAST_FOUR_SLOW`, `FIVE_SIMILAR`, `OUTLIER_HEAVY`) w następującej konfiguracji narzędzia JMH:

- `BenchmarkMode` – tryb benchmarku: `SampleTime`,
- `Warmup` – liczba iteracji rozgrzewkowych: 5, każda trwająca 10 sekund,
- `Measurement` – liczba iteracji pomiarów właściwych: 10, każda trwająca 10 sekund,
- `Fork` – liczba rozgałęzień: 3,
- `Threads` – liczba wątków jednocześnie wykonujących kod benchmarku: 1.

Uwaga: Analogicznie jak w ramach scenariusza $\mathcal{A}$, wartość parametru `Threads` ustawiono na 1 ze względu na fakt, że zarządzanie wątkami zostało przeniesione bezpośrednio do metod benchmarkowanych. Liczbę równoległe uruchamianych serwisów w ramach jednego wariantu ustalono na 5, zgodnie z opisem zawartym w sekcji 5.3.2.

5.5 Kryteria oceny wyników

W celu oceny wyników badań przeprowadzonych w ramach scenariuszy $\mathcal{A}$ i $\mathcal{B}$ zdefiniowano trzy grupy kryteriów: miary wydajnościowe i latencji, analizę skalowalności oraz jakościową ocenę aspektów implementacyjnych.

5.5.1 Metryki wydajnościowe

Pomiary wydajnościowe przeprowadzono z wykorzystaniem dwóch trybów narzędzia JMH, opisanego w sekcji 5.4.3: trybu pomiaru przepustowości (`Throughput`) i trybu próbkowania czasu (`SampleTime`).

Tryb pomiaru przepustowości (`Throughput`) mierzy liczbę operacji wykonanych przez badany fragment kodu w jednostce czasu [65]. Wynikiem jest wartość *przepustowości* (oznaczana jako `THRPT`), wyrażona w operacjach na sekundę. Tryb ten zastosowano w ramach scenariusza $\mathcal{A}$.

Tryb próbkowania czasu (`SampleTime`) mierzy czas wykonania pojedynczych wywołań benchmarkowanej metody na podstawie próbek zbieranych w trakcie iteracji pomiarowych [65]. Jak podaje Luszczewicz [66], mianem *percentyla rzędu* p , gdzie $p \in [0, 100]$, określa się taką wartość x , że co najmniej $p\%$ obserwacji ma

wartość mniejszą lub równą x , a co najwyżej $(100 - p)\%$ obserwacji ma wartość większą lub równą x . Na podstawie zebranych próbek wyznacza się następujące metryki:

- AVG – średni czas wykonania badanego fragmentu kodu,
- P_n – percentyl rzędu n czasu wykonania, umożliwiający ocenę latencji i zachowania w ogonie rozkładu.

Tryb `SampleTime` zastosowano w obu scenariuszach. Pomiar w obu scenariuszach obejmuje metrykę AVG, zaś mierzonymi wartościami percentyli są P50, P95 i P99 dla scenariusza $\mathcal{A}$ oraz P50, P95, P99 i P99.9 dla scenariusza $\mathcal{B}$.

5.5.2 Analiza skalowalności

Skalowalność poszczególnych wariantów oceniana jest na podstawie zmian wartości metryk wydajnościowych wraz ze wzrostem poziomu współbieżności w ramach scenariusza $\mathcal{A}$. Za wskaźnik skalowalności przyjmuje się zachowanie przepustowości i czasu wykonania dla kolejnych poziomów współbieżności (16, 32, 64, 256, 1024), co pozwala zidentyfikować warianty wykazujące proporcjonalną degradację wydajności oraz te, w których degradacja jest gwałtowna i następuje po przekroczeniu określonego progu obciążenia.

5.5.3 Ocena aspektów implementacyjnych

Ocena aspektów implementacyjnych ma charakter jakościowy i obejmuje trzy aspekty. Pierwszym z nich jest *czytelność i przejrzystość kodu*, rozumiana jako zdolność danego modelu do odwzorowania logiki biznesowej w imperatywnym (sekwencyjnym) modelu, który jest naturalny dla większości programistów. Drugim aspektem jest *złożoność zarządzania cyklem życia zadań*, obejmująca sposób obsługi błędów, propagacji przerwań i anulowania współbieżnych operacji. Trzeci aspekt stanowi *stopień integracji z istniejącymi bibliotekami* dostępnymi w środowisku JVM, określający w jakim stopniu dany model wymaga modyfikacji istniejącego kodu lub stosowania dedykowanych adapterów.

Rozdział 6

Wyniki badań

Szósty rozdział pracy prezentuje wyniki benchmarków przeprowadzonych w ramach scenariuszy $\mathcal{A}$ i $\mathcal{B}$. Obejmuje on szczegółową prezentację wyników pomiarów w postaci tabel i wykresów dla obu trybów benchmarku: pomiaru przepustowości oraz pomiaru latencji, a także analizę porównawczą uzyskanych wyników.

6.1 Prezentacja wyników pomiarów

6.1.1 Wyniki pomiarów w ramach scenariusza $\mathcal{A}$

Tryb pomiaru przepustowości (Throughput)

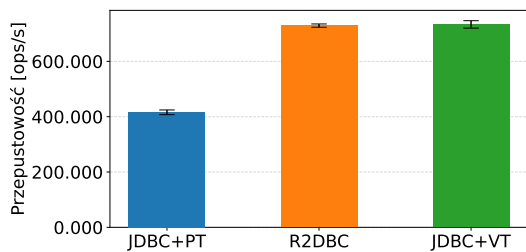
Tabele 6.1, 6.2 i 6.3 oraz odpowiadające im wykresy przedstawione na rysunkach 6.1, 6.2 i 6.3 prezentują wyniki benchmarku przeprowadzonego w ramach scenariusza $\mathcal{A}$ w trybie pomiaru przepustowości, tj. w trybie **Throughput**. Wartości przedstawione w tabelach i na odpowiadających im wykresach zostały wyznaczone w zależności od wariantu modelu współbieżności, poziomu współbieżności i zapytania. Dodatkowe uwagi do tabel 6.1, 6.2 i 6.3:

- (i) Wyniki we wspomnianych wyżej tabelach podane są jako liczba operacji na sekundę. Wykresy przedstawione na odpowiadających im rysunkach prezentują te dane na osi pionowej w jednostce oznaczonej jako [ops/s].
- (ii) Wyniki we wspomnianych wyżej tabelach przedstawione są w formacie $S \pm E$, gdzie S oznacza uzyskany wynik, a E oznacza błąd pomiaru.
- (iii) Wyniki i wartości błędów pomiaru we wspomnianych wyżej tabelach zapisane zostały z kropką jako separatorem dziesiętnym i zmierzone z dokładnością do części tysięcznych, tj. 0.001.

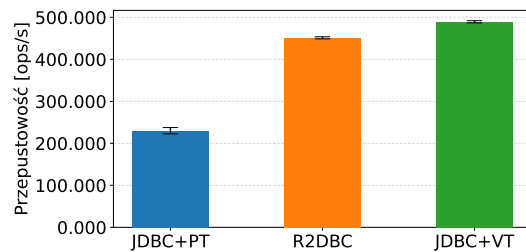
Poz. współb.	JDBC+PT	R2DBC	JDBC+VT
16	415.881 ± 8.497	729.356 ± 5.935	733.642 ± 13.414
32	230.098 ± 7.511	451.219 ± 2.474	489.411 ± 2.623
64	116.672 ± 4.463	240.370 ± 4.193	250.862 ± 0.876
256	28.871 ± 1.067	67.178 ± 0.352	66.336 ± 0.378
1024	6.694 ± 0.306	17.139 ± 0.105	16.782 ± 0.111

Tabela 6.1: Wartości przepustowości dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

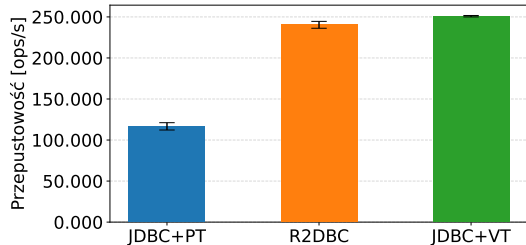
(źródło: opracowanie własne)



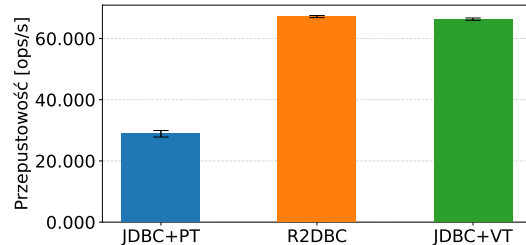
(a) poziom współbieżności: 16



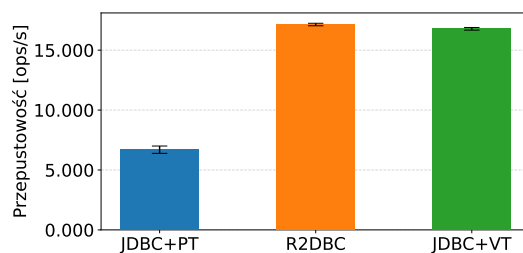
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

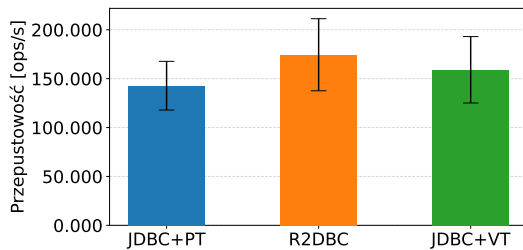
Rysunek 6.1: Wykresy kolumnowe przepustowości dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

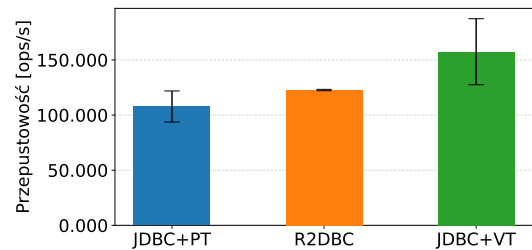
Poz. współb.	JDBC+PT	R2DBC	JDBC+VT
16	142.817 $\pm$ 24.898	174.474 $\pm$ 36.850	159.069 $\pm$ 33.964
32	107.766 $\pm$ 14.097	122.683 $\pm$ 0.635	157.456 $\pm$ 29.941
64	55.572 $\pm$ 5.507	74.120 $\pm$ 10.150	70.235 $\pm$ 4.447
256	14.765 $\pm$ 1.268	19.392 $\pm$ 2.232	19.884 $\pm$ 1.825
1024	3.736 $\pm$ 0.365	4.600 $\pm$ 0.031	5.042 $\pm$ 0.379

Tabela 6.2: Wartości przepustowości dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

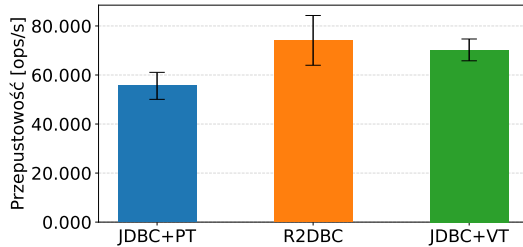
(źródło: opracowanie własne)



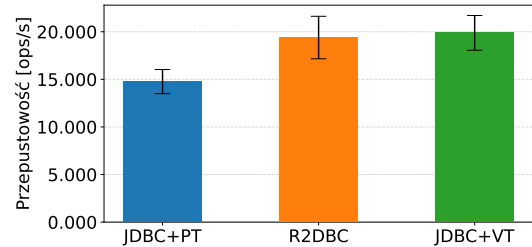
(a) poziom współbieżności: 16



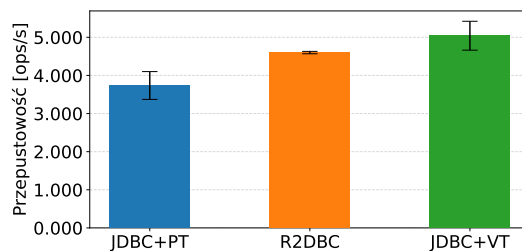
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

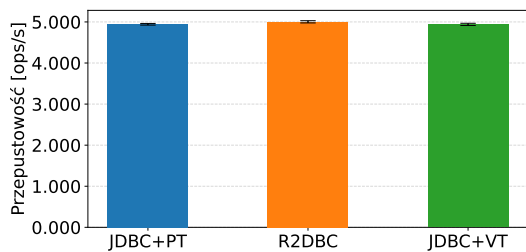
Rysunek 6.2: Wykresy kolumnowe przepustowości dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

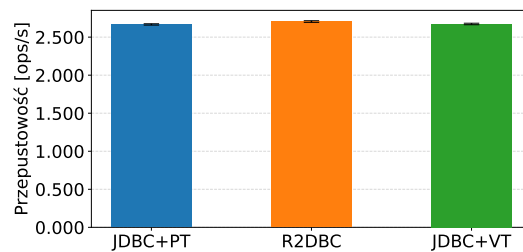
Poz. współb.	JDBC+PT	R2DBC	JDBC+VT
16	4.943 ± 0.020	5.003 ± 0.027	4.940 ± 0.027
32	2.667 ± 0.009	2.705 ± 0.011	2.673 ± 0.010
64	1.319 ± 0.004	1.332 ± 0.006	1.314 ± 0.008
256	0.330 ± 0.002	0.335 ± 0.001	0.330 ± 0.002
1024	0.187 ± 0.001	0.084 ± 0.001	0.192 ± 0.001

Tabela 6.3: Wartości przepustowości dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

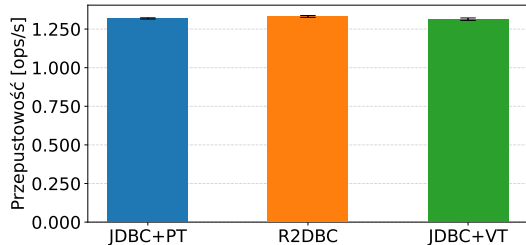
(źródło: opracowanie własne)



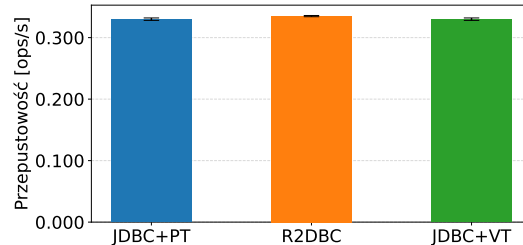
(a) poziom współbieżności: 16



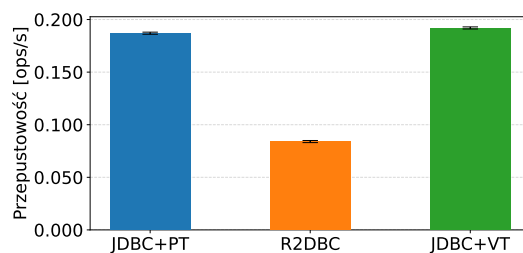
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

Rysunek 6.3: Wykresy kolumnowe przepustowości dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Tryb próbkowania czasu (SampleTime)

Tabele 6.4, 6.5 i 6.6 oraz odpowiadające im wykresy przedstawione na rysunkach 6.4, 6.5 i 6.6 prezentują wyniki benchmarku przeprowadzonego w ramach scenariusza $\mathcal{A}$ w trybie próbkowania czasu, tj. w trybie **SampleTime**. Wartości przedstawione w tabelach i na odpowiadających im wykresach zostały wyznaczone w zależności od wariantu modelu współbieżności, poziomu współbieżności i zapytania.

Dodatkowe uwagi do tabel 6.4, 6.5 i 6.6:

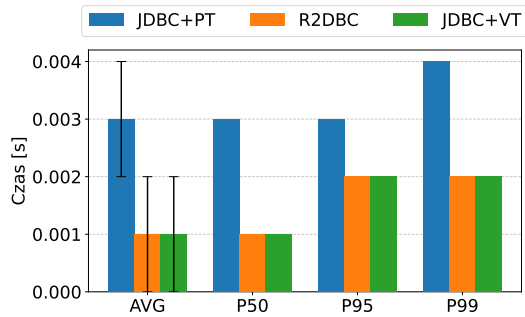
- (i) Wyniki we wspomnianych wyżej tabelach uwzględniają średni czas (AVG) i wartości percentyli P50, P95, P99 czasu wykonania badanych wariantów scenariusza i podane są w sekundach. Wykresy przedstawione na odpowiadających im rysunkach prezentują te dane na osi pionowej w jednostce oznaczonej jako [s].
- (ii) Wyniki we wierszach oznaczonych przez AVG we wspomnianych wyżej tabelach są przedstawione w formacie $S \pm E$, gdzie S oznacza uzyskany wynik, a E oznacza błąd pomiaru.
- (iii) Wyniki i wartości błędów we wspomnianych wyżej tabelach zapisane zostały z kropką jako separatorem dziesiętnym, z dokładnością do części tysięcznych, tj. 0.001.

Dla zachowania czytelności wyniki znajdują się na następnych stronach.

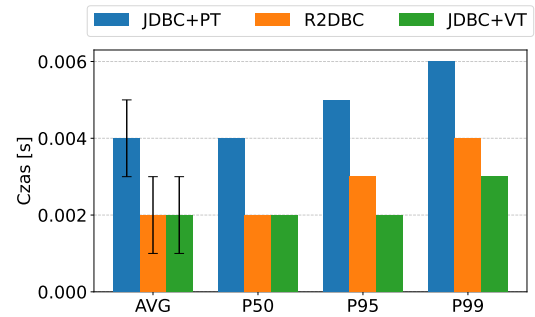
Poz. współb.	Metryka	JDBC+PT	R2DBC	JDBC+VT
16	AVG	0.003 ± 0.001	0.001 ± 0.001	0.001 ± 0.001
	P50	0.003	0.001	0.001
	P95	0.003	0.002	0.002
	P99	0.004	0.002	0.002
32	AVG	0.004 ± 0.001	0.002 ± 0.001	0.002 ± 0.001
	P50	0.004	0.002	0.002
	P95	0.005	0.003	0.002
	P99	0.006	0.004	0.003
64	AVG	0.009 ± 0.001	0.004 ± 0.001	0.004 ± 0.001
	P50	0.009	0.004	0.004
	P95	0.010	0.005	0.005
	P99	0.011	0.006	0.006
256	AVG	0.036 ± 0.001	0.015 ± 0.001	0.015 ± 0.001
	P50	0.035	0.015	0.015
	P95	0.039	0.017	0.018
	P99	0.043	0.020	0.020
1024	AVG	0.159 ± 0.001	0.058 ± 0.001	0.060 ± 0.001
	P50	0.158	0.057	0.058
	P95	0.177	0.065	0.066
	P99	0.187	0.070	0.071

Tabela 6.4: Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

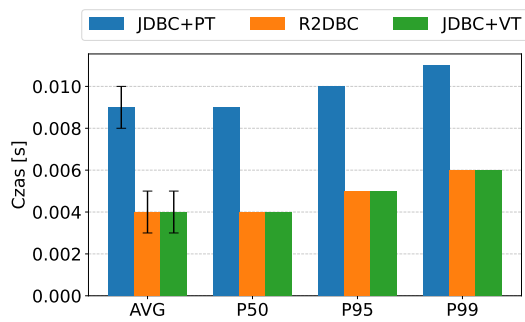
(źródło: opracowanie własne)



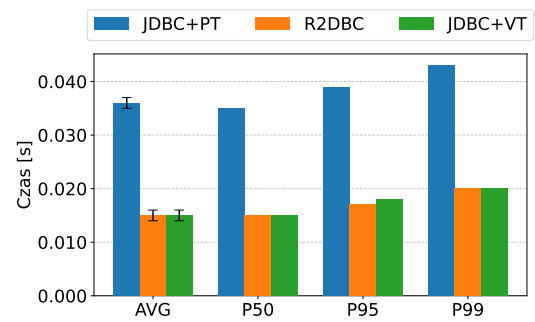
(a) poziom współbieżności: 16



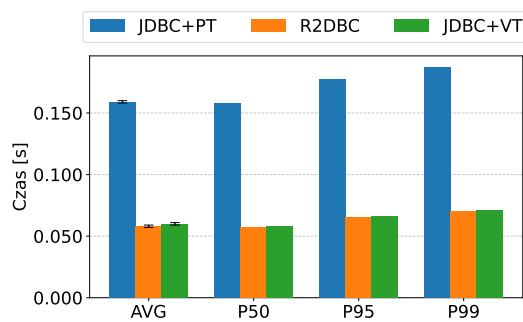
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

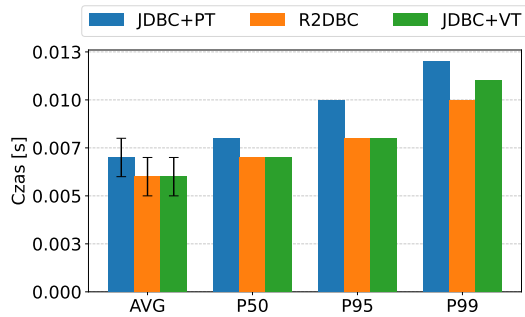
Rysunek 6.4: Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

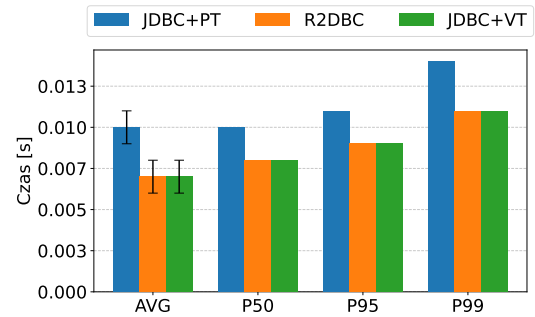
Poz. współb.	Metryka	JDBC+PT	R2DBC	JDBC+VT
16	AVG	0.007 ± 0.001	0.006 ± 0.001	0.006 ± 0.001
	P50	0.008	0.007	0.007
	P95	0.010	0.008	0.008
	P99	0.012	0.010	0.011
32	AVG	0.010 ± 0.001	0.007 ± 0.001	0.007 ± 0.001
	P50	0.010	0.008	0.008
	P95	0.011	0.009	0.009
	P99	0.014	0.011	0.011
64	AVG	0.019 ± 0.001	0.013 ± 0.001	0.014 ± 0.001
	P50	0.019	0.014	0.014
	P95	0.021	0.016	0.016
	P99	0.025	0.020	0.022
256	AVG	0.068 ± 0.001	0.054 ± 0.001	0.053 ± 0.001
	P50	0.071	0.054	0.052
	P95	0.077	0.059	0.056
	P99	0.096	0.081	0.079
1024	AVG	0.297 ± 0.003	0.214 ± 0.001	0.200 ± 0.002
	P50	0.299	0.212	0.202
	P95	0.329	0.236	0.223
	P99	0.364	0.264	0.257

Tabela 6.5: Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

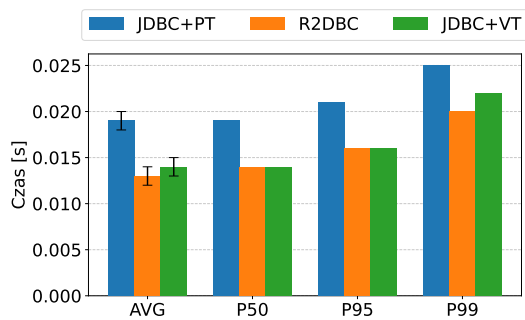
(źródło: opracowanie własne)



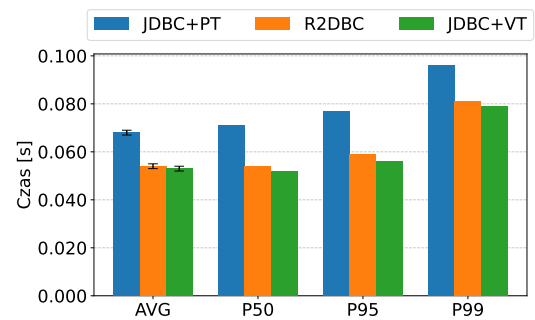
(a) poziom współbieżności: 16



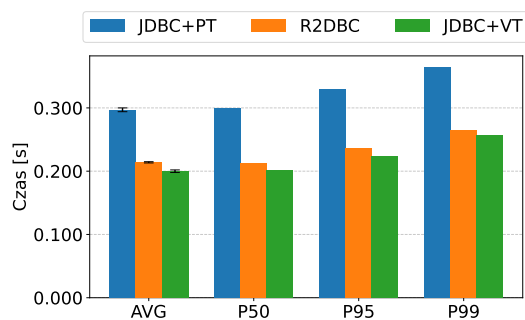
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

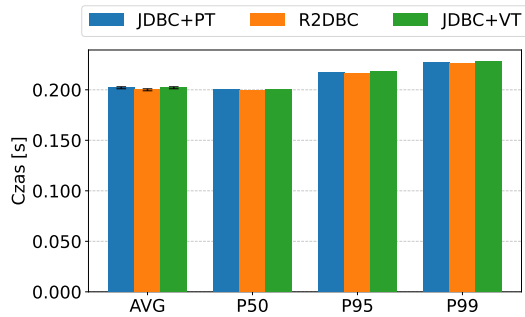
Rysunek 6.5: Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

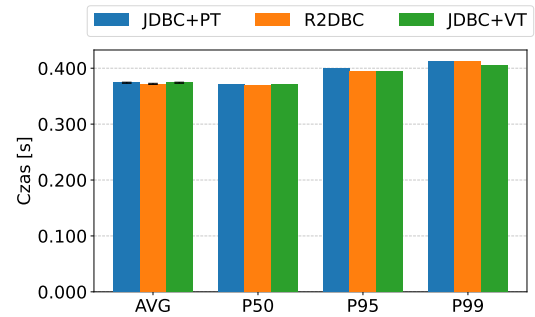
Poz. współb.	Metryka	JDBC+PT	R2DBC	JDBC+VT
16	AVG	0.202 ± 0.001	0.200 ± 0.001	0.202 ± 0.001
	P50	0.200	0.199	0.200
	P95	0.217	0.216	0.218
	P99	0.227	0.226	0.228
32	AVG	0.374 ± 0.001	0.372 ± 0.001	0.374 ± 0.001
	P50	0.371	0.369	0.372
	P95	0.400	0.394	0.394
	P99	0.412	0.412	0.406
64	AVG	0.757 ± 0.003	0.751 ± 0.003	0.760 ± 0.003
	P50	0.753	0.748	0.755
	P95	0.784	0.778	0.793
	P99	0.820	0.820	0.830
256	AVG	3.023 ± 0.012	2.999 ± 0.016	3.007 ± 0.013
	P50	3.012	2.991	3.001
	P95	3.091	3.078	3.095
	P99	3.176	3.311	3.148
1024	AVG	5.336 ± 0.010	11.954 ± 0.054	5.206 ± 0.009
	P50	5.335	11.929	5.201
	P95	5.377	12.148	5.234
	P99	5.385	12.231	5.260

Tabela 6.6: Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

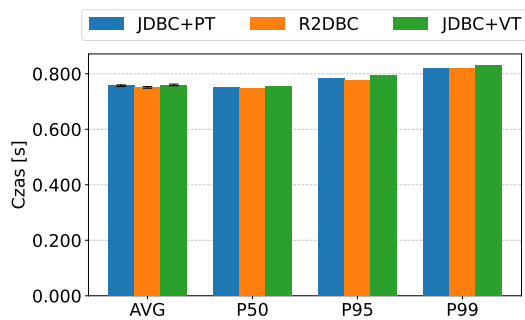
(źródło: opracowanie własne)



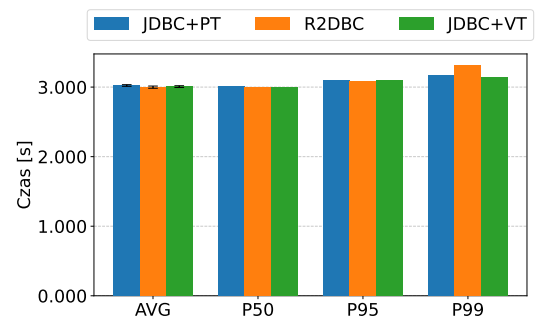
(a) poziom współbieżności: 16



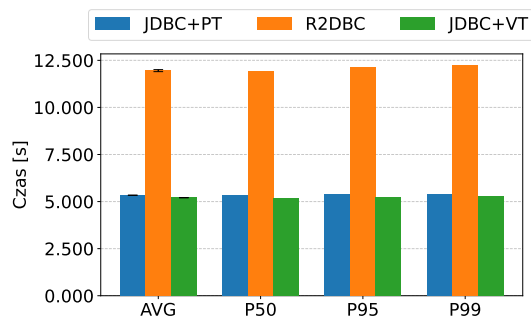
(b) poziom współbieżności: 32



(c) poziom współbieżności: 64



(d) poziom współbieżności: 256



(e) poziom współbieżności: 1024

Rysunek 6.6: Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

6.1.2 Wyniki pomiarów w ramach scenariusza $\mathcal{B}$

Tryb próbkowania czasu (`SampleTime`)

Tabela 6.7 oraz odpowiadające jej wykresy przedstawione na rysunku 6.7 prezentują wyniki benchmarku przeprowadzonego w ramach scenariusza $\mathcal{B}$ w trybie próbkowania czasu, tj. w trybie `SampleTime`. Wartości przedstawione w tabeli i na odpowiadających jej wykresach zostały wyznaczone w zależności od wariantu modelu współbieżności i konfiguracji serwisów.

Dodatkowe uwagi do tabeli 6.7:

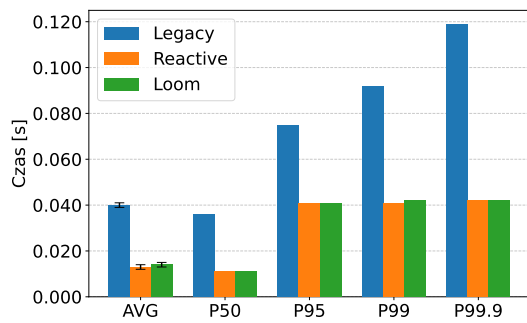
- (i) Wyniki we wspomnianej wyżej tabeli uwzględniają średni czas (AVG) i wartości percentyli P50, P95, P99, P99.9 czasu wykonania badanych wariantów scenariusza i podane są w sekundach. Wykresy przedstawione na odpowiadającym jej rysunku prezentują te dane na osi pionowej w jednostce oznaczonej jako [s].
- (ii) Wyniki we wierszach oznaczonych przez AVG we wspomnianej wyżej tabeli są przedstawione w formacie $S \pm E$, gdzie S oznacza uzyskany wynik, a E oznacza błąd pomiaru.
- (iii) Wyniki i wartości błędów we wspomnianej wyżej tabeli zapisane zostały z kropką jako separatorem dziesiętnym, z dokładnością do części tysięcznych, tj. 0.001.

Dla zachowania czytelności wyniki znajdują się na następnych stronach.

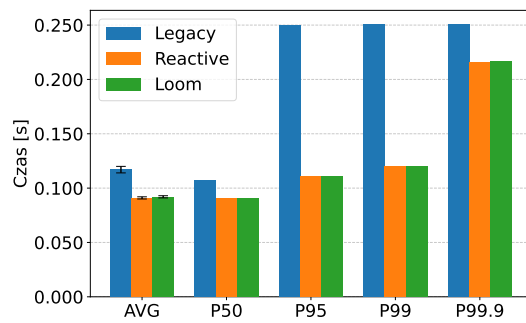
Konfig. serwisów	Metryka	Legacy	Reactive	Loom
THREE_FAST_TWO_SLOW	AVG	0.040 ± 0.001	0.013 ± 0.001	0.014 ± 0.001
	P50	0.036	0.011	0.011
	P95	0.075	0.041	0.041
	P99	0.092	0.041	0.042
	P99.9	0.119	0.042	0.042
ONE_FAST_FOUR_SLOW	AVG	0.117 ± 0.003	0.091 ± 0.001	0.092 ± 0.001
	P50	0.107	0.091	0.091
	P95	0.250	0.111	0.111
	P99	0.251	0.120	0.120
	P99.9	0.252	0.216	0.217
FIVE_SIMILAR	AVG	0.053 ± 0.001	0.039 ± 0.001	0.039 ± 0.001
	P50	0.045	0.038	0.039
	P95	0.150	0.045	0.045
	P99	0.151	0.048	0.048
	P99.9	0.153	0.051	0.052
OUTLIER_HEAVY	AVG	0.166 ± 0.010	0.050 ± 0.003	0.051 ± 0.003
	P50	0.058	0.037	0.037
	P95	0.301	0.300	0.300
	P99	0.324	0.301	0.301
	P99.9	0.341	0.302	0.302

Tabela 6.7: Wartości średniego czasu i percentyli P50, P95, P99, P99.9 dla scenariusza $\mathcal{B}$ w zależności od konfiguracji serwisów i modelu współbieżności

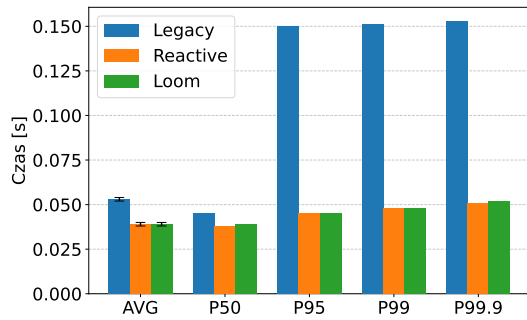
(źródło: opracowanie własne)



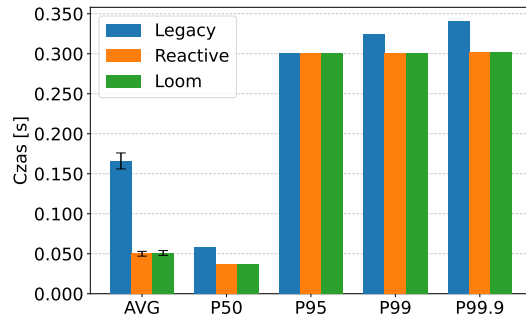
(a) konfiguracja THREE_FAST_TWO_SLOW



(b) konfiguracja ONE_FAST_FOUR_SLOW



(c) konfiguracja FIVE_SIMILAR



(d) konfiguracja OUTLIER_HEAVY

Rysunek 6.7: Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99, P99.9 dla scenariusza $\mathcal{B}$ w zależności od konfiguracji serwisów i modelu współbieżności

(źródło: opracowanie własne)

6.2 Analiza uzyskanych wyników

6.2.1 Scenariusz A

Zapytanie Q1 (proste pobranie danych po kluczu głównym)

Warianty JDBC+VT i R2DBC osiągnęły zbliżone wartości przepustowości na każdym poziomie współbieżności, wyraźnie wyprzedzając wariant JDBC+PT: przy poziomie współbieżności równym 16 o około 76%, zaś przy poziomie równym 1024 o około 151%. Różnice między wariantami JDBC+VT a R2DBC mieszczą się w granicach błędów pomiarowych. Wyniki uzyskane w ramach trybu próbkowania czasu (`SampleTime`) potwierdzają tę obserwację, tj. przy poziomie współbieżności równym 1024 warianty JDBC+VT i R2DBC uzyskały wartości średniego czasu o około 63% mniejsze niż wariant JDBC+PT. Ponadto wartości percentyli P50, P95 i P99 dla wariantów JDBC+VT i R2DBC są zbliżone i wyraźnie mniejsze w porównaniu do wariantu JDBC+PT.

Zapytanie Q2 (prosta zmiana wartości po kluczu głównym)

Wyniki dla zapytania Q2 wykazują podobny trend do wyników zapytania Q1, lecz przy wyższej zmienności, tj. przy niskich poziomach współbieżności błędy pomiaru osiągają wysoką wartość. Przy wyższych poziomach wyniki stabilizują się, tj. wariant JDBC+VT osiągnął najwyższą przewagę przy poziomach współbieżności równych 32 i 1024. Wyniki uzyskane w ramach trybu próbkowania czasu pokrywają się z tą obserwacją. We wszystkich metrykach wariant JDBC+VT uzyskał wyniki nieznacznie lepsze od wariantu R2DBC, a oba przewyższają wariant JDBC+PT na każdym badanym poziomie współbieżności.

Zapytanie Q3 (złożone zapytanie z agregacją i wielokrotnym złączeniem)

Przy poziomach współbieżności od 16 do 256 wyniki wszystkich trzech wariantów są zbliżone, mieszcząc się w granicach błędów pomiaru. Przy poziomie równym 1024 pojawia się wyraźna anomalia: przepustowość wariantu R2DBC jest ponad dwukrotnie niższa w porównaniu do pozostałych wariantów, a w trybie próbkowania czasu wyniki wskazują na ponad dwukrotny wzrost wartości średniego czasu i percentyli względem wariantów JDBC+PT i JDBC+VT. Jest to jedyna konfiguracja w scenariuszu A, w której wariant R2DBC uzyskał wyniki istotnie gorsze od wariantów opartych na sterowniku JDBC.

6.2.2 Scenariusz $\mathcal{B}$

We wszystkich czterech konfiguracjach serwisów warianty **Reactive** i **Loom** uzyskiwały niemal identyczne wyniki, a różnice między nimi mieszczą się w granicach błędu pomiarowego. Wariant **Legacy** był wolniejszy od pozostałych wariantów we wszystkich konfiguracjach, przy czym skala dysproporcji różni się w zależności od konfiguracji serwisów:

- w konfiguracji **THREE_FAST_TWO_SLOW** średnio trzykrotnie wolniejszy,
- w konfiguracji **ONE_FAST_FOUR_SLOW** średnio około 28% wolniejszy (najmniejsza różnica),
- w konfiguracji **FIVE_SIMILAR** średnio około 36% wolniejszy,
- w konfiguracji **OUTLIER_HEAVY** średnio ponad trzykrotnie wolniejszy (największa różnica).

We wszystkich czterech konfiguracjach percentyle P95, P99 i P99.9 dla wariantu **Legacy** są istotnie wyższe niż dla wariantów **Reactive** i **Loom**, a ich wartości skupiają się blisko wartości L_{outlier} dla najwolniejszego serwisu w danej konfiguracji. Ponadto konfiguracja **OUTLIER_HEAVY** wyróżnia się istotną rozbieżnością między wartością percentyla P50 a wartością AVG dla wariantu **Legacy**.

Rozdział 7

Dyskusja i wnioski

Siódmy rozdział pracy zawiera dyskusję wyników uzyskanych w ramach przeprowadzonych badań. Obejmuje on interpretację wyników dla obu scenariuszy badawczych, ocenę złożoności implementacyjnej porównywanych modeli współbieżności oraz weryfikację hipotez badawczych sformułowanych w rozdziale piątym. Rozdział zawiera również rekomendacje dotyczące doboru mechanizmów Project Loom w praktycznych zastosowaniach, omawia ograniczenia przeprowadzonych badań oraz wskazuje kierunki dalszych badań.

7.1 Interpretacja uzyskanych wyników

Poniżej została zawarta interpretacja przyczyn zaobserwowanych różnic wydajnościowych pomiędzy porównywanymi wariantami w ramach obu scenariuszy badawczych. Analiza opiera się na wynikach pomiarów przedstawionych w rozdziale szóstym oraz na teoretycznych właściwościach porównywanych mechanizmów opisanych w rozdziałach drugim i trzecim.

7.1.1 Scenariusz $\mathcal{A}$

Wątki wirtualne a wątki platformowe

Wyraźna degradacja przepustowości wariantu JDBC+PT wraz ze wzrostem poziomu współbieżności wynika z właściwości modelu opartego na wątkach platformowych, tj. odwzorowania na wątki systemu operacyjnego w relacji jeden do jednego [3]. Każde wywołanie JDBC blokuje wykonywany wątek na czas trwania całej operacji, na którą składa się głównie nawiązanie połączenia, przesłanie zapytania i odebranie odpowiedzi od serwera bazodanowego. W tym czasie wątek systemu operacyjnego pozostaje w stanie bezczynności, nie wykonując żadnej pracy, przy tym zajmując zasoby [3, 4]. Przy rosnącej liczbie równoległych żądań rośnie zarówno łączne zużycie tych zasobów, jak i częstotliwość wymuszonego przełączania

kontekstu pomiędzy wątkami systemu operacyjnego, co powoduje dodatkowy narzut obniżający efektywność wykonania [6].

Wariant JDBC+VT unika tych problemów dzięki mechanizmowi dynamicznego przełączania wykonywania operacji przez wątki nośne w momencie wykonywania operacji blokujących przez wątki wirtualne, co zostało szczegółowo opisane w rozdziale trzecim, w sekcji 3.2.1. W rezultacie liczba wątków nośnych, a tym samym wątków systemu operacyjnego, aktywnie zarządzanych przez planistę wątków w środowisku JVM pozostaje stała i jest równa liczbie dostępnych procesorów logicznych, niezależnie od poziomu współbieżności aplikacji [11, 9]. W użytym środowisku testowym przekłada się to na 12 wątków nośnych obsługujących do 1024 wątków wirtualnych. Eksperyment przeprowadzony w ramach scenariusza $\mathcal{A}$ wykazał, że wspomniany wyżej mechanizm dynamicznego przełączania wykonywania operacji przez wątki nośne ma istotny wpływ na wzrost przepustowości w porównaniu do analogicznego podejścia opartego na wątkach platformowych, co jest zgodne z ideą przyświecającą wprowadzeniu tego mechanizmu [10].

Konfiguracja bazy danych z ograniczeniem maksymalnej liczby połączeń do 32 stanowi kluczowy czynnik zwiększający obserwowane różnice między wariantami. Dla wszystkich poziomów współbieżności przekraczających tę wartość istnieje co najmniej jedno żądanie, które przed rozpoczęciem przetwarzania oczekuje na zwolnienie połączenia. Dla wariantu JDBC+PT czas oczekiwania jest tożsamy z całkowitym czasem blokowania wątku platformowego, gdzie wątek systemu operacyjnego pozostaje w stanie zawieszenia przy jednoczesnym zajmowaniu zasobów planisty. Dla wariantu JDBC+VT to samo oczekiwanie wiąże się jedynie z parkowaniem wątku wirtualnego, a zwolniony wątek nośny może w tym samym czasie realizować inne żądania. Dla zapytań Q1 i Q2 wzrost przepustowości wariantu JDBC+VT w porównaniu do wariantu JDBC+PT oraz spadek średniego czasu przetwarzania operacji i wartości percentyli przy jednoczesnym wzroście poziomu współbieżności są zgodne z opisanymi w literaturze obserwacjami. Zgodnie z publikacją Pandity [54], wirtualne wątki wykazują wzrost przepustowości i redukcję latencji w warunkach wysokiej współbieżności dla operacji wejścia-wyjścia.

Wątki wirtualne a podejście reaktywne

Wyniki uzyskane w ramach wariantów JDBC+VT i R2DBC są zbliżone dla zapytań Q1 i Q2 na wszystkich badanych poziomach współbieżności. Obie implementacje osiągają porównywalną efektywność. Wynika to z faktu, że oba podejścia, w przeciwieństwie do wariantu JDBC+PT, nie blokują wątków systemu operacyjnego podczas wykonywania operacji. Wariant R2DBC wykorzystuje w tym celu nieblokujący protokół sieciowy oparty na bibliotece Netty [59]. Wariant JDBC+VT osiąga ten sam efekt za pomocą wspomnianego wyżej mechanizmu parkowania wirtualnych wątków.

Zbliżone wyniki obu wariantów w tego rodzaju obciążeniach zaobserwowali również Ponnampalam [14] oraz Gustafsson i Persson [15].

Jedyny wyjątek od opisanej zależności stanowi wynik wariantu R2DBC dla zapytania Q3 przy poziomie współbieżności równym 1024. W tym przypadku średni czas odpowiedzi był ponad dwukrotnie wyższy niż dla wariantów JDBC+PT i JDBC+VT przy jednoczesnym dwukrotnym spadku przepustowości. Warto jednak zauważyć, że dla poziomów współbieżności od 16 do 256 wszystkie trzy warianty osiągały niemal identyczne wyniki. Sugeruje to, że pogorszenie wydajności nie wynika z charakterystyki samego zapytania, lecz z zachowania poszczególnych modeli przy bardzo wysokim obciążeniu i zwiększonej walki o zasoby.

Zbliżone wyniki wariantów JDBC+PT i JDBC+VT przy poziomie współbieżności równym 1024 wskazują, że głównym ograniczeniem obu podejść był maksymalny rozmiar puli połączeń do bazy danych, a nie sposób realizacji współbieżności. Pogorszenie wyników wariantu R2DBC można natomiast wiązać ze specyfiką reaktywnego przetwarzania danych. W modelu reaktywnym kolejne wiersze wyniku są propagowane jako osobne zdarzenia w strumieniu danych. Przy 1024 równolegle wykonywanych instancjach złożonego zapytania Q3 liczba zdarzeń obsługiwanych jednocześnie przez planistę biblioteki Reactor mogła prowadzić do wzrostu opóźnień związanych z kolejkowaniem i planowaniem zadań [7]. Warianty oparte na sterowniku blokującym JDBC przetwarzają natomiast wyniki sekwencyjnie, w obrębie pojedynczego wątku i przy użyciu blokującego mechanizmu `ResultSet`, dzięki czemu nie obciążają współdzielonego mechanizmu wykonawczego zadań. Podobne zjawiska spadku wydajności systemów reaktywnych przy intensywnym przetwarzaniu wyników złożonych zapytań i wysokiej współbieżności opisuje również Shahoor [8].

7.1.2 Scenariusz *B*

Zbliżona wydajność wariantów Loom i Reactive we wszystkich analizowanych konfiguracjach serwisów wynika z faktu, że oba modele realizują ten sam schemat obsługi mechanizmu kworum. W obu przypadkach wszystkie pięć serwisów jest wywoływanych równolegle, a operacja kończy się natychmiast po uzyskaniu wymaganych trzech odpowiedzi wraz z anulowaniem pozostałych żądań. Wariant Reactive realizuje ten mechanizm za pomocą operatora agregującego wiele strumieni i zwracającego pierwsze k wyników, gdzie k jest wartością kworum, który po spełnieniu warunku kworum propaguje sygnał anulowania do pozostałych strumieni danych [32]. W wariancie Loom analogiczne zachowanie uzyskano przy użyciu klasy `StructuredTaskScope` oraz własnej implementacji interfejsu `Joiner` [43], wywołującej metodę zamykającą zakres zadań po odebraniu k -tej poprawnej odpo-

wiedzi. Operacja ta przerywa aktywne zadania potomne, a wirtualne wątki kończą wykonywanie po otrzymaniu sygnału przerwania [12]. W efekcie czas odpowiedzi obu wariantów jest wyznaczany przez moment uzyskania k -tej odpowiedzi spośród n równoległe wykonywanych wywołań, co odpowiada optymalnemu sposobowi realizacji mechanizmu kworum [11].

Słabsze wyniki wariantu **Legacy** wynikają natomiast z ograniczeń mechanizmu anulowania zadań w klasie **CompletableFuture**. Zgodnie z dokumentacją tej klasy [60], metoda `cancel`, odpowiedzialna za anulowanie zadania, nie gwarantuje natychmiastowego zatrzymania zadania wykonywanego przez pulę wątków. Anulowanie ma charakter sygnału, który może zostać obsłużony lub zignorowany przez wykonywany kod. W praktyce operacje wejścia-wyjścia nie zawsze reagują na przerwanie od razu, co opisuje również Goetz [4]. Oznacza to, że po uzyskaniu wymaganej wartości kworum zadania obsługujące wolniejsze serwisy mogą nadal zajmować wątki aż do naturalnego zakończenia operacji. W konsekwencji całkowity czas odpowiedzi może być zależny nie od czasu uzyskania k -tej odpowiedzi, lecz od czasu zakończenia wolniejszych żądań. Al-Dossari i Al-Fahad [56] wskazują, że ograniczona kontrola nad cyklem życia podzadań stanowi jedną z istotnych wad podejścia opartego na **CompletableFuture**.

Wynik uzyskany w ramach konfiguracji **OUTLIER_HEAVY** potwierdza powyższą obserwację. Znaczna różnica pomiędzy medianą (percentylem rzędu 50) a średnią dla wariantu **Legacy** wskazuje na silnie asymetryczny rozkład czasów odpowiedzi [66]. W większości przypadków kworum zostaje osiągnięte zanim najwolniejsze serwisy zwrócą odpowiedź, jednak pojawienie się opóźnień skrajnych powoduje wydłużenie czasu działania zadań, które nie zostały skutecznie anulowane. Zjawisko to jest widoczne szczególnie w wartościach percentyla rzędu 95 i wyższych. Warianty **Reactive** i **Loom** osiągają niższe wartości percentyli ogonowych, co wskazuje, że skuteczne anulowanie zbędnych zadań pozwala ograniczyć wpływ opóźnień skrajnych na całkowity czas odpowiedzi systemu.

7.2 Ocena złożoności implementacyjnej

Poniżej zawarto jakościową analizę porównawczą badanych modeli współbieżności pod kątem modelu programowania, zarządzania błędami i zasobami oraz integracji z istniejącymi mechanizmami dostępnymi w środowisku JVM. Ocena została oparta na analizie implementacji scenariuszy opisanych w rozdziale piątym, w sekcjach 5.3.1 i 5.3.2. W przeciwieństwie do analizy wydajnościowej przedstawionej wyżej, wnioski zawarte poniżej nie wynikają z pomiarów ilościowych ani statycznej analizy kodu, lecz z jakościowej oceny złożoności implementacji, czytelności kodu oraz sposobu zarządzania współbieżnością i zasobami.

7.2.1 Model programowania

Warianty JDBC+PT i JDBC+VT w scenariuszu $\mathcal{A}$ wykorzystują klasyczny imperatywny model programowania oparty na sekwencyjnym wykonywaniu instrukcji. Implementacje obu wariantów różnią się wyłącznie sposobem tworzenia mechanizmu wykonawczego zadań, opartym odpowiednio na pulach wątków platformowych (patrz linie 8.–9. w listingu 5.4) i wykorzystanie paradygmatu *thread-per-request* przy użyciu wątków wirtualnych (patrz linie 6.–7. w listingu 5.5). Tym samym migracja z wątków platformowych na wirtualne w większości przypadków nie wymaga zmian w logice biznesowej, co znacząco obniża koszt adaptacji istniejącego kodu.

Wariant R2DBC opiera się na modelu reaktywnym, w którym logika aplikacji wyrażana jest poprzez operacje na strumieniach danych, o czym wspomina Davis [5]. Model ten wymaga znajomości mechanizmów asynchronicznych i podejścia reaktywnego, w tym operatorów reaktywnych oraz modelu subskrypcji. Pomimo zwięzłości kodu, jego analiza i debugowanie są bardziej złożone niż w przypadku kodu imperatywnego [8, 14].

Różnice między modelami współbieżności są szczególnie widoczne w scenariuszu $\mathcal{B}$. Wariant **Legacy** oparty na mechanizmach dostępnych w ramach klasy `CompletableFuture` wymaga stosunkowo rozbudowanej logiki synchronizacji oraz ręcznego zarządzania przetwarzaniem wyników. Implementacja optymalnego mechanizmu kworum przy użyciu tego podejścia jest istotnie bardziej złożona niż w modelu reaktywnym lub przy użyciu mechanizmu współbieżności strukturalnej.

Wariant **Reactive** w scenariuszu $\mathcal{B}$ umożliwia naturalną implementację mechanizmu kworum przy użyciu operatorów reaktywnych, jednak kosztem większej złożoności poznawczej. Wariant **Loom** osiąga podobny efekt pod kątem wydajności i skalowalności przy zachowaniu imperatywnego modelu programowania. Klasa `StructuredTaskScope`, będąca częścią mechanizmu współbieżności strukturalnej, pozwala zachować czytelną strukturę kodu, a logika synchronizacji może zostać wydzielona do osobnych komponentów przy wykorzystaniu interfejsu `Joiner` [12, 43, 45].

7.2.2 Zarządzanie błędami i zasobami

Warianty wykorzystujące blokujący sterownik JDBC w scenariuszu $\mathcal{A}$ opierają obsługę błędów na standardowych mechanizmach języka Java. Odmianą charakterystykę posiada podejście reaktywne, w którym diagnostyka błędów jest bardziej złożona ze względu na sposób propagacji operacji asynchronicznych. Generowane ślady stosu (ang. *stack trace*) zawierają liczne wywołania wewnętrzne biblioteki, które utrudniają identyfikację rzeczywistego miejsca wystąpienia błędu w ko-

dzie [8]. Dodatkowym utrudnieniem są przełączenia wątków pomiędzy operatorami reaktywnymi, co znacznie utrudnia odtworzenie pełnej ścieżki wykonania. Należy jednak zaznaczyć, że biblioteki reaktywne udostępniają mechanizmy poprawiające czytelność diagnostyki, choć ich wykorzystanie wiąże się z dodatkowym narzutem wydajnościowym [5, 67].

W scenariuszu **B** wariant **Legacy** wymaga ręcznego anulowania zadań oraz samodzielnego zarządzania cyklem życia operacji współbieżnych (patrz linia 13. w listingu 5.9). Istotnym ograniczeniem klasy **CompletableFuture**, wykorzystanej w tym wariantcie, jest brak gwarancji skutecznego anulowania wykonywanego zadania. Wywołanie metody **cancel** powoduje jedynie wysłanie sygnału przerwania do wątku wykonawczego, natomiast faktyczne zatrzymanie zadania zależy od implementacji wykonywanego kodu oraz sposobu obsługi przerwania [60, 4].

Mechanizm współbieżności strukturalnej zapewnia bardziej spójny model zarządzania cyklem życia zadań. Klasa **StructuredTaskScope** automatycznie synchronizuje zakończenie podzadań oraz propaguje anulowanie i wyjątki zgodnie z przyjętą strategią synchronizacji [12]. Ogranicza to ryzyko pozostawienia aktywnych zadań po zakończeniu operacji nadrzędnej i upraszcza implementację mechanizmów współbieżnych.

7.2.3 Integracja z istniejącymi mechanizmami dostępnymi w środowisku JVM

Warianty **JDBC+PT** i **JDBC+VT** korzystają ze standardowego API sterownika **JDBC**, które posiada szerokie wsparcie w bibliotekach oraz frameworkach dostępnych w środowisku JVM. Wirtualne wątki zachowują pełną kompatybilność z tym modelem, co oznacza brak konieczności modyfikacji warstwy aplikacyjnej [10].

Należy jednak uwzględnić ograniczenia wynikające z implementacji sterowników **JDBC** w starszych wersjach, które korzystają z mechanizmów synchronizacji opartych na słowie kluczowym **synchronized** w obrębie operacji wejścia-wyjścia. W tym wypadku może dojść do zjawiska przypisania wątku wirtualnego do wątku nośnego bez możliwości zmiany tego stanu, co zaburza cykl życia wirtualnych wątków opisany w rozdziale trzecim, w sekcji 3.2.1.

Wariant **R2DBC** wymaga zastosowania dedykowanych sterowników reaktywnych i nie zapewnia bezpośredniej kompatybilności z bibliotekami lub frameworkami opartymi na blokujących API [59]. Kluczowym ograniczeniem tego podejścia jest również konieczność utrzymania spójnego modelu reaktywnego w całym stosie aplikacyjnym. Jak wskazuje Shahoor [8], łączenie modeli blokującego i reaktywnego może skutkować trudnymi do diagnozy problemami wydajnościowymi.

Mechanizm współbieżności strukturalnej zachowuje kompatybilność z istniejącym ekosystemem dzięki oparciu o wątki wirtualne i blokujący model wykonania, co pozwala na wykorzystanie istniejących bibliotek. Należy jednak uwzględnić fakt, że mechanizm współbieżności strukturalnej w JDK 25 dostępny jest w wersji poglądowej, co znacząco ogranicza jego zastosowanie w środowiskach produkcyjnych wymagających pełnej stabilności [12].

7.3 Weryfikacja hipotez badawczych

Poniżej przedstawiono ocenę każdej z hipotez badawczych sformułowanych w podrozdziale 5.2 na podstawie wyników pomiarów zaprezentowanych w podrozdziale 6.1 i przeanalizowanych w podrozdziale 7.1 oraz jakościowej oceny złożoności implementacyjnej przeprowadzonej w podrozdziale 7.2.

Hipotezy szczegółowe dla scenariusza $\mathcal{A}$

H1. *Zastosowanie wirtualnych wątków w przypadku operacji wejścia-wyjścia opartych na komunikacji z bazą danych pozwala osiągnąć wyższą przepustowość niż podejście oparte na wątkach platformowych.*

Hipoteza potwierdzona: Wariant JDBC+VT przewyższył wariant JDBC+PT o około 76% przy poziomie współbieżności równym 16 i o około 151% przy poziomie współbieżności równym 1024 w pomiarze przepustowości dla zapytania Q1. Analogiczna tendencja zaobserwowana została dla zapytania Q2. Różnice wynikają z ograniczenia blokowania wątków systemu operacyjnego podczas oczekiwania na odpowiedź bazy danych, opisanej szczegółowo w podrozdziale 6.2.

H2. *Wraz ze wzrostem poziomu współbieżności rośnie różnica w wydajności pomiędzy podejściem opartym na wątkach platformowych a wątkach wirtualnych, szczególnie dla operacji o charakterze blokującym.*

Hipoteza potwierdzona: Wraz ze wzrostem poziomu współbieżności iloraz przepustowości w wariantach JDBC+VT do wariantu JDBC+PT dla zapytania Q1 również wzrastał, co potwierdza systematyczne rozszerzanie się różnicy wydajnościowej wraz ze wzrostem obciążenia.

H3. *Podejście wykorzystujące reaktywny sterownik dostępu do bazy danych umożliwia osiągnięcie wysokiej przepustowości i niskich czasów odpowiedzi, jednak wiąże się z większą złożonością implementacyjną niż rozwiązania wykorzystujące wirtualne wątki.*

Hipoteza częściowo potwierdzona: Wydajność wariantu R2DBC była porównywalna do wariantu JDBC+VT dla zapytań Q1 i Q2. Dla zapytania Q3 przy poziomie współbieżności równym 1024 wariant R2DBC uzyskał wyniki istotnie gorsze od obu wariantów opartych na blokującym sterowniku JDBC, co przeczy tezie o konsekwentnej wysokiej przepustowości i niskich czasach odpowiedzi podejścia reaktywnego. Wyższy poziom złożoności implementacyjnej wariantu R2DBC względem wariantu JDBC+VT potwierdzają natomiast w pełni wnioski jakościowej analizy zawartej w podrozdziale 7.2, w szczególności w zakresie reaktywnego modelu programowania, nieczytelności śladów stosu oraz konieczności utrzymania spójnego modelu reaktywnego w całym stosie aplikacyjnym.

- H4.** *Zastosowanie wirtualnych wątków w połączeniu z blokującym sterownikiem dostępu do bazy danych pozwala osiągnąć wydajność porównywalną do podejścia reaktywnego przy zachowaniu prostszego, imperatywnego modelu programowania.*

Hipoteza potwierdzona: Dla zapytania Q1 wyniki wariantów JDBC+VT i R2DBC były statystycznie nierozróżnialne na wszystkich poziomach współbieżności. Dla zapytania Q2 wariant JDBC+VT osiągnął wyniki nieznacznie wyższe od R2DBC. Dla zapytania Q3 przy poziomie współbieżności równym 1024 wariant JDBC+VT przewyższył wariant R2DBC ponad dwukrotnie w pomiarze wartości średniego czasu. Prostszy, imperatywny model programowania wariantu JDBC+VT i niższy poziom złożoności implementacyjnej przy porównywalnej lub wyższej wydajności potwierdzają zakładaną przewagę wirtualnych wątków nad podejściem reaktywnym, co szczegółowo uzasadniono w podrozdziale 7.2.

Hipotezy szczegółowe dla scenariusza B

- H5.** *Mechanizm współbieżności strukturalnej upraszcza implementację scenariuszy równoległej agregacji wyników względem podejścia reaktywnego oraz klasycznego modelu opartego na pulach wątków.*

Hipoteza potwierdzona: Analiza jakościowa przeprowadzona w podrozdziale 7.2 wykazała, że implementacja mechanizmu kworum przy użyciu mechanizmu współbieżności strukturalnej cechuje się czytelniejszą strukturą i wymaga mniejszej liczby elementów synchronizacyjnych niż wariant Legacy. W porównaniu do wariantu Reactive podejście zastosowane w wariancie Loom zachowuje imperatywny model programowania i unika reaktywnej złożoności poznawczej. Wariant Loom osiągnął ponadto wydajność porównywalną

z wariantem **Reactive** i wyraźnie lepszą niż wariant **Legacy** we wszystkich konfiguracjach serwisów scenariusza $\mathcal{B}$.

- H6.** *W wariantach scenariusza obejmujących serwisy o wysokich opóźnieniach mechanizmy Project Loom umożliwiają osiągnięcie czasów odpowiedzi porównywalnych z modelem reaktywnym.*

Hipoteza potwierdzona: Wariant Loom osiągnął wyniki zbliżone do wariantu **Reactive** we wszystkich czterech konfiguracjach serwisów scenariusza $\mathcal{B}$, w tym w konfiguracji **OUTLIER_HEAVY**, charakteryzującej się najwyższymi opóźnieniami i największą zmiennością czasów odpowiedzi.

- H7.** *Współbieżność strukturalna zapewnia bardziej przewidywalny model zarządzania cyklem życia zadań oraz propagacją błędów niż klasyczne podejścia wykorzystujące natywne rozwiązania lub programowanie reaktywne.*

Hipoteza potwierdzona: Wariant Loom wykazał niższą i stabilniejszą latencję ogonową niż wariant **Legacy** we wszystkich konfiguracjach. Skuteczność mechanizmu zarządzania cyklem życia zadań w klasie **StructuredTaskScope** stanowi istotną przewagę nad natywnymi mechanizmami oferowanymi przez klasę **CompletableFuture**, której metoda **cancel** nie gwarantuje natychmiastowego zatrzymania wykonywanego zadania. W porównaniu do modelu reaktywnego mechanizm współbieżności strukturalnej oferuje bardziej deterministyczny i czytelny model obsługi błędów, co zostało uzasadnione w podrozdziale 7.2.

Hipotezy szczegółowe dotyczące modelu programowania

- H8.** *Podejście reaktywne charakteryzuje się wyższym poziomem złożoności implementacyjnej i większą trudnością analizy przepływu wykonania niż podejścia imperatywne oparte na wątkach platformowych lub wirtualnych.*

Hipoteza potwierdzona: Analiza złożoności implementacyjnej przedstawiona w podrozdziale 7.2 wykazała, że model reaktywny wymaga znajomości operatorów i asynchronicznego modelu subskrypcji, a generowane ślady stosu zawierają ramki wewnętrzne biblioteki utrudniające ich czytelność i skuteczność znalezienia faktycznej przyczyny problemu. Implementacje oparte na wątkach platformowych i wirtualnych zachowują sekwencyjny przepływ wykonania i standardowe ślady stosu, co istotnie upraszcza analizę i debugowanie kodu.

- H9.** *Mechanizmy oferowane przez Project Loom umożliwiają implementację aplikacji współbieżnych z wykorzystaniem modelu programowania bliższego sekwen-*

cyjnemu przepływowi wykonania, co upraszcza implementację i analizę kodu.

Hipoteza potwierdzona: W scenariuszu $\mathcal{A}$ wariant JDBC+VT różni się od wariantu JDBC+PT wyłącznie sposobem tworzenia mechanizmu wykonawczego zadań, co potwierdza, że zastosowanie mechanizmu wirtualnych wątków nie wymaga modyfikacji logiki biznesowej. Wariant Loom w scenariuszu $\mathcal{B}$ implementuje mechanizm kworum w imperatywnym stylu, zachowując sekwencyjny przepływ sterowania i standardową obsługę wyjątków, przy jednoczesnym zachowaniu pełnej współbieżności wykonania.

Na podstawie przeprowadzonej weryfikacji hipotez szczegółowych można stwierdzić, że hipoteza główna została **potwierdzona**, tj. mechanizmy oferowane przez Project Loom, w szczególności wirtualne wątki i współbieżność strukturalna, umożliwiają osiągnięcie wydajności porównywalnej lub wyższej względem istniejących modeli współbieżności, przy jednoczesnym zachowaniu imperatywnego modelu programowania i uproszczeniu implementacji wybranych scenariuszy współbieżnych.

7.4 Rekomendacje i wzorce zastosowań mechanizmów Project Loom

Na podstawie wyników badań przedstawionych w rozdziale szóstym oraz jakościowej analizy złożoności implementacyjnej przeprowadzonej w podrozdziale 7.2 sformułowano poniżej praktyczne rekomendacje dotyczące doboru modelu współbieżności w zależności od charakterystyki problemu, wymagań systemowych i kontekstu projektowego, ze szczególnym uwzględnieniem badanych mechanizmów współbieżności oferowanych przez Project Loom, tj. wątków wirtualnych i współbieżności strukturalnej.

7.4.1 Zastosowania wirtualnych wątków

Wyniki uzyskane w ramach scenariusza $\mathcal{A}$ wskazują, że wirtualne wątki stanowią naturalne i mało kosztowne usprawnienie klasycznego modelu opartego na puli wątków platformowych w przypadku operacji o charakterze blokującym. Migracja z wariantu opartego na puli wątków platformowych do podejścia wykorzystującego wątki wirtualne wymagała zmiany wyłącznie jednej linii kodu, co przełożyło się na wyraźny wzrost przepustowości bez modyfikacji logiki biznesowej.

Wirtualne wątki są zatem szczególnie wskazane w systemach o wysokiej współbieżności zadań realizujących operacje wejścia-wyjścia, takich jak zapytania do baz

danych, wywołania zewnętrznych serwisów lub operacje na systemie plików, w których dotychczasowym rozwiązaniem była pula wątków platformowych. Zastąpienie jej poprzez zastosowanie paradygmatu *thread-per-request* w oparciu o wątki wirtualne pozwala osiągnąć znaczące korzyści skalowania bez ponoszenia kosztów związanych ze zrozumieniem programowania reaktywnego i ograniczeń z tym związanych.

Ponadto klasyczna pula wątków platformowych pozostaje właściwym wyborem dla obciążeń o charakterze obliczeniowym, gdzie czas przetwarzania przewyższa czas oczekiwania na operacje wejścia-wyjścia, a liczba aktywnych wątków jest niska i kontrolowana. W takich przypadkach wirtualne wątki nie przynoszą wymiernych korzyści, natomiast pula wątków pozostaje rozwiązaniem sprawdzonym i dobrze zrozumianym.

Przed migracją do wątków wirtualnych należy jednak zweryfikować kompatybilność używanych zależności z mechanizmem parkowania wątków wirtualnych. Biblioteki korzystające wewnętrznie z blokad opartych na słowie kluczowym **synchronized** w obrębie operacji wejścia-wyjścia mogą powodować blokowanie powiązania wątku wirtualnego z wątkiem nośnym, co niweluje korzyści płynące z ich stosowania.

Jak wskazuje przegląd literatury przytoczony w rozdziale czwartym, wirtualne wątki nie przynoszą wymiernych korzyści w przypadku obciążeń o charakterze obliczeniowym, wykorzystujących w głównej mierze procesor (ang. *CPU-bound*). W takich scenariuszach czynnikiem ograniczającym jest liczba dostępnych procesorów logicznych, a nie blokowanie wątków systemowych, wobec czego liczba aktywnych wątków nośnych odpowiada liczbie procesorów logicznych, niezależnie od zastosowanego modelu.

Ponadto wirtualne wątki mogą być rozważane jako alternatywa w nowych projektach, w których model reaktywny byłby stosowany wyłącznie ze względu na potrzebę obsługi wysokiej współbieżności operacji wejścia-wyjścia, bez faktycznej potrzeby przetwarzania strumieni z mechanizmem backpressure.

7.4.2 Zastosowania współbieżności strukturalnej

Wyniki scenariusza *B* wskazują, że mechanizm współbieżności strukturalnej stanowi właściwy wybór dla wzorców równoległego pobierania danych z wielu źródeł, w szczególności dla mechanizmu kworum oraz scenariuszy, w których niezbędne jest niezawodne zarządzanie cyklem życia podzadań. Wariant Loom osiągnął wydajność porównywalną z modelem reaktywnym przy istotnie prostszej implementacji, zachowując przy tym imperatywny model programowania.

Klasa **StructuredTaskScope** z interfejsem **Joiner** umożliwia naturalne wyrażenie logiki agregacji w sekwencyjnym stylu, z automatyczną gwarancją zakończenia

podzadań i strukturalną propagacją wyjątków. Podejście to eliminuje potrzebę ręcznego zarządzania cyklem życia obiektów, charakterystyczną dla wariantu **Legacy**, i zastępuje ją zakresem zadań z gwarancją braku aktywnych podzadań po opuszczeniu tego zakresu. Al-Dossari i Al-Fahad [56] wskazują, że ograniczona kontrola cyklu życia zadań jest jedną z kluczowych wad podejścia opartego na **CompletableFuture**, którą mechanizm współbieżności strukturalnej bezpośrednio adresuje.

Mechanizm współbieżności strukturalnej bardzo dobrze sprawdza się w połączeniu z wirtualnymi wątkami, które stanowią jego naturalne uzupełnienie i wspólnie tworzą spójny imperatywny model współbieżności w ramach Project Loom. Zgodnie z rekomendacją twórców obu mechanizmów [10, 12], zaleca się ich łączne stosowanie w nowych projektach i systemach wymagających koordynacji zadań równoległych.

Kluczowym ograniczeniem jest jednak status współbieżności strukturalnej jako funkcji poglądowej w JDK 25. API tego mechanizmu może ulec zmianie w kolejnych wersjach platformy, co ogranicza jej stosowanie w środowiskach produkcyjnych wymagających długoterminowej stabilności. Wirtualne wątki, dostępne jako stabilna funkcja od JDK 21, nie są obciążone tym ograniczeniem.

7.5 Ograniczenia badań

Syntetyczność benchmarków JMH

Eksperymenty przeprowadzono przy użyciu syntetycznych benchmarków JMH, które z natury upraszczają warunki rzeczywistych aplikacji produkcyjnych. Benchmarkowany kod nie uwzględnia warstw typowych dla środowisk produkcyjnych, takich jak frameworki zarządzania pulą połączeń, mechanizmy cachowania czy złożona logika biznesowa, a obciążenie generowane przez narzędzie JMH jest jednorodne i deterministyczne w przeciwieństwie do dynamicznego obciążenia obserwowanego w rzeczywistych systemach [65].

Ograniczona liczba scenariuszy

W ramach pracy zdefiniowano i zbadano dwa scenariusze testowe, reprezentujące wyselekcjonowane wzorce współbieżności: integrację z bazą danych oraz agregację opartą na mechanizmie kworum. Wyniki niniejszej pracy mogą nie być w pełni miarodajne dla innych kategorii zastosowań, takich jak obciążenia obliczeniowe, przetwarzanie strumieniowe z mechanizmem backpressure, komunikacja w czasie rzeczywistym czy długotrwałe zadania wsadowe.

Wersja pogładowa mechanizmu współbieżności strukturalnej

Mechanizm współbieżności strukturalnej był dostępny w użytej wersji platformy (JDK 25) wyłącznie jako funkcja pogładowa. Oznacza to, że API tego mechanizmu może ulec zmianie w kolejnych wersjach platformy, co implikuje ryzyko dezaktualizacji wniosków z analizy implementacyjnej i weryfikacji hipotez dotyczących scenariusza $\mathcal{B}$.

Brak metryk statycznej analizy kodu

Ocena złożoności implementacyjnej przeprowadzona w podrozdziale 7.2 ma charakter wyłącznie jakościowy. Planowano przeprowadzenie ilościowej analizy kodu przy użyciu narzędzia SonarQube, jednak okazało się ono niekompatybilne z używaną wersją narzędzia Gradle. W konsekwencji analiza opierała się na subiektywnej ocenie czytelności kodu i struktury implementacji, a nie na obiektywnych metrykach takich jak złożoność cyklomatyczna czy wskaźniki spójności modułów oferowanych przez to narzędzie.

7.6 Kierunki dalszych badań

Ponowne zbadanie mechanizmu współbieżności strukturalnej

Naturalną kontynuacją niniejszych badań jest powtórzenie eksperymentów po wprowadzeniu mechanizmu współbieżności strukturalnej do oficjalnej specyfikacji platformy JVM. W momencie przeprowadzania badań mechanizm ten był dostępny wyłącznie jako funkcja pogładowa, co uniemożliwiło jego zastosowanie w pełnych warunkach produkcyjnych. Zbadanie ewentualnych zmian interfejsu oraz charakterystyki wydajnościowej finalnej wersji API pozwoliłoby na sformułowanie pełniejszych wniosków dotyczących dojrzałości tego mechanizmu.

Mechanizm wartości zakresowych

Istotnym kierunkiem dalszych badań jest analiza mechanizmu wartości zakresowych. Zbadanie jego wpływu na czytelność kodu, wydajność i poprawność zarządzania kontekstem w złożonych scenariuszach współbieżnych stanowiłoby cenne uzupełnienie analizy przeprowadzonej w niniejszej pracy, w szczególności w kontekście przekazywania danych kontekstowych, takich jak tożsamość użytkownika, kontekst transakcyjny czy dane diagnostyczne, między wirtualnymi wątkami i podzadaniami w ramach zakresu współbieżności strukturalnej.

Integracja mechanizmów Project Loom z popularnymi frameworkami

Kolejnym wartościowym kierunkiem jest weryfikacja uzyskanych wyników w warunkach rzeczywistych obciążeń produkcyjnych, z wykorzystaniem popularnych frameworków aplikacyjnych środowiska JVM, takich jak Spring Boot, Quarkus czy Micronaut. Większość ze wspomnianych frameworków oferuje dedykowane wsparcie dla wirtualnych wątków, a ich integracja z istniejącymi mechanizmami takimi jak pula połączeń HikariCP, warstwy ORM czy kontener servletów może wpływać na rzeczywistą charakterystykę wydajnościową w sposób niemożliwy do odwzorowania w syntetycznych benchmarkach.

Rozdział 8

Podsumowanie

Celem niniejszej pracy była analiza porównawcza mechanizmów oferowanych przez Project Loom, w szczególności wirtualnych wątków i współbieżności strukturalnej, z istniejącymi modelami współbieżności w środowisku JVM pod kątem wydajności i złożoności implementacyjnej. Badania przeprowadzono w ramach dwóch scenariuszy: scenariusza $\mathcal{A}$, opartego na integracji z bazą danych przy użyciu sterownika blokującego i reaktywnego oraz scenariusza $\mathcal{B}$, implementującego równoległą agregację wyników opartą na mechanizmie kworum.

Wyniki potwierdziły, że wirtualne wątki oferują znaczącą przewagę skalowalności nad wątkami platformowymi w warunkach wysokiej współbieżności operacji wejścia-wyjścia, co zostało zrealizowane w ramach scenariusza $\mathcal{A}$, osiągając na każdym poziomie współbieżności wyższą przepustowość i niższe opóźnienia. Wariant oparty na wirtualnych wątkach i blokującym sterowniku JDBC dorównał reaktywnemu podejściu. Mechanizm współbieżności strukturalnej osiągnął wydajność porównywalną z modelem reaktywnym we wszystkich zbadanych konfiguracjach scenariusza $\mathcal{B}$, jednocześnie wykazując przewagę nad wariantem opartym na natywnym podejściu, którego ograniczona skuteczność anulowania zadań negatywnie wpływa na czas odpowiedzi przy skrajnych opóźnieniach.

Jakościowa analiza implementacji wykazała, że migracja z wątków platformowych na wirtualne wymaga minimalnych zmian w kodzie, bez modyfikacji logiki biznesowej. Mechanizm współbieżności strukturalnej upraszcza implementację scenariusza równoległej agregacji z wykorzystaniem mechanizmu kworum przy zachowaniu imperatywnego modelu programowania. Model reaktywny, choć wydajny, wiąże się z wyraźnie wyższą złożonością poznawczą i trudniejszą diagnostyką błędów, jak wskazano w sekcjach 7.2.1 i 7.2.2.

Niniejsza praca posiada ograniczenia wynikające przede wszystkim z syntezy charakteru benchmarków i ograniczonej liczby zbadanych scenariuszy. Naturalnym kierunkiem kontynuacji badań jest powtórzenie eksperymentów po stabilizacji API współbieżności strukturalnej, które w momencie przeprowadzania

badania pozostawało funkcją pogładową. Warte zbadania jest również zastosowanie mechanizmu wartości zakresowych jako uzupełnienia współbieżności strukturalnej w scenariuszach wymagających przekazywania danych kontekstowych. Szerszy obraz mogłoby dostarczyć zbadanie obu mechanizmów w środowiskach frameworków produkcyjnych, takich jak Spring Boot, Quarkus czy Micronaut.

Na podstawie przeprowadzonych badań potwierdzono hipotezę główną pracy, tj. mechanizmy Project Loom umożliwiają osiągnięcie wydajności porównywalnej lub wyższej względem istniejących modeli współbieżności przy jednoczesnym zachowaniu prostszego, imperatywnego modelu programowania. Project Loom oferuje odpowiedź na problem, który przez lata skłaniał programistów do wyboru między prostotą kodu a skalowalnością systemu. Wyniki uzyskane w ramach niniejszej pracy wskazują, że wybór ten nie musi być już konieczny.

Bibliografia

- [1] Tim Lindholm i in. *The Java Virtual Machine Specification, Java SE 25 Edition*. URL: <https://docs.oracle.com/javase/specs/jvms/se25/html> (dostęp 26.05.2026).
- [2] Stack Overflow. *Stack Overflow Developer Survey 2025 – Technology*. URL: <https://survey.stackoverflow.co/2025/technology> (dostęp 26.05.2026).
- [3] Andrew S. Tanenbaum i Herbert Bos. *Modern Operating Systems*. Wydanie V. Pearson Education, 2023, s. 85–178.
- [4] Brian Goetz i in. *Java Concurrency in Practice*. Pearson Education, 2006, s. 1–9, 15–32, 113–115, 167–180, 205–220, 221–241.
- [5] Adam L. Davis. *Reactive Streams in Java: Concurrency with RxJava, Reactor, and Akka Streams*. Apress, 2019.
- [6] Jeffrey C. Mogul i Anita Borg. *The Effect of Context Switches on Cache Performance*. ACM SIGPLAN Notices, 1991, s. 75–84. URL: <https://dl.acm.org/doi/pdf/10.1145/106973.106982>.
- [7] Jonas Bonér i in. *The Reactive Manifesto*. Wersja druga. 2014. URL: <https://reactivemanifesto.org> (dostęp 11.04.2026).
- [8] Arooba Shahoor, Jooyong Yi i Dongsun Kim. *Preserving Reactiveness: Understanding and Improving the Debugging Practice of Blocking-call Bugs*. ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2024). 2024.
- [9] Ron Pressler (OpenJDK). *Project Loom: Fibers and Continuations for the Java Virtual Machine*. URL: <https://cr.openjdk.org/~rpressler/loom/Loom-Proposal.html> (dostęp 12.12.2025).
- [10] OpenJDK. *JEP 444: Virtual Threads*. URL: <https://openjdk.org/jeps/444> (dostęp 1.11.2025).
- [11] Ron Veen i David Vlijmincx. *Virtual Threads, Structured Concurrency, and Scoped Values: Explore Java’s New Threading Model*. Apress Berkeley, CA, 2024.

- [12] OpenJDK. *JEP 525: Structured Concurrency (Sixth Preview)*. URL: <https://openjdk.org/jeps/525> (dostęp 1.11.2025).
- [13] OpenJDK. *JEP 506: Scoped Values*. URL: <https://openjdk.org/jeps/506> (dostęp 1.11.2025).
- [14] Sinthujan Ponnampalam. *Evaluation of Virtual Threads and RxJava: A performance and code complexity analysis in a real-time clearing system*. Umeå University. 2025.
- [15] Elias Gustafsson i Oliver Nederlund Persson. *Comparison of Concurrency Technologies in Java*. Lund University. 2024.
- [16] Mordechai Ben-Ari. *Principles of Concurrent Programming*. Prentice-Hall International, 1982, s. 1–15, 18–27, 29–43, 51–55, 109–116.
- [17] Maurice Herlihy i Nir Shavit. *The Art of Multiprocessor Programming*. Elsevier, 2012, s. 6–10, 21–44.
- [18] Vincent Gramoli. *More than you ever wanted to know about synchronization: Synchrobench, measuring the impact of the synchronization on concurrent algorithms*. URL: <http://sydney.edu.au/engineering/it/~gramoli/doc/pubs/gramoli-synchrobench.pdf> (dostęp 27.03.2026).
- [19] Abraham Silberschatz, Peter B. Galvin i Greg Gagne. *Podstawy systemów operacyjnych, Tom I*. Wydanie X (I w WN PWN). Wydawnictwo Naukowe PWN, 2021.
- [20] Microsoft Learn. *Warunki wyścigu i zakleszczenia*. Przetłumaczone oryginalnie z języka angielskiego. URL: <https://learn.microsoft.com/pl-pl/troubleshoot/developer/visualstudio/visual-basic/language-compilers/race-conditions-deadlocks> (dostęp 28.03.2026).
- [21] Edsger W. Dijkstra. *Twenty-eight years*. Transkrypcja odręcznych notatek z 1965 roku. URL: <https://www.cs.utexas.edu/~EWD/transcriptions/EWD10xx/EWD1000.html> (dostęp 28.03.2026).
- [22] Dokumentacja klasy `java.lang.Thread` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Thread.html> (dostęp 5.04.2026).
- [23] Dokumentacja interfejsu `java.lang.Runnable` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Runnable.html> (dostęp 5.04.2026).
- [24] Jeff Friesen. *Java Threads and the Concurrency Utilities*. Apress, 2015, s. 3–49.
- [25] Doug Lea. *Concurrent Programming in Java: Design Principles and Patterns, Second Edition*. Addison Wesley, 1999, s. 5–31, 32–37.

-
- [26] Dokumentacja słowa kluczowego `synchronized` w specyfikacji języka Java. URL: <https://docs.oracle.com/javase/specs/jls/se25/html/jls-14.html#jls-14.19> (dostęp 5.04.2026).
 - [27] Dokumentacja interfejsu `java.util.concurrent.ExecutorService` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ExecutorService.html> (dostęp 5.04.2026).
 - [28] Scott L. Bain. *The Design Patterns Companion*. Project Management Institute, 2020, s. 48–49, 50–51.
 - [29] Dokumentacja klasy `java.util.concurrent.ThreadPoolExecutor` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ThreadPoolExecutor.html> (dostęp 6.04.2026).
 - [30] Dokumentacja klasy `java.lang.Runtime` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Runtime.html> (dostęp 6.04.2026).
 - [31] Strona internetowa inicjatywy Reactive Streams. URL: <https://www.reactive-streams.org> (dostęp 11.04.2026).
 - [32] Strona internetowa projektu Project Reactor. URL: <https://projectreactor.io> (dostęp 11.04.2026).
 - [33] Kod źródłowy projektu RxJava. URL: <https://github.com/ReactiveX/RxJava> (dostęp 11.04.2026).
 - [34] Dokumentacja projektu Akka Streams. URL: <https://doc.akka.io/libraries/akka-core/current/stream/index.html> (dostęp 11.04.2026).
 - [35] Oleh Dokuka i Igor Lozynskyi. *Hands-On Reactive Programming in Spring 5*. Packt Publishing, 2018, s. 6–40, 49–56, 72.
 - [36] Dokumentacja klasy `java.util.concurrent.Flow` dla JDK w wersji 9. URL: <https://docs.oracle.com/javase/9/docs/api/java/util/concurrent/Flow.html> (dostęp 12.04.2026).
 - [37] Erich Gamma i in. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995, s. 293–304.
 - [38] OpenJDK. *JEP 1: JDK Enhancement-Proposal & Roadmap Process*. URL: <https://openjdk.org/jeps/1> (dostęp 1.11.2025).
 - [39] John D.C. Little i Stephen C. Graves. *Little's Law*. URL: <https://web.archive.org/web/20100215131801/http://web.mit.edu/sgraves/www/papers/Little's%20Law-Published.pdf> (dostęp 26.01.2015).

- [40] Thomas H. Cormen i in. *Introduction to Algorithms, Third Edition*. Massachusetts Institute of Technology, 2009, s. 232–236.
- [41] Alan Bateman (OpenJDK). *Project Loom: The challenges of introducing Virtual Threads to the Java Platform*. URL: <https://cr.openjdk.org/~alanb/jvmls2023/loom-jvmls2023.pdf> (dostęp 11.03.2026).
- [42] Dokumentacja klasy `java.util.concurrent.Semaphore` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/Semaphore.html> (dostęp 6.04.2026).
- [43] Dokumentacja klasy `java.util.concurrent.StructuredTaskScope` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/StructuredTaskScope.html> (dostęp 12.04.2026).
- [44] Oracle (Dokumentacja języka Java). *The try-with-resources Statement*. URL: <https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html> (dostęp 13.04.2026).
- [45] Dokumentacja interfejsu `java.util.concurrent.StructuredTaskScope.Joiner` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/StructuredTaskScope.Joiner.html> (dostęp 12.04.2026).
- [46] Dokumentacja klasy `java.lang.ThreadLocal` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ThreadLocal.html> (dostęp 13.04.2026).
- [47] Dokumentacja klasy `java.lang.ScopedValue` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/ScopedValue.html> (dostęp 13.04.2026).
- [48] Dokumentacja korutyn w języku Kotlin. URL: <https://kotlinlang.org/docs/coroutines-overview.html> (dostęp 13.06.2026).
- [49] Rob Pike. *Go Concurrency Patterns*. 2012. URL: <https://go.dev/talks/2012/concurrency.slide> (dostęp 13.06.2026).
- [50] Joe Armstrong. *Programming Erlang: Software for a Concurrent World*. Pragmatic Bookshelf, 2007, s. 3–10.
- [51] Dokumentacja interfejsu `CoroutineScope` w języku Kotlin. URL: <https://kotlinlang.org/api/kotlinx.coroutines/kotlinx-coroutines-core/kotlinx.coroutines/-coroutine-scope> (dostęp 13.06.2026).
- [52] Dokumentacja klasy `asyncio.TaskGroup` w języku Python. 2025. URL: <https://docs.python.org/3/library/asyncio-task.html> (dostęp 13.06.2026).

-
- [53] John McCall i in. Dokumentacja mechanizmu Structured Concurrency w języku Swift. 2021. URL: <https://github.com/apple/swift-evolution/blob/main/proposals/0304-structured-concurrency.md> (dostęp 13.06.2026).
 - [54] Vishesh Pandita. *Benchmarking the Performance of Java Virtual Threads in High-Throughput Workloads*. National College of Ireland. 2024.
 - [55] Sebastian Iwanowski i Grzegorz Kozieł. *Analiza porównawcza podejścia reaktywnego i imperatywnego w tworzeniu aplikacji internetowych w języku Java*. Journal of Computer Sciences Institute. 2022.
 - [56] Yazeed Al-Dossari i Layla Al-Fahad. *Java Concurrency Demystified: Thread Pools, Completable Future, and Virtual Threads*. International Journal of Informatics and Data Science Research. 2024.
 - [57] Andy Georges, Dries Buytaert i Lieven Eeckhout. *Statistically Rigorous Java Performance Evaluation*. ACM. 2007.
 - [58] Dokumentacja API narzędzia JDBC. URL: <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc> (dostęp 10.04.2026).
 - [59] Strona internetowa projektu R2DBC. URL: <https://r2dbc.io> (dostęp 10.04.2026).
 - [60] Dokumentacja klasy `java.util.concurrent.CompletableFuture` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/CompletableFuture.html> (dostęp 7.05.2026).
 - [61] Strona internetowa bazy danych PostgreSQL. URL: <https://www.postgresql.org/about> (dostęp 10.04.2026).
 - [62] Strona internetowa narzędzia Docker. *Use containers to Build, Share and Run your applications*. URL: <https://www.docker.com/resources/what-container> (dostęp 5.05.2026).
 - [63] Kod źródłowy narzędzia Java Microbenchmark Harness (JMH). URL: <https://github.com/openjdk/jmh> (dostęp 9.04.2026).
 - [64] Dokumentacja klasy `Blackhole` z narzędzia JMH w wersji 1.37. URL: <https://javadoc.io/doc/org.openjdk.jmh/jmh-core/latest/org/openjdk/jmh/infra/Blackhole.html> (dostęp 30.12.2025).
 - [65] Dokumentacja API narzędzia Java Microbenchmark Harness (JMH) w wersji 1.37. URL: <https://javadoc.io/doc/org.openjdk.jmh/jmh-core/1.37/index.html> (dostęp 9.04.2026).

- [66] Andrzej Luszczewicz. *Statystyka ogólna*. Państwowe Wydawnictwo Ekonomiczne, 1970, s. 9–15.
- [67] Dokumentacja projektu Project Reactor w wersji 3.8.2. URL: <https://projectreactor.io/docs/core/3.8.2/api> (dostęp 25.05.2026).

Spis rysunków

2.1	Rozróżnienie procesów i wątków w systemie operacyjnym	16
2.2	Ilustracja zjawiska <i>race condition</i> na podstawie wywołania procedury <code>increaseByOne</code>	18
2.3	Wywołanie procedury <code>increaseByOne</code> z wykorzystaniem synchronizacji wątków	18
2.4	Ilustracja problemu uczujących filozofów	19
3.1	Cykl życia wirtualnych wątków w środowisku JVM	30
3.2	Schemat działania zakresu zadań w ramach klasy <code>StructuredTaskScope</code> w środowisku JVM	33
5.1	Schemat bazy danych wykorzystanej w ramach scenariusza $\mathcal{A}$	59
6.1	Wykresy kolumnowe przepustowości dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	64
6.2	Wykresy kolumnowe przepustowości dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	65
6.3	Wykresy kolumnowe przepustowości dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	66
6.4	Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	69
6.5	Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	71
6.6	Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99 dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	73

6.7	Wykresy kolumnowe średniego czasu i percentyli P50, P95, P99, P99.9 dla scenariusza $\mathcal{B}$ w zależności od konfiguracji serwisów i modelu współbieżności	76
-----	---	----

Spis tabel

5.1	Typy i parametry symulowanych serwisów wykorzystywanych w ramach scenariusza $\mathcal{B}$	53
5.2	Konfiguracja sprzętowa środowiska eksperymentalnego	58
5.3	Wersje narzędzi i bibliotek wykorzystanych w eksperymencie	58
6.1	Wartości przepustowości dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	64
6.2	Wartości przepustowości dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	65
6.3	Wartości przepustowości dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	66
6.4	Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q1 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	68
6.5	Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q2 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	70
6.6	Wartości średniego czasu i percentyli P50, P95, P99 dla zapytania Q3 w ramach scenariusza $\mathcal{A}$ w zależności od poziomu współbieżności i wariantu modelu współbieżności	72
6.7	Wartości średniego czasu i percentyli P50, P95, P99, P99.9 dla scenariusza $\mathcal{B}$ w zależności od konfiguracji serwisów i modelu współbieżności	75

Spis kodów źródłowych

5.1	Zapytanie Q1 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$	47
5.2	Zapytanie Q2 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$	47
5.3	Zapytanie Q3 w języku SQL wykorzystane w ramach scenariusza $\mathcal{A}$	47
5.4	Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+PT	48
5.5	Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie JDBC+VT	49
5.6	Uproszczona implementacja współbieżnego wykonania zapytania w ramach scenariusza $\mathcal{A}$ w wariancie R2DBC	50
5.7	Uproszczona implementacja metody wykonującej zapytanie do bazy danych przy użyciu sterownika JDBC w ramach scenariusza $\mathcal{A}$	50
5.8	Uproszczona implementacja metody wykonującej zapytanie do bazy danych przy użyciu sterownika R2DBC w ramach scenariusza $\mathcal{A}$	51
5.9	Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Legacy	54
5.10	Metoda zawierająca właściwą logikę agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Legacy	55
5.11	Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Reactive	56
5.12	Uproszczona implementacja agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Loom	57
5.13	Implementacja własnej strategii agregacji wyników w oparciu o mechanizm kworum w ramach scenariusza $\mathcal{B}$ w wariancie Loom	57